PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 1

Verteilte Systeme · Skript

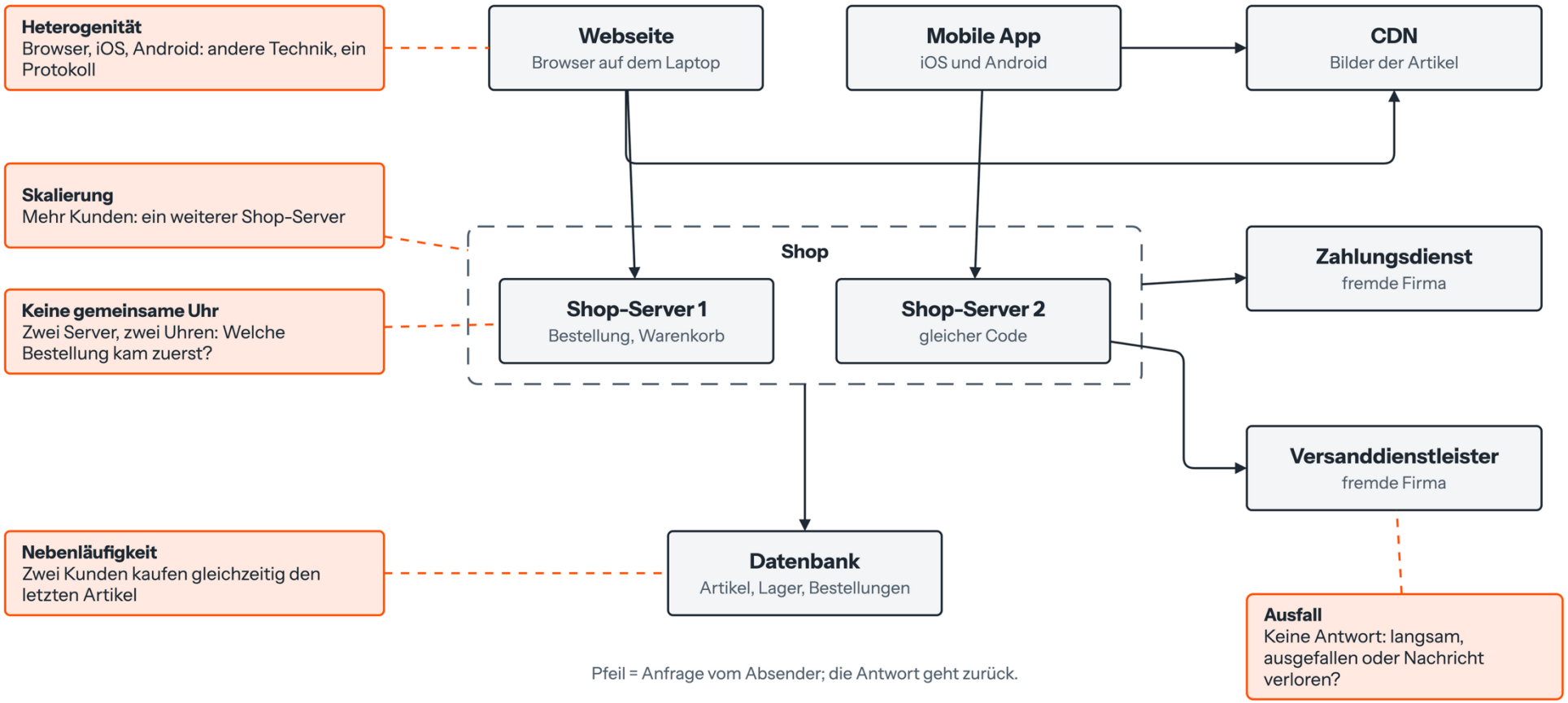
Worum es in dieser Einheit geht

Eine Web-Anwendung besteht aus vielen Teilen: Clients, Servern, einer Datenbank und Diensten anderer Firmen. Jeder Teil kann allein ausfallen. Wer die Teile kennt, weiß, was bei einem Ausfall stillsteht und was er kostet. Für die regionalen Brauereien ist das die Grundlage, um zu entscheiden, welche Teile der Beer App besonders verfügbar sein müssen.

Am Ende der Einheit kannst du:

- ein System in seine Teile zerlegen und als Skizze darstellen,
- die Folgen eines Ausfalls benennen und eine Verfügbarkeit in Ausfallzeit umrechnen.

Ein Online-Shop als verteiltes System



Webseite und Mobile App senden Anfragen an zwei Shop-Server und laden die Bilder von einem CDN. Die Server senden Anfragen an eine Datenbank, einen Zahlungsdienst und einen Versanddienstleister. Orange Karten zeigen, wo Heterogenität, Skalierung, fehlende gemeinsame Uhr, Nebenläufigkeit und Ausfall entstehen.

Teile, die nur über Nachrichten zusammenarbeiten

Ein verteiltes System besteht aus eigenständigen Rechnern oder Prozessen und erscheint seinen Nutzern als ein einziges System (van Steen und Tanenbaum). Die Teile haben keinen gemeinsamen Speicher; ein Teil weiß vom anderen nur, was eine Nachricht ihm sagt. Die fünf Herausforderungen stehen auf Folie 1.9 der Vorlesung, die Transparenzarten auf Folie 1.10.

- Bleibt eine Antwort aus, kann der Absender nicht unterscheiden: Ist der Empfänger langsam, ausgefallen oder ging die Nachricht verloren?
- Die erste der acht Fallacies of Distributed Computing lautet „The network is reliable“.

Dokumentation: https://www.distributed-systems.net/index.php/books/ds4/

Verfügbarkeit und SLA

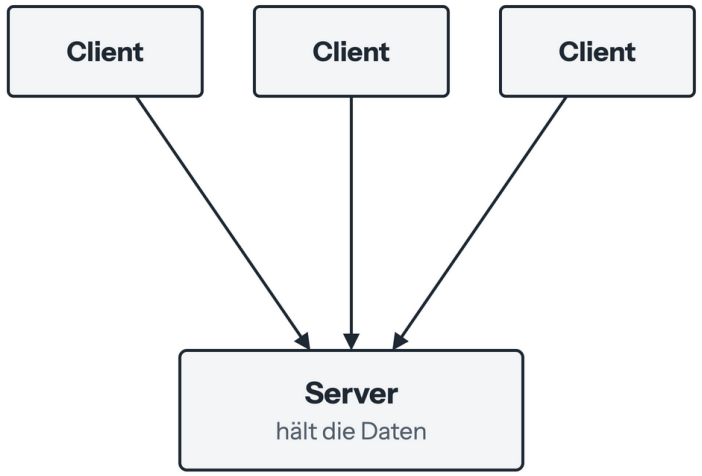
Verfügbarkeit ist der Anteil der Zeit, in der ein Dienst Anfragen beantwortet. Ein Service Level Agreement (SLA) legt fest, wie viel Ausfall erlaubt ist und was der Anbieter bei mehr Ausfall zahlt. Die Umrechnung in Ausfallzeit steht auf Folie 1.5 der Vorlesung.

- Rechnung: Minuten im Jahr mal erlaubter Ausfallanteil. Die Tabelle rechnet mit 365 Tagen, also 525 600 Minuten.
- Viele SLAs messen die Verfügbarkeit je Monat.
- Beispiel 99,9 %: 525 600 × 0,001 = 525,6 Minuten, rund 8 Stunden 46 Minuten.
- Jede weitere Neun verlangt Redundanz: zweite Server, zweites Rechenzentrum, Personal in Bereitschaft.

Dokumentation: https://sre.google/sre-book/availability-table/

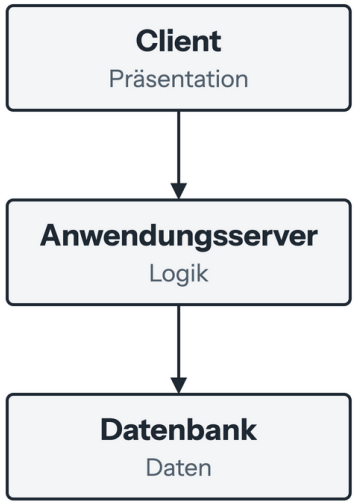
Drei Architekturmodelle

Client-Server



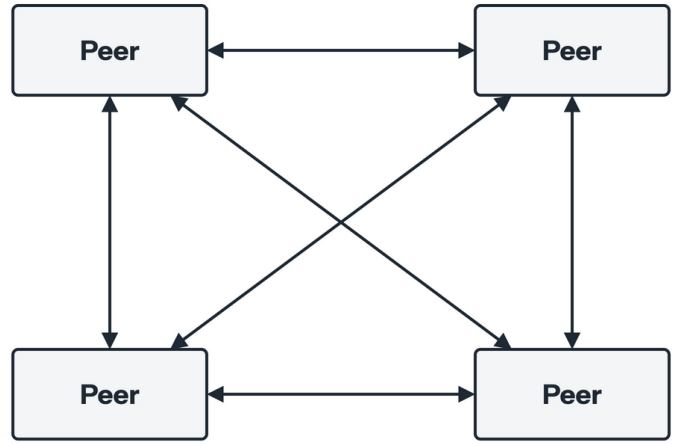
Viele Clients fragen einen Server, der Server antwortet. Pfeil = Anfrage.

Mehrschichten



Jede Schicht läuft auf eigenen Rechnern und skaliert einzeln.

Peer-to-Peer



Jeder Peer ist Client und Server. Es gibt keinen zentralen Teil.

Links Client-Server: drei Clients senden Nachrichten an einen Server. Mitte Mehrschichten: Client, Anwendungsserver und Datenbank übereinander. Rechts Peer-to-Peer: vier Peers, jeder mit jedem verbunden.

Architekturmodelle im Web

Stärken und Schwächen der drei Modelle stehen auf Folie 1.13 der Vorlesung. Das Web folgt dem Client-Server-Modell. Die Beer App folgt der Mehrschichtenarchitektur (Multi-Tier) mit drei Schichten (Folien 1.14 und 1.15 der Vorlesung).

- Die Präsentation zeigt an und nimmt Eingaben entgegen. Die Logik prüft Regeln. Die Datenschicht speichert.
- BitTorrent ist ein Beispiel für Peer-to-Peer. Damit könnte der Betreiber die Stammdaten nicht zentral pflegen.
- Rollen von Client und Server und den Weg einer Anfrage behandelt Einheit 2.

Dokumentation: https://www.distributed-systems.net/index.php/books/ds4/

Beispiel: Teile einer Seite im Network-Tab von Chrome

Stelle	Wirkung	Beispiel an https://de.wikipedia.org/wiki/Verteiltes_System
Tab Network mit Disable cache	zeigt jeden Request, ohne Kopien aus dem Cache	37 bis 42 Requests
Spalte Domain (Rechtsklick auf den Spaltenkopf)	zeigt den Host jedes Requests	fünf Hosts, 29 Requests von de.wikipedia.org
More tools → Request conditions	blockiert Requests nach einem URL-Muster	*://*.wikimedia.org/*: Text vollständig, Logo-Dateien fehlen
Enable blocking and throttling	schaltet die Bedingungen ein; Status (blocked:devtools)	*://de.wikipedia.org/w/load.php*: Text ohne CSS

Aufgabe 1.1 · Ein System in seine Teile zerlegen

ZIEL

Du zerlegst ein bekanntes System in seine Teile und stellst es als Skizze dar.

AUSGANGSLAGE

Partnerarbeit auf Papier oder in draw.io (https://app.diagrams.net). Fremde Dienste sind zum Beispiel Zahlung, CDN oder Login.

SCHRITTE

1. Wähle einen Streaming-Dienst, einen Online-Shop oder einen Messenger.
2. Zeichne mindestens zwei Clients und einen Server.
3. Zeichne den Ort der Daten und einen fremden Dienst.
4. Zeichne jede Verbindung als Pfeil vom Absender der Anfrage.
5. Beschrifte jeden Pfeil mit der Nachricht.

FERTIG, WENN

- Die Skizze enthält Clients, Server, Ort der Daten und einen fremden Dienst.
- Jeder Pfeil hat eine Richtung und eine Beschriftung.

Aufgabe 1.2 · Den Ausfall von drei Teilen durchspielen

ZIEL

Du benennst für drei Teile deiner Skizze, was bei ihrem Ausfall weiterläuft.

AUSGANGSLAGE

Aufgabe 1.1 ist gelöst.

SCHRITTE

1. Wähle drei Teile: einen Client, einen Server und einen fremden Dienst.
2. Trage je Teil in die Vorlage ein, was weiterläuft und was stillsteht.
3. Trage ein, ob die Nutzer den Ausfall bemerken.
4. Markiere in der Skizze den Teil, dessen Ausfall die meisten Nachrichten scheitern lässt.

FERTIG, WENN

- Die Vorlage hat drei ausgefüllte Zeilen.
- Ein Teil der Skizze ist markiert, und du kannst die Wahl mit den Pfeilen begründen.

Vorlage zu Aufgabe 1.2

Teil fällt aus	Was läuft weiter?	Was steht still?	Bemerken es die Nutzer?
Client:			
Server:			
fremder Dienst:			

Zusatz 1.3 · Die Verfügbarkeit einer Kette berechnen

ZIEL

Du berechnest, wie verfügbar die Beer App ist, wenn zwei Teile in Reihe arbeiten.

AUSGANGSLAGE

Eine Anfrage gelingt nur, wenn Beer API und Datenbank beide verfügbar sind. Die Beer API hat 99,9 % Verfügbarkeit, die Datenbank 99,95 %. Die Ausfälle sind voneinander unabhängig.

SCHRITTE

1. Berechne die Verfügbarkeit der Kette in Prozent.
2. Rechne sie in Ausfallzeit pro Jahr um.

FERTIG, WENN

- Du hast die Verfügbarkeit in Prozent mit zwei Nachkommastellen und die Ausfallzeit in Stunden und Minuten.
- Du begründest, warum die Kette unter jedem Teil liegt.

Hinweise zu Aufgabe 1.1

- Stufe 1: Denk an einen Tag, an dem du das System nutzt. Welche Geräte, welche Dinge passieren?
- Stufe 2: Ein Kasten „Server“ reicht selten. Woher kommen Videos oder Bilder, wohin geht die Zahlung?
- Stufe 3: Zeichne für jede Handlung einen Pfeil vom Gerät, das die erste Nachricht schickt, und schreibe die Handlung daran.

Hinweise zu Aufgabe 1.2

- Stufe 1: Ausfall heißt: Der Teil antwortet nicht mehr. Die anderen Teile laufen weiter.
- Stufe 2: Es scheitern die Pfeile zum ausgefallenen Teil und die Pfeile, die er selbst schickt.
- Stufe 3: Zähle je Teil die Pfeile, die bei seinem Ausfall scheitern. Vergleiche die Zahlen der Teile.

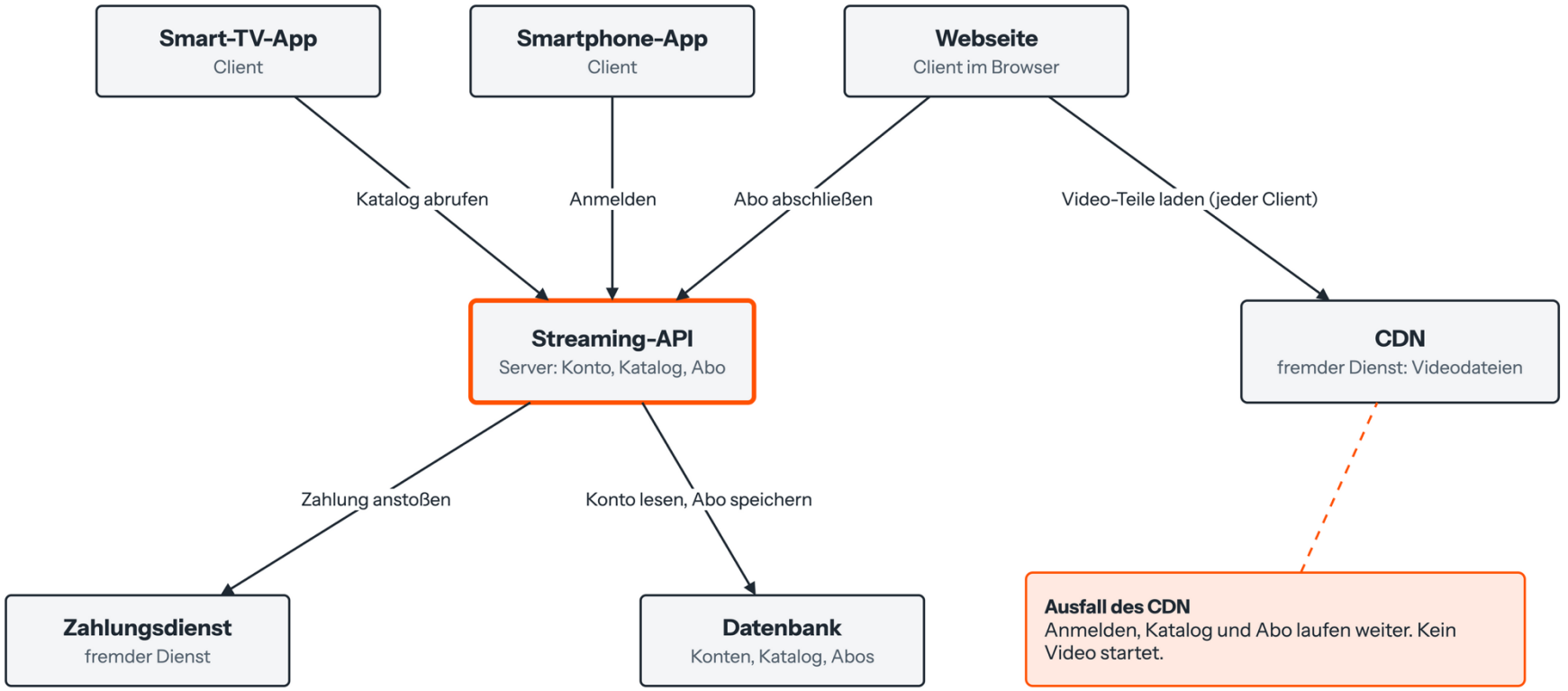
Hinweise zu Zusatz 1.3

- Stufe 1: Die Kette ist nur verfügbar, wenn beide Teile gleichzeitig verfügbar sind.
- Stufe 2: Bei unabhängigen Teilen ist die Wahrscheinlichkeit, dass beide verfügbar sind, ein Produkt.
- Stufe 3: Setze beide Werte als Dezimalzahl ein. Die Umrechnung in Ausfallzeit steht in Skript 1.5.

Typische Fehler

Symptom	Ursache
Die Skizze hat nur einen Kasten „Server“.	Der Weg von Videos, Zahlung und Daten ist nicht verfolgt.
Die Pfeile haben keine Richtung.	Es ist nicht festgelegt, wer die erste Nachricht schickt.
In der Vorlage steht dreimal „alles fällt aus“.	Die Folge ist nicht entlang der Pfeile verfolgt.
Zusatz 1.3 ergibt 99,925 %.	Die Verfügbarkeiten sind gemittelt statt multipliziert.

Musterskizze: ein Streaming-Dienst



Pfeil = Anfrage vom Absender; die Antwort geht zurück. Orange: Teil, dessen Ausfall die meisten Nachrichten scheitern lässt.

Drei Clients, Smart-TV-App, Smartphone-App und Webseite, senden Anfragen an die Streaming-API. Die API liest und schreibt die Datenbank und stößt den Zahlungsdienst an. Jeder Client lädt Video-Teile vom CDN. Pfeile zeigen die Anfrage vom Absender. Fällt das CDN aus, laufen Anmelden, Katalog und Abo weiter, aber kein Video startet. Orange markiert: die Streaming-API, deren Ausfall die meisten Nachrichten scheitern lässt.

Musterlösung zu Aufgabe 1.2 für den Streaming-Dienst

Teil fällt aus	Was läuft weiter?	Was steht still?	Bemerken es die Nutzer?
Client: Smart-TV-App	Smartphone-App und Webseite	nur das Fernsehgerät	nur, wer am Fernseher schaut
Server: Streaming-API	Videos, die schon laufen, kommen weiter vom CDN	Anmelden, Katalog, Abo für alle Clients	ja, alle
fremder Dienst: CDN	Anmelden, Katalog, Abo	jedes neue Video	ja, beim nächsten Video

Lösung zu Zusatz 1.3

- Verfügbarkeit der Kette: 0,999 × 0,9995 = 0,9985005, also 99,85 %.
- Ausfallanteil: 0,15 % von 525 600 Minuten, rund 788 Minuten.
- Ausfallzeit: rund 13 Stunden 8 Minuten im Jahr, statt 8 Stunden 46 Minuten für die Beer API allein.
- Jeder Ausfall eines Teils legt die Kette still. Deshalb liegt die Kette unter jedem Teil.

Das solltest du jetzt können

- ✓ Ein verteiltes System nach van Steen und Tanenbaum erklären
- ✓ Fünf Herausforderungen verteilter Systeme an einem Beispiel nennen
- ✓ Client-Server, Mehrschichten und Peer-to-Peer unterscheiden
- ✓ Ein System skizzieren und die Folge eines Ausfalls benennen
- ✓ Eine Verfügbarkeit in Ausfallzeit pro Jahr umrechnen

Als Nächstes: das Client-Server-Modell und der Browser – was beim Aufruf einer URL passiert.

WEB-PROGRAMMIERUNG · EINHEIT 2

Client-Server-Modell und der Browser · Skript

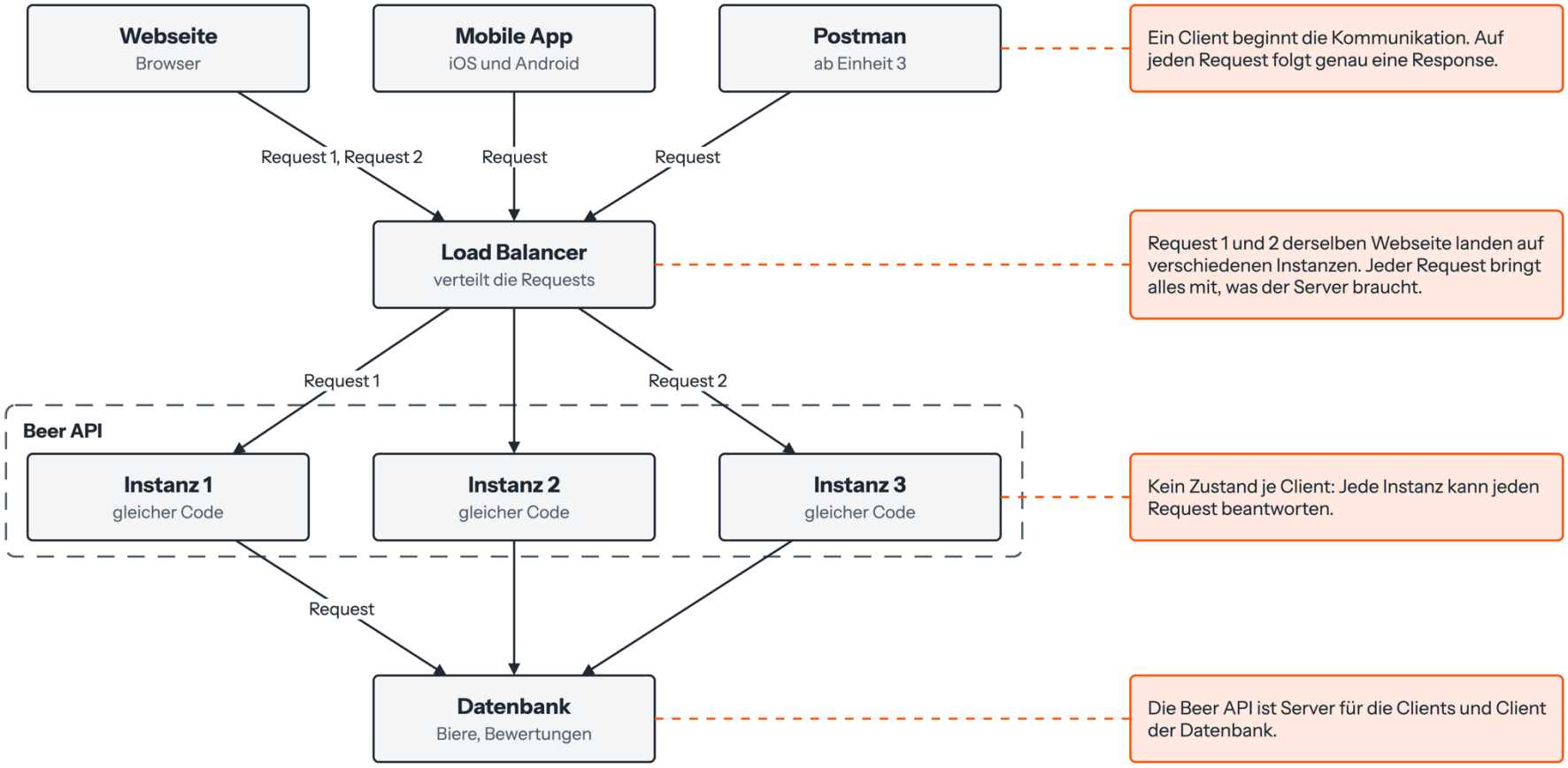
Worum es in dieser Einheit geht

Eine Web-Anwendung besteht aus Clients und Servern, die über Requests und Responses zusammenarbeiten. Wer die Rollen kennt, sieht, welche Teile die regionalen Brauereien für die Beer App selbst bauen und welche sie einkaufen. Der Network-Tab in Chrome zeigt jeden Request einer Seite. An ihm erkennst du die Schritte beim Aufruf einer URL.

Am Ende der Einheit kannst du:

- Client und Server einer Verbindung zuordnen und erklären, warum zustandslose Server skalieren,
- die Requests einer Seite im Network-Tab messen und den Schritten einer Anfrage zuordnen.

Zustandslose Server hinter einem Load Balancer



Webseite, Mobile App und Postman senden Requests an einen Load Balancer. Er verteilt sie auf drei Instanzen der Beer API mit gleichem Code; Request 1 und 2 der Webseite landen auf Instanz 1 und Instanz 3. Alle Instanzen fragen dieselbe Datenbank.

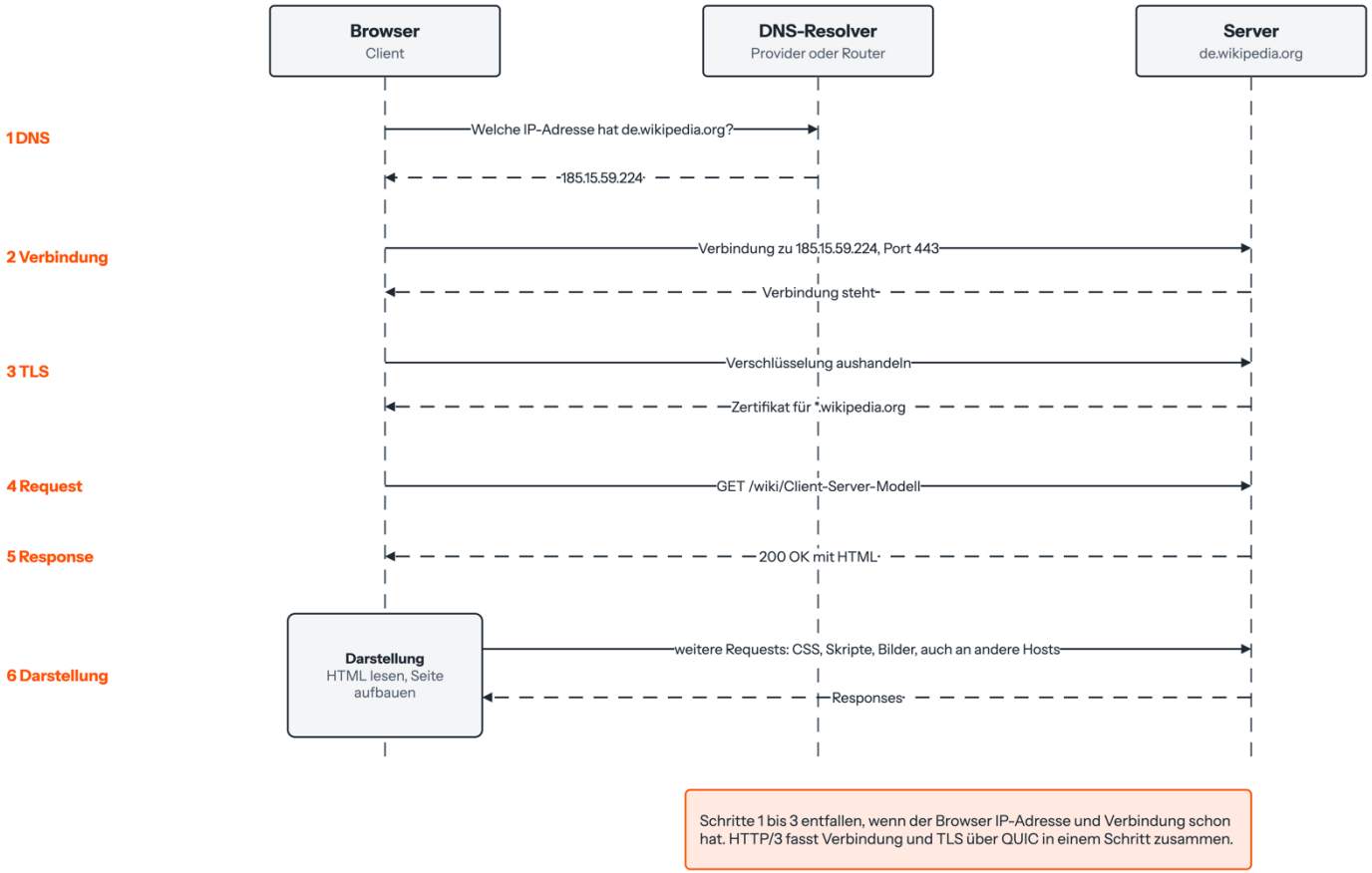
Rollen, Zustand und Make or Buy

Die Rollen Client und Server stehen auf Folie 2.5 der Vorlesung, die Entscheidung Make or Buy für die Beer App auf Folie 2.4. Das Bild zeigt, warum ein zustandsloser Server skaliert.

- Eine Instanz ist ein laufendes Exemplar desselben Programms. Der Load Balancer verteilt die Requests auf die Instanzen.
- Merkt sich eine Instanz etwas über einen Client, fehlt es, sobald der nächste Request auf einer anderen Instanz landet.
- Den Nachweis einer Anmeldung bringt der Client bei jedem Request mit, als Cookie (Einheit 3) oder Token (Einheit 13).
- Ein gekaufter Dienst übernimmt den Betrieb. Verantwortlich für die Daten bleibt der Betreiber (DSGVO Art. 24 und 28).

Dokumentation: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm#sec_5_1_3

Der Aufruf einer URL als Sequenz



Sequenz zwischen Browser, DNS-Resolver und dem Server de.wikipedia.org: 1 DNS übersetzt den Namen in 185.15.59.224, 2 Verbindung zu Port 443, 3 TLS mit Zertifikat für *.wikipedia.org, 4 Request GET /wiki/Client-Server-Modell, 5 Response 200 OK mit HTML, 6 Darstellung mit weiteren Requests für CSS, Skripte und Bilder, auch an andere Hosts.

Die sechs Schritte und der Network-Tab

Folie 2.9 der Vorlesung ordnet die Schritte den Einträgen im Timing-Tab zu, Folie 2.11 erklärt die Spalten des Network-Tabs. Die DevTools öffnest du mit F12 (macOS: Cmd + Option + I).

- DNS: Der Browser fragt seinen Cache, das Betriebssystem und dann den Resolver. nslookup de.wikipedia.org ergab 185.15.59.224.
- Verbindung: Port 443 gilt für https, Port 80 für http.
- TLS: Mit dem Zertifikat bestätigt eine Zertifizierungsstelle, dass der Server zum Namen gehört.
- Die Filter Doc, CSS, JS, Img und Fetch/XHR fassen Types zusammen. Bilder stehen in Type mit Format, etwa png.

Dokumentation: https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/How_browsers_work

Beispiel: der Network-Tab an https://de.wikipedia.org/wiki/Client-Server-Modell

Stelle in Chrome	Wirkung	Beispiel
Preserve log und Disable cache	behalten alle Requests, auch über Umleitungen, und laden alles vom Server	39 bis 46 Requests von fünf Hosts
Tab Timing eines Requests	zeigt die Schritte 1 bis 5 mit ihrer Dauer	DNS Lookup 8 ms, Initial connection 45 ms, davon SSL 24 ms
Filter Fetch/XHR	zeigt nur Requests, die ein Skript auslöst	Maus auf dem Link „Server“: Fetch nach /api/rest_v1/page/summary/Server, Response JSON
Spalte Size beim Neuladen	zeigt, ob die Datei aus dem Cache kam	22 von 25 Requests aus (memory cache) oder (disk cache)
View site information links in der Adresszeile	zeigt das Zertifikat	*.wikipedia.org, Let's Encrypt, gültig bis 2. Januar 2027

Aufgabe 2.1 · Drei Seiten im Network-Tab vermessen

ZIEL

Du misst, aus welchen Requests drei Seiten bestehen.

AUSGANGSLAGE

Partnerarbeit mit Chrome. Zwei URLs stehen in der Vorlage. Die dritte ist die Startseite eines Online-Shops, bei dem du bestellt hast, ohne Anmeldung.

SCHRITTE

1. Trage je Seite deine Schätzung der Requests ein.
2. Öffne je Seite einen neuen Tab mit DevTools, Tab Network.
3. Schalte Preserve log und Disable cache ein.
4. Lade die Seite, ohne zu scrollen.
5. Trage deine Messung in die Vorlage ein.

FERTIG, WENN

- Die Vorlage hat für drei Seiten Schätzung und Messung.
- Jede Zeile nennt den ersten Request mit Type und Initiator.

Vorlage zu Aufgabe 2.1

Seite	Requests geschätzt / gemessen	Drei häufigste Types nach Filter (Doc, CSS, JS, Img)	Statuscodes außer 200	Erster Request: Type, Initiator
https://de.wikipedia.org/wiki/Hypertext_Transfer_Protocol	/			
https://github.com/nestjs/nest	/			
Online-Shop: https://	/			

Aufgabe 2.2 · Das Timing zweier Requests vergleichen

ZIEL

Du vergleichst das Timing zweier Requests an denselben Host.

AUSGANGSLAGE

Aufgabe 2.1 ist gelöst. Du misst an https://github.com/nestjs/nest.

SCHRITTE

1. Öffne ein neues Incognito-Fenster (Strg + Umschalt + N, macOS: Cmd + Shift + N).
2. Öffne die DevTools und schalte Disable cache ein.
3. Lade die Seite und öffne beim ersten Request den Tab Timing.
4. Öffne den Tab Timing beim Fetch-Request latest-commit.
5. Trage beide Werte je Eintrag in die Vorlage ein.

FERTIG, WENN

- Die Vorlage hat für beide Requests je Eintrag einen Wert oder einen Strich.
- Du erklärst, warum dem späteren Request Einträge fehlen.

Vorlage zu Aufgabe 2.2

Eintrag im Timing-Tab	Schritt	Erster Request: nestjs/nest	Späterer Request: latest-commit
DNS Lookup	1 DNS		
Initial connection	2 Verbindung		
SSL	3 TLS		
Request sent	4 Request		
Waiting for server response	5 Response		

Zusatz 2.3 · Daten-Requests finden und Make or Buy zuordnen

ZIEL

Du findest die Daten-Requests des Online-Shops und ordnest ihre Hosts nach Make or Buy.

AUSGANGSLAGE

Aufgabe 2.1 ist gelöst. Du arbeitest mit der Seite des Online-Shops.

SCHRITTE

1. Lade die Seite ohne Anmeldung und wähle den Filter Fetch/XHR.
2. Wähle drei Requests an verschiedene Hosts.
3. Lies je Request im Tab Preview, was die Response enthält.
4. Trage ein, wem der Host gehört und ob der Shop diesen Teil eingekauft hat.

FERTIG, WENN

- Die Vorlage hat drei Zeilen mit Host, Inhalt und Entscheidung.
- Für jeden gekauften Teil nennst du die Aufgabe des Anbieters.

Vorlage zu Zusatz 2.3

Nr.	Host	Gehört zu	Inhalt der Response	Gebaut oder gekauft? Aufgabe des Anbieters
1				
2				
3				

Hinweise zu Aufgabe 2.1

- Stufe 1: Die Zahl steht in der Statusleiste unten, Type und Statuscode in den gleichnamigen Spalten.
- Stufe 2: Ein Filter wie Img oder JS zeigt nur seine Zeilen. Die Statusleiste zählt dann „x / y requests“.
- Stufe 3: Sortiere nach Status, um Codes außer 200 zu finden. Sortiere danach nach Waterfall; der erste Request steht oben.

Hinweise zu Aufgabe 2.2

- Stufe 1: Der Tab Timing gehört zu genau einem Request. Er erscheint nach einem Klick auf den Namen der Zeile.
- Stufe 2: latest-commit findest du mit dem Filter Fetch/XHR. Ein Eintrag, der fehlt, erscheint im Tab Timing gar nicht.
- Stufe 3: Vergleiche die fehlenden Einträge mit der Karte unten im Bild auf Skript 2.5.

Hinweise zu Zusatz 2.3

- Stufe 1: Der Host steht im Tab Headers in der Request URL hinter https://.
- Stufe 2: Öffne die Startseite des Hosts im Browser. Sie nennt meist die Firma und ihr Angebot.
- Stufe 3: Der Host einer anderen Firma ist ein eingekaufter Teil. Ordne seine Response einer Aufgabe zu, etwa Statistik.

Typische Fehler

Symptom	Ursache
Die Zahl der Requests ändert sich bei jedem Laden stark.	Die Seite lädt beim Scrollen nach, oder Disable cache ist aus.
Die Zahl ist viel höher als geschätzt.	Preserve log ist an, und mehrere Seiten liegen im selben Tab.
Der Network-Tab zeigt nur „Recording network activity…“.	Die DevTools waren beim Laden geschlossen. Lade die Seite neu.
Die Liste zeigt nur wenige Requests.	Ein Filter wie Fetch/XHR ist noch aktiv.
Der Online-Shop lädt keine fremden Hosts.	Das Cookie-Banner ist noch offen; viele Shops laden fremde Dienste erst nach der Auswahl.

Messwerte zum Vergleich vom 25. Oktober 2026 mit Chrome 155

Seite	Requests	Häufigste Types	Statuscodes außer 200	Erster Request
https://de.wikipedia.org/wiki/Hypertext_Transfer_Protocol	42 bis 45	Img 28, JS 9 bis 11, CSS 3	302, in einem Lauf 204	document, Initiator Other
https://github.com/nestjs/nest	232	JS 133, Img 59, CSS 24	204 zweimal, 404	document, Initiator Other

Messwerte zu den Aufgaben 2.1 bis 2.3

- Erster Request an github.com: DNS Lookup 5 bis 15 ms, Initial connection 32 ms, davon SSL 17 bis 18 ms.
- latest-commit: keine Einträge DNS Lookup, Initial connection und SSL; Waiting for server response 22 bis 28 ms.
- Der spätere Request nutzt die Verbindung des ersten. Die Schritte 1 bis 3 entfallen.
- Am Beispiel GitHub: Alle neun Fetch-Requests gehen an github.com selbst, also an keinen fremden Dienst.
- Die 302 bei Wikipedia ist ein Skript, das die Anmeldung bei auth.wikimedia.org prüft.
- Die 204 sind Requests vom Type ping für die Statistik der Seite.

Das solltest du jetzt können

- ✓ Client und Server einer Verbindung zuordnen, auch wenn ein Server Client ist
- ✓ Erklären, warum zustandslose Server hinter einem Load Balancer skalieren
- ✓ Die sechs Schritte beim Aufruf einer URL nennen
- ✓ Im Network-Tab ersten Request, Timing, Initiator und Daten-Requests finden
- ✓ Make or Buy für einen Teil eines Systems begründen

Als Nächstes: HTTP – Aufbau von Request und Response, Methoden, Statuscodes und Header.

WEB-PROGRAMMIERUNG · EINHEIT 3

HTTP · Skript

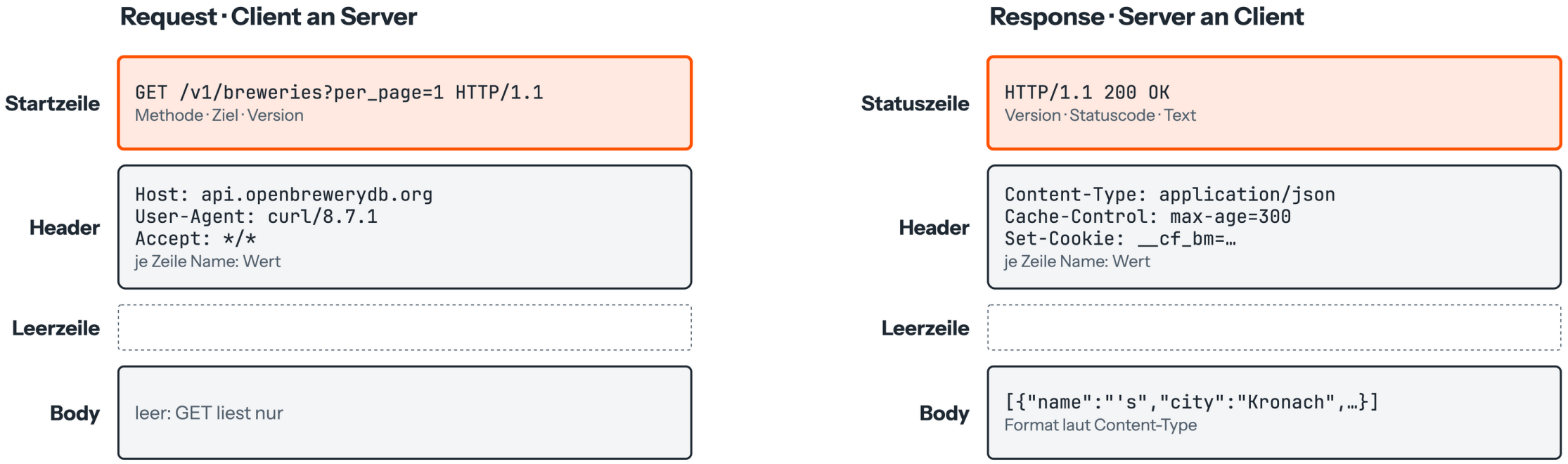
Worum es in dieser Einheit geht

Jede Anfrage an eine API und jede Antwort ist eine HTTP-Nachricht mit festem Aufbau. Wer Statuscode, Header und Body lesen kann, erkennt, ob eine API funktioniert und wer einen Fehler beheben muss. Für die regionalen Brauereien ist das die Grundlage, um die Beer API zu betreiben. Geübt wird an einem Capture the Flag: Ein Server stellt Rätsel, die nur löst, wer seine Antworten genau liest.

Am Ende der Einheit kannst du:

- eine HTTP-Nachricht in ihre Teile zerlegen und Statuscodes und Header deuten,
- aus einer Antwort die nächste Anfrage ableiten und mit Postman oder curl senden.

Request und Response haben denselben Aufbau



Eine HTTP-Nachricht besteht aus Startzeile, Headern, einer Leerzeile und dem Body. Beim Request nennt die Startzeile die Methode, bei der Response den Statuscode.

RFC 9110 und RFC 9112 legen HTTP fest

RFC 9110 beschreibt die Bedeutung von Methoden, Statuscodes und Headern, RFC 9112 die Nachricht als Text. HTTP/2 und HTTP/3 übertragen dieselben Teile binär; curl, Postman und die DevTools zeigen sie trotzdem als Text. Deshalb sehen die Header in curl oft anders aus als auf den Folien:

- HTTP/2 schreibt alle Namen von Headern klein. Groß- und Kleinschreibung zählt im Namen nicht.
- Die Statuszeile von HTTP/2 hat keinen Text, nur den Code: HTTP/2 200.

Dokumentation: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages

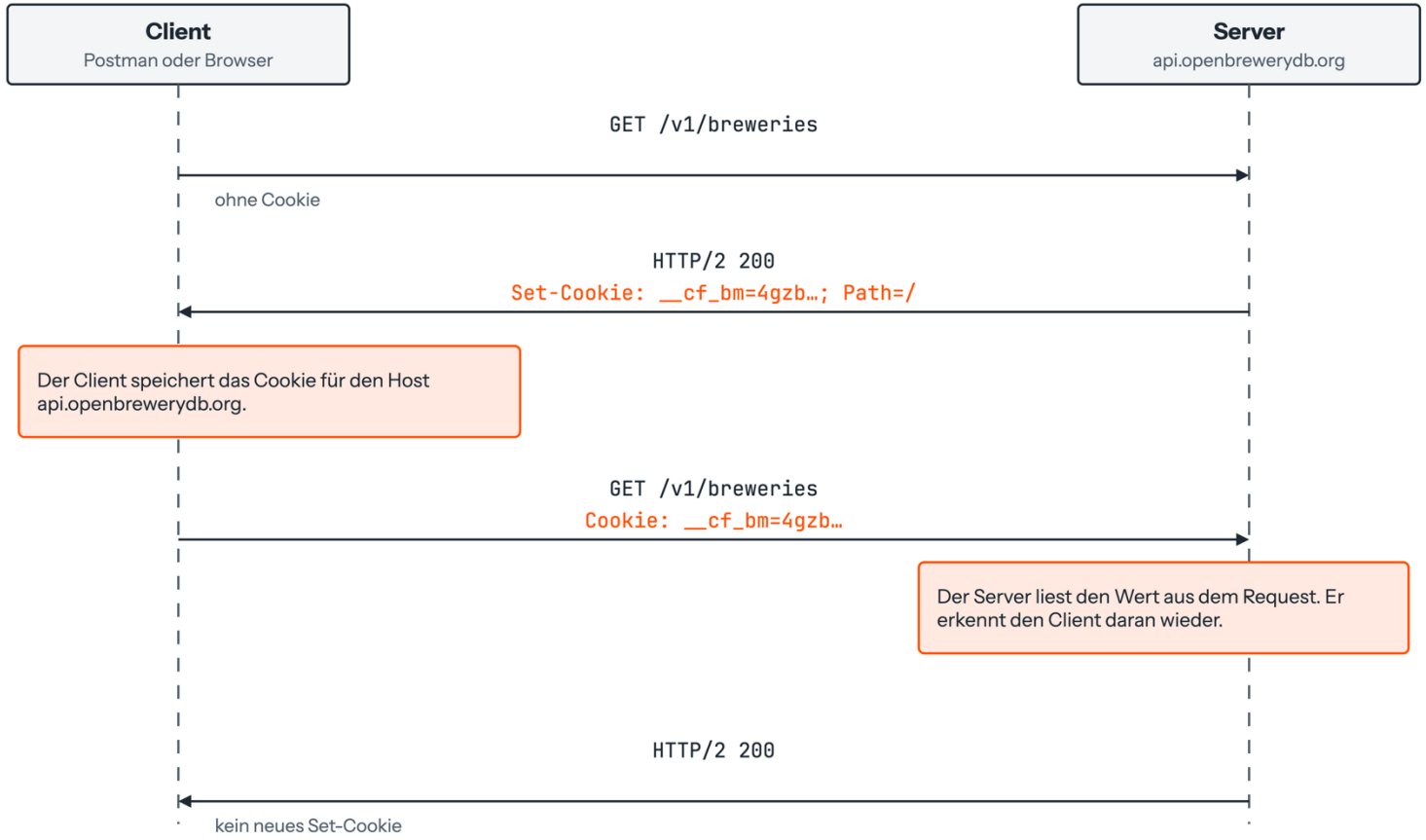
Erlaubte Methoden und Umleitungen

Methoden, Statuscodes und Header-Gruppen mit ihrer Bedeutung stehen auf den Folien 3.7 bis 3.11 der Vorlesung, die Methode HEAD auf Folie 3.18.

- Die erlaubten Methoden einer Ressource nennt der Header Allow, in der Antwort auf OPTIONS und in jeder Antwort 405.
- Bei einer Umleitung (3xx) nennt der Header Location das Ziel. Der Client stellt dafür eine neue Anfrage.
- Jede Antwort einer Kette von Umleitungen hat eigene Header und einen eigenen Body.
- Browser und Postman folgen einer Umleitung automatisch und zeigen nur die letzte Antwort. curl folgt nur mit der Option -L.

Dokumentation: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

Cookies bringen Zustand in ein zustandsloses Protokoll



Der Server setzt ein Cookie mit Set-Cookie. Der Client speichert es und schickt es bei jeder weiteren Anfrage an denselben Host im Header Cookie mit, solange Path und Expires des Cookies passen.

Cookies im Client

Ein Cookie besteht aus Name und Wert, zum Beispiel sprache=de. Attribute im Header Set-Cookie legen fest, wofür der Client es mitschickt: Path begrenzt es auf einen Pfad, Expires auf eine Zeit. HttpOnly verbirgt es vor JavaScript im Browser.

- Browser und Postman speichern Cookies und senden sie automatisch mit.
- In Postman zeigt der Link Cookies unter dem Button Send die gespeicherten Cookies; dort lassen sie sich bearbeiten.
- curl speichert Cookies nur auf Anweisung (Option -c). Ein Cookie sendest du mit der Option -b.

Dokumentation: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies

Beispiel: ein Request in Postman

Die Live-Demo der Vorlesung sendet GET https://api.openbrewerydb.org/v1/breweries?per_page=3. Postman zeigt die Teile der Nachricht an fünf Stellen:

1. Über dem Body stehen Statuscode, Dauer und Größe der Antwort.
2. Der Tab Headers der Response listet alle Header der Antwort.
3. Die Console im Footer zeigt den Request als Text, mit allen Headern, die Postman selbst setzt.
4. Der Link Cookies unter dem Button Send zeigt gespeicherte Cookies.
5. Der Tab Settings einer Anfrage enthält Einstellungen wie das Folgen von Umleitungen.

Dokumentation: https://learning.postman.com/docs/sending-requests/requests/

curl: fünf Optionen für das Capture the Flag

Option	Wirkung	Beispiel
-i	zeigt Statuszeile und Header der Response vor dem Body	curl -i http://localhost:3001/ctf
-v	zeigt zusätzlich den Request; > geht zum Server, < kommt zurück	curl -v http://localhost:3001/ctf
-X	sendet eine andere Methode als GET	curl -i -X POST http://localhost:3001/ctf
-H	sendet einen Header	curl -i -H "Accept: text/html" http://localhost:3001/ctf
-b	sendet ein Cookie	curl -i -b "sprache=de" http://localhost:3001/ctf

Den CTF-Server starten

Den Ordner ctf-server bekommst du mit den Unterlagen dieser Einheit. Du brauchst Node.js 24.

1. Öffne ein Terminal im Ordner ctf-server.
2. Installiere die Pakete mit npm install (nur beim ersten Start).
3. Starte den Server mit npm start.
4. Sende in Postman GET http://localhost:3001/ctf. Der Server antwortet mit Level 0.

Achtung: In der Windows PowerShell 5 ist curl ein anderer Befehl. Schreibe dort curl.exe.

Ein Neustart des Servers setzt alle Level zurück. Euer Team-Token gilt weiter.

So funktioniert das Capture the Flag

Jedes Level ist ein Rätsel aus mehreren Anfragen. Die Antworten nennen, was fehlt: im Statuscode, in einem Header oder im Body. Ein gelöstes Level liefert ein Flag und im Feld next den Pfad und einen Hinweis zum nächsten Level, nie die fertige Anfrage.

- Ein Flag hat die Form FLAG{…} und endet auf vier Zeichen, die zu deinem Team gehören.
- Level 0 legt dein Team an. Ab Level 1 sendet jede Anfrage das Team-Token; wie, nennt die Antwort.
- Die Übung ist Partnerarbeit an einem Rechner. Eine Person sendet, die andere liest und notiert.
- Die Rollen wechseln je Level.

Aufgabe 3.1 · Anmelden und Siegel

ZIEL

Du legst ein Team an und öffnest das versiegelte Lager (Level 0 und 1).

AUSGANGSLAGE

Der CTF-Server läuft. Du hast Postman oder ein Terminal mit curl.

SCHRITTE

1. Sende GET http://localhost:3001/ctf und lies Statuscode, Header und Body.
2. Hole das Flag von Level 0.
3. Folge dem Feld next zu Level 1.
4. Hole das Flag von Level 1.

FERTIG, WENN

- Du hast die Flags von Level 0 und 1 notiert.
- GET /ctf/1/lager antwortet mit 200.

Aufgabe 3.2 · Umzug und Ausweis

ZIEL

Du findest eine umgezogene Brauerei und kommst ins Sudhaus (Level 2 und 3).

AUSGANGSLAGE

Aufgabe 3.1 ist gelöst. Du hast dein Team-Token.

SCHRITTE

1. Folge dem Feld next zu Level 2.
2. Hole das Flag von Level 2.
3. Folge dem Feld next zu Level 3.
4. Hole das Flag von Level 3.

FERTIG, WENN

- Du hast die Flags von Level 2 und 3 notiert.
- GET /ctf/3/sudhaus antwortet mit 200.

Zusatz 3.3 · Feierabend

ZIEL

Du kommst in den Keller, der gerade geschlossen hat (Level 4). Der Zusatz ist keine Voraussetzung für Einheit 4.

AUSGANGSLAGE

Aufgabe 3.2 ist gelöst.

SCHRITTE

1. Folge dem Feld next zu Level 4.
2. Hole das Flag von Level 4.

FERTIG, WENN

- Du hast das Flag von Level 4 notiert.
- GET /ctf/4 antwortet mit 200.

Vorlage: deine Flags

Level	Flag	Welcher Teil der Antwort hat weitergeholfen?
0 Anmelden		
1 Siegel		
2 Umzug		
3 Ausweis		
4 Feierabend (Zusatz)		

Selbstkontrolle mit Postman

Die Lösung steht in der Collection Web-Programmierung aus den Unterlagen (web-programmierung.postman_collection.json). Importiere die Datei einmal in Postman mit **Import**; sie enthält einen Ordner je Einheit. Starte den Ordner der Einheit 3 erst nach der Übung mit **Run**. Er enthält einen Ordner je Aufgabe, zum Beispiel „Aufgabe 3.1 – Anmelden und Siegel“, und eine Anfrage je Schritt.

Jeder Lauf legt ein neues Team an. Die Tests prüfen deshalb nur die Form FLAG{…}, nicht den Wert; deine Flags unterscheiden sich von denen der Collection. Lege deine Anfragen je Level neben die der Lösung.

Hinweise zu Aufgabe 3.1

- Stufe 1, Level 0: Der Body der ersten Antwort nennt den ersten Schritt.
- Stufe 1, Level 1: Der Server sagt dir, welche Methoden eine Ressource erlaubt.
- Stufe 2, Level 0: Lies das Feld start. Die Antwort mit 201 enthält im Body das Feld token.
- Stufe 2, Level 1: Frage mit OPTIONS nach dem Header Allow. Auch eine Antwort 405 enthält ihn.
- Stufe 3, Level 0: Sende POST /ctf/teams ohne Body. Sende das Token ab jetzt im Header X-Team.
- Stufe 3, Level 1: Sende OPTIONS und DELETE an /ctf/1/siegel. Lies danach GET /ctf/1/lager.

Hinweise zu Aufgabe 3.2

- Stufe 1, Level 2: Die Antwort, die du siehst, ist nicht die erste.
- Stufe 1, Level 3: Der Server erkennt dich an etwas, das dein Client mitschickt.
- Stufe 2, Level 2: Achte auf die Statuscodes 301, 302 und 307 und den Header Location.
- Stufe 2, Level 3: Achte auf Set-Cookie und Cookie. Die Antwort 403 nennt die nötige Rolle.
- Stufe 3, Level 2: Schalte Automatically follow redirects im Tab Settings aus. Folge jeder Location einzeln.
- Stufe 3, Level 3: Hole den Ausweis unter /ctf/3. Ändere das Cookie ausweis auf brauer und sende erneut.

Hinweise zu Zusatz 3.3

- Stufe 1, Level 4: Der Server sagt dir, wann du wiederkommen sollst.
- Stufe 2, Level 4: Lies den Header Retry-After. Die Statuscodes 503 und 429 haben verschiedene Gründe.
- Stufe 3, Level 4: Warte nach GET /ctf/4 die Sekunden aus Retry-After ab, ohne zu fragen. Sende dann erneut.

Typische Fehler

Symptom	Ursache
400 mit „Der Header X-Team fehlt“	Das Token fehlt in dieser Anfrage oder steht als Query-Parameter in der URL.
400 mit „Das Team-Token im Header X-Team ist ungültig“	Das Token ist unvollständig kopiert oder steht in Anführungszeichen.
Level 2 antwortet „Hier gibt es kein Flag“	Der Client ist den Weiterleitungen automatisch gefolgt; das Flag stand in einer Antwort davor.
Level 3 antwortet 403 „Ohne Ausweis kommst du nicht ins Sudhaus“	Der Request enthält das Cookie nicht; curl sendet es nur mit der Option -b.
Level 4 antwortet 429 „Zu viele Anfragen“	Dreimal vor Ablauf der Wartezeit gefragt; die Wartezeit beginnt von vorn.

Das solltest du jetzt können

- ✓ Request, Response und URL in ihre Teile zerlegen
- ✓ Zu einer Absicht die Methode wählen und als sicher oder idempotent einordnen
- ✓ An einem Statuscode erkennen, wer handeln muss
- ✓ Aus Statuscode und Headern einer Antwort die nächste Anfrage ableiten
- ✓ Dieselbe Anfrage in Postman und mit curl senden

Als Nächstes: JSON und Datenaustausch – wie Client und Server sich auf ein Format einigen.

WEB-PROGRAMMIERUNG · EINHEIT 4

JSON und Datenaustausch · Skript

Worum es in dieser Einheit geht

JSON ist das Format, in dem die Beer API ab Einheit 5 Daten liefert und annimmt. Ein festes Format ist ein Vertrag mit allen Clients: Benennt der Betreiber ein Feld um, zeigen die Webseiten der Brauereien leere Stellen. Du übst an vier Level des Capture the Flag aus Einheit 3 und am JSON von Brauerei, Bieren und Bewertungen.

Am Ende der Einheit kannst du:

- gültiges JSON nach dem Datenmodell der Beer App schreiben,
- mit Accept, Content-Type, PATCH und ETag Daten anfordern, senden und ändern.

Ein Objekt im Programm wird für die Leitung zu Text



Der Server macht aus einem Objekt mit JSON.stringify einen JSON-Text. Der Text steht im Body mit Content-Type application/json. Der Client macht daraus mit JSON.parse wieder ein Objekt.

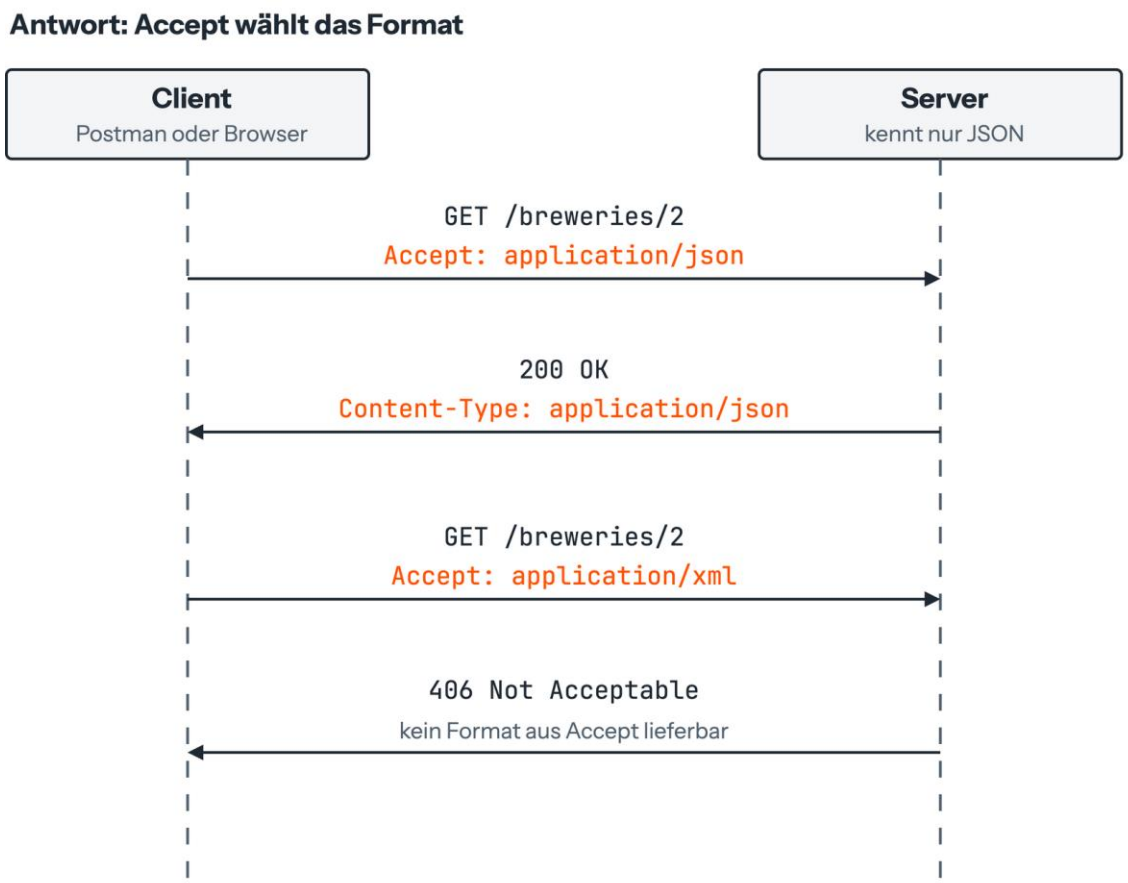
JSON-Text, Datentypen und Konventionen

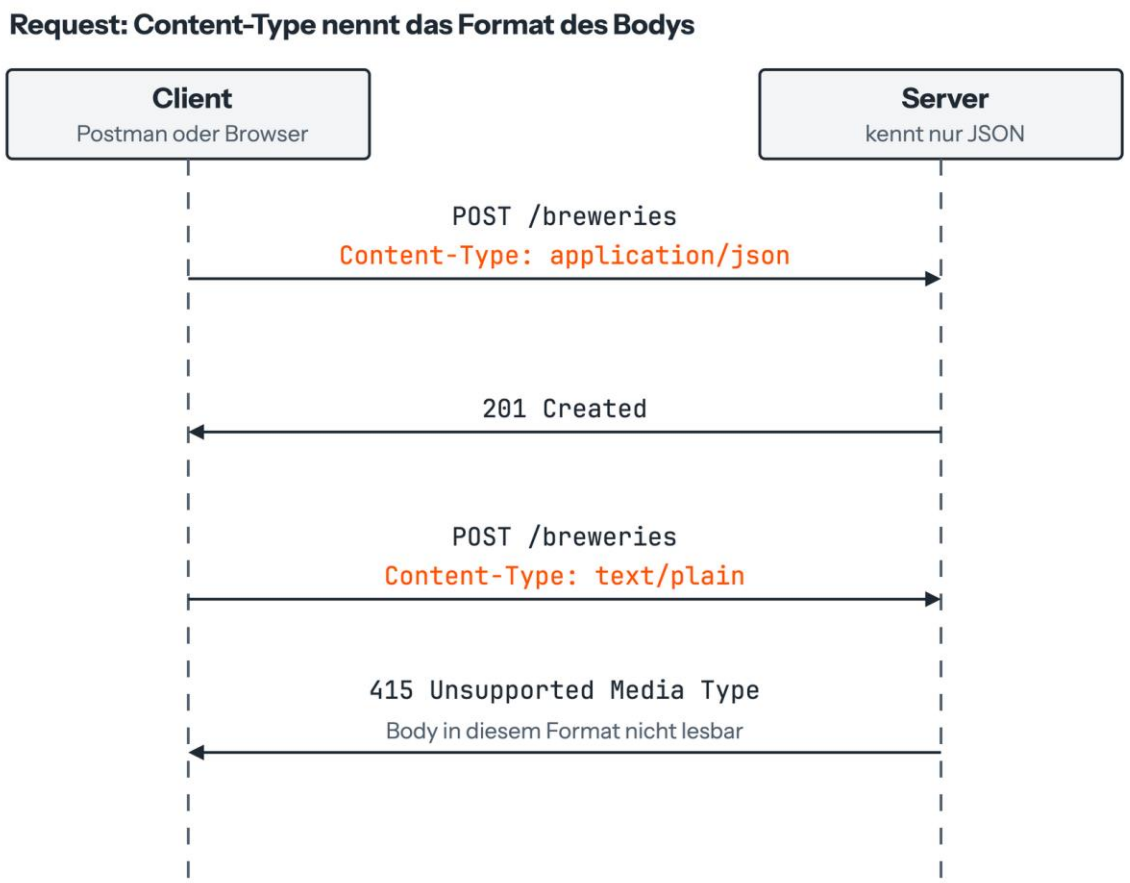
Datentypen, die Unterschiede zu JavaScript und die Konventionen der Beer API stehen auf den Folien 4.5, 4.6 und 4.8 der Vorlesung.

- Was JSON.stringify aus einem Datum und aus undefined macht, zeigt Folie 4.20 der Vorlesung.
- JSON.parse macht aus einem Zeitpunkt kein Datum; das Umrechnen in die eigene Zeitzone übernimmt der Client.
- Ein Zeitpunkt nach ISO 8601 besteht aus Datum, dem Trenner T, Uhrzeit und Zeitzone. Z steht für UTC.
- 10:15 Uhr deutscher Winterzeit (UTC+1) ist 09:15 Uhr UTC: 2026-10-27T09:15:00.000Z.

Dokumentation: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON

Accept wählt das Format der Antwort, Content-Type nennt das Format des Bodys





Links fragt der Client mit Accept application/json und bekommt 200 mit Content-Type application/json; mit Accept application/xml bekommt er 406 Not Acceptable. Rechts sendet der Client einen Body mit Content-Type application/json und bekommt 201; mit Content-Type text/plain bekommt er 415 Unsupported Media Type.

Das Format mit Accept und Content-Type aushandeln

Accept kann mehrere Formate mit Gewicht nennen, zum Beispiel Accept: text/html;q=0.5, application/json;q=0.9. Der Server wählt das Format mit dem höchsten Gewicht, das er liefern kann. Dass er statt 406 auch sein Standardformat liefern darf, zeigt Folie 4.16 der Vorlesung.

- */* heißt: jedes Format. Das senden Postman und curl, wenn du nichts anderes angibst.
- Vary: Accept in der Antwort zeigt, dass sie vom Header Accept abhängt.
- Postman setzt Content-Type selbst, wenn du im Tab Body raw und JSON wählst.
- curl setzt bei -d ohne eigenen Header Content-Type: application/x-www-form-urlencoded.

Dokumentation: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Content_negotiation

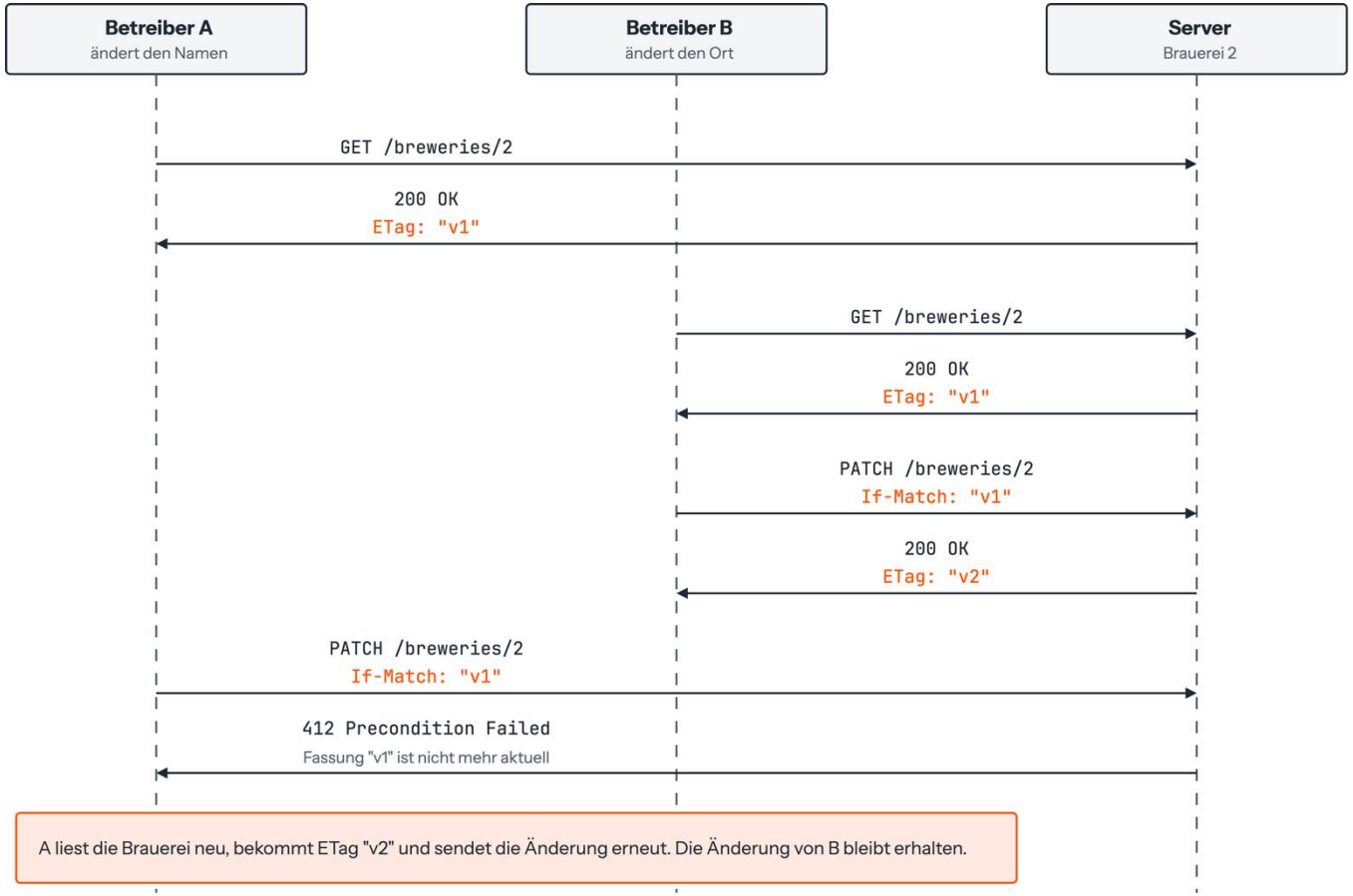
PUT, PATCH und JSON Merge Patch

Folie 4.10 der Vorlesung zeigt PUT und PATCH an Bier 2. PUT sendet die vollständige neue Darstellung; was im Body fehlt, fehlt danach in der Ressource. PATCH im Format JSON Merge Patch ändert nur die gesendeten Felder.

- Ein Feld, das im Body fehlt, bleibt unverändert.
- Ein Feld mit dem Wert null wird entfernt.
- Der Media Type ist application/merge-patch+json. Viele Server nehmen auch application/json an.
- Der Header Accept-Patch nennt die Formate, die ein Server für PATCH versteht, zum Beispiel in der Antwort auf OPTIONS.

Dokumentation: https://www.rfc-editor.org/rfc/rfc7396

ETag und If-Match verhindern, dass eine Änderung eine andere überschreibt



Betreiber A und Betreiber B lesen Brauerei 2 mit ETag v1. B ändert sie mit If-Match v1 und bekommt ETag v2. A sendet seine Änderung mit If-Match v1 und bekommt 412 Precondition Failed. A liest neu und sendet die Änderung erneut; die Änderung von B bleibt erhalten.

Fassungen mit ETag erkennen

Ein ETag kennzeichnet eine Fassung einer Ressource. Der Server vergibt mit jeder Änderung einen neuen ETag. Die Statuscodes dazu stehen auf Folie 4.12 der Vorlesung.

- Sende den ETag unverändert zurück, mit den Anführungszeichen: If-Match: "v2".
- If-Match macht eine Änderung zur Bedingung. Ist die Fassung veraltet, antwortet der Server mit 412.
- Verlangt der Server die Bedingung, antwortet er ohne If-Match mit 428.
- If-None-Match fragt beim Lesen, ob sich die Ressource geändert hat. Wenn nicht, antwortet der Server mit 304 ohne Body.

Dokumentation: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/ETag

Beispiel: JSON lesen und senden

Die Live-Demo der Vorlesung liest eine Brauerei der Open Brewery DB und sendet einen JSON-Body an Postman Echo. Postman und Chrome zeigen die Teile an diesen Stellen:

1. Im Body formatiert die Ansicht Pretty das JSON. Raw zeigt den Text, wie der Server ihn gesendet hat.
2. Im Tab Headers des Requests setzt du Accept, If-Match und andere Header.
3. Im Tab Body wählst du raw und JSON. Postman markiert Syntaxfehler rot.
4. POST https://postman-echo.com/post zeigt im Feld json, wie der Server deinen Body gelesen hat.
5. Die Console von Chrome öffnest du mit Strg + Shift + J, macOS: Cmd + Option + J.

Dokumentation: https://learning.postman.com/docs/sending-requests/requests/

curl: fünf Optionen für JSON

Option	Wirkung	Beispiel
-H "Content-Type: …"	nennt das Format des Bodys	curl -i -H "Content-Type: application/json" -d '{"name": "Augustiner"}' https://postman-echo.com/post
-d '…'	sendet einen Body; ohne -X ist die Methode POST	curl -i -d 'name=Augustiner' https://postman-echo.com/post
-d @datei	liest den Body aus einer Datei	curl -i -H "Content-Type: application/json" -d @brauerei.json https://postman-echo.com/post
-X PATCH	sendet PATCH statt POST	curl -i -X PATCH -H "Content-Type: application/json" -d '{}' https://postman-echo.com/patch
-H 'If-Match: "…"'	sendet einen ETag mit Anführungszeichen	curl -i -H 'If-Match: "v2"' https://postman-echo.com/get

Das Capture the Flag fortsetzen

Einheit 4 nutzt den CTF-Server aus Einheit 3. Starte ihn im Ordner ctf-server mit npm start.

- Dein Team-Token aus Einheit 3 gilt weiter. Ein Neustart setzt Kisten und Rezept zurück; Token und Flags bleiben gültig.
- Level 5 beginnt mit GET http://localhost:3001/ctf/5. Level 4 ist dafür keine Voraussetzung.
- Jede Anfrage sendet das Team-Token im Header X-Team.
- Hebe das Feld token aus der Antwort von Level 8 auf. Level 10 in Einheit 13 braucht es.

Achtung: Unter Windows laufen die Beispiele mit '…' in cmd.exe nicht, und die PowerShell 5 entfernt Anführungszeichen aus JSON für curl.exe. Sende den Body dort mit -d @datei oder arbeite mit Postman.

Aufgabe 4.1 · Lieferschein

ZIEL

Du holst den Lieferschein in der Darstellung, die das Flag enthält (Level 5).

AUSGANGSLAGE

Der CTF-Server läuft. Du hast dein Team-Token.

SCHRITTE

1. Sende GET http://localhost:3001/ctf/5 mit dem Header X-Team.
2. Lies Statuscode, Header und Body der Antwort.
3. Fordere den Lieferschein in anderen Formaten an.
4. Hole das Flag von Level 5.

FERTIG, WENN

- Du hast das Flag von Level 5 notiert.
- Die Antwort mit dem Flag nennt im Feld next den Weg zu Level 6.

Aufgabe 4.2 · Kiste anlegen und füllen

ZIEL

Du legst im Lager eine Kiste an und füllst sie auf 24 Flaschen, ohne ihr Etikett zu verlieren (Level 6 und 7).

AUSGANGSLAGE

Aufgabe 4.1 ist gelöst.

SCHRITTE

1. Folge dem Feld next zu Level 6 und lies die Beschreibung der Kiste.
2. Lege eine Kiste an und hole das Flag von Level 6.
3. Folge dem Feld next zu Level 7.
4. Fülle die Kiste und hole das Flag von Level 7.

FERTIG, WENN

- Du hast die Flags von Level 6 und 7 notiert.
- Deine Kiste hat 24 Flaschen und ihr ursprüngliches Etikett.

Aufgabe 4.3 · Rezept

ZIEL

Du änderst ein Rezept, an dem gleichzeitig der Braumeister arbeitet, ohne seine Änderung zu überschreiben (Level 8).

AUSGANGSLAGE

Aufgabe 4.2 ist gelöst.

SCHRITTE

1. Folge dem Feld next zu Level 8 und lies den Auftrag.
2. Lies das Rezept.
3. Ändere die Stammwürze so, wie Level 8 es verlangt.
4. Hole das Flag von Level 8 und hebe das Feld token auf.

FERTIG, WENN

- Du hast das Flag und das Token von Level 8 notiert.
- Das Rezept enthält deine Stammwürze und die Änderung des Braumeisters.

Aufgabe 4.4 · JSON für die Beer App

ZIEL

Du schreibst Brauerei, Biere und Bewertungen als JSON für die Beer API.

AUSGANGSLAGE

VS Code und ein Terminal. Datenmodell und Testdaten: Skript 4.17 bis 4.20.

SCHRITTE

1. Lege einen Ordner beer-api-entwurf an und öffne ihn in VS Code.
2. Lege brauerei.json mit der Brauerei 3 an.
3. Lege biere.json mit den vier Bieren an; eines mit eigener Beschreibung.
4. Lege bewertungen.json mit den zwei Bewertungen an.
5. Prüfe jede Datei im Terminal mit node -p "require('./biere.json')".

FERTIG, WENN

- node -p "require('./biere.json').length" gibt 4 aus.
- Jeder Schlüssel und jeder Typ stimmt mit dem Datenmodell überein.

Datenmodell zu Aufgabe 4.4: Brauerei und Bier

Alle Schlüssel sind englisch in camelCase. Jede id ist eine ganze Zahl, die der Server vergibt.

- Eine Brauerei hat die Pflichtfelder id, name, city und country. Der Name ist nicht leer, Ort und Land sind Strings.
- Brauerei 3 ist das Augustiner-Bräu in München, Deutschland.
- Ein Bier hat die Pflichtfelder id, name, style, abv und breweryId. Der Name ist ein String und nicht leer.
- style ist helles, pils, spezial, weizen oder dunkel. abv ist der Alkoholgehalt in Volumenprozent als Zahl.
- breweryId ist die id der Brauerei. description ist eine Kurzbeschreibung als String und kein Pflichtfeld.

Regel: Ein Bier gehört zu genau einer Brauerei. Eine Brauerei hat beliebig viele Biere.

Datenmodell zu Aufgabe 4.4: Bewertung

Eine Bewertung (Rating) gehört zu einem Bier und zu einem User.

- id: ganze Zahl, vergibt der Server.
- beerId: id des bewerteten Biers. userId: id des Users, der bewertet.
- score: Sterne als ganze Zahl von 1 bis 5.
- comment: Freitext als String, kein Pflichtfeld. Ohne Kommentar fehlt das Feld.
- createdAt: Zeitpunkt als String nach ISO 8601 in UTC. Den Wert setzt der Server.

Regel: Ein User bewertet jedes Bier höchstens einmal.

Testdaten zu Aufgabe 4.4: die vier Biere

id	name	style	abv	breweryId
1	Nattheimer Spezial	spezial	5.4	1
2	Wildbräu Grafing Helles	helles	5.0	2
3	Augustiner Lagerbier Hell	helles	5.2	3
4	Farny Kristall-Weizen	weizen	5.3	4

Testdaten zu Aufgabe 4.4: zwei Bewertungen für das Nattheimer Spezial

id	userId	score	comment	Zeitpunkt
1	1	4	Süffig	27. Oktober 2026, 09:15 Uhr UTC
2	2	5	–	27. Oktober 2026, 18:30 Uhr deutscher Zeit

Zusatz 4.5 · Frische

ZIEL

Du zeigst dem Server, welche Fassung einer Antwort du schon kennst (Level 9). Der Zusatz ist keine Voraussetzung für Einheit 5.

AUSGANGSLAGE

Aufgabe 4.3 ist gelöst.

SCHRITTE

1. Folge dem Feld next zu Level 9.
2. Hole das Flag von Level 9.

FERTIG, WENN

- Du hast das Flag von Level 9 notiert.
- GET /ctf/9 antwortet dir mit 304.

Vorlage: deine Flags

Level	Flag	Welcher Teil der Antwort hat weitergeholfen?
5 Lieferschein		
6 Kiste anlegen		
7 Kiste füllen		
8 Rezept		
9 Frische (Zusatz)		

Selbstkontrolle mit Postman

Die Lösung steht im Ordner der Einheit 4 in der Collection Web-Programmierung. Starte ihn erst nach der Übung mit **Run**. Er enthält einen Ordner je Aufgabe, zum Beispiel „Aufgabe 4.2 – Kiste anlegen und füllen“, und eine Anfrage je Schritt. Der Ordner zu Level 10 gehört zu Einheit 13.

Jeder Lauf legt ein neues Team an. Die Tests prüfen deshalb nur die Form FLAG{…}, nicht den Wert. Aufgabe 4.4 prüfst du mit node wie unter „Fertig, wenn“. Ab Einheit 5 vergleichst du biere.json mit der Antwort der Beer API.

Hinweise zu Aufgabe 4.1

- Stufe 1, Level 5: Dieselbe Ressource gibt es in mehreren Formaten.
- Stufe 2, Level 5: Der Header Accept wählt das Format. Eine Antwort 406 nennt die verfügbaren Formate.
- Stufe 3, Level 5: Sende GET /ctf/5 mit Accept: application/xml. Fordere danach jedes Format aus der Antwort einzeln an.

Hinweise zu Aufgabe 4.2

- Stufe 1, Level 6: Die Antwort auf das Anlegen sagt, wo die neue Ressource liegt.
- Stufe 1, Level 7: PUT und PATCH ändern eine Ressource auf verschiedene Art.
- Stufe 2, Level 6: Achte auf Content-Type im Request und auf den Header Location der Antwort 201.
- Stufe 2, Level 7: PUT ersetzt die ganze Kiste. PATCH ändert nur die gesendeten Felder.
- Stufe 3, Level 6: Sende POST /ctf/6/kisten mit JSON aus sorte und flaschen. Sende GET an den Pfad aus Location.
- Stufe 3, Level 7: Sende PATCH mit dem Feld flaschen an deine Kiste. Fehlt das Etikett, lege eine neue an.

Hinweise zu Aufgabe 4.3

- Stufe 1, Level 8: Zwei ändern dieselbe Ressource. Der Server schützt die neuere Fassung.
- Stufe 2, Level 8: Achte auf den Header ETag der Antwort und auf die Statuscodes 428 und 412.
- Stufe 3, Level 8: Sende PATCH mit dem gelesenen ETag in If-Match. Nach 412 liest du neu.

Hinweise zu Aufgabe 4.4

- Stufe 1: Jeder Schlüssel und jeder Typ steht im Datenmodell. Vergleiche deine Dateien Feld für Feld damit.
- Stufe 2: Achte auf die Typen von abv und breweryId, auf createdAt und auf Felder ohne Wert.
- Stufe 3: Schreibe Zahlen ohne Anführungszeichen, Zeitpunkte in UTC mit Z am Ende. Lass description weg, wo keine angegeben ist.

Hinweise zu Zusatz 4.5

- Stufe 1, Level 9: Der Server muss eine unveränderte Antwort nicht noch einmal senden.
- Stufe 2, Level 9: Achte auf den Header ETag und auf If-None-Match. Eine Antwort 304 hat keinen Body.
- Stufe 3, Level 9: Sende GET /ctf/9 erneut mit If-None-Match und dem ETag mit Anführungszeichen. Lies die Header der Antwort.

Typische Fehler im Capture the Flag

Symptom	Ursache
415 mit „Der Server versteht hier nur application/json. Der Header Content-Type nennt das Format des Bodys.“	Der Body ist in Postman als Text gewählt, oder curl sendet -d ohne Header Content-Type.
400 mit „Expected double-quoted property name in JSON at position …“	Im Body steht ein Komma nach dem letzten Feld.
400 mit „flaschen ist eine ganze Zahl von 1 bis 24.“	Die Zahl steht in Anführungszeichen und ist damit ein String.
Level 7 antwortet „Das Etikett der Kiste fehlt oder ist ein anderes. Ein PUT ersetzt die ganze Kiste. …“	Ein PUT ohne etikett hat die Kiste ersetzt.
412 mit „Das Rezept hat sich geändert, seit du es gelesen hast. Lies es neu.“	Der ETag stammt aus einer älteren Antwort.

Typische Fehler im JSON der Beer App

Symptom	Ursache
node meldet „Expected double-quoted property name in JSON at position …“	Nach dem letzten Feld eines Objekts steht ein Komma.
node meldet „Unexpected token ']', … is not valid JSON“	Nach dem letzten Element des Arrays steht ein Komma.
node meldet „Expected property name or '}' in JSON at position …“	Ein Schlüssel steht in einfachen oder ohne Anführungszeichen.
In biere.json steht "abv": "5.2"	Der Alkoholgehalt ist ein String statt einer Zahl.
In bewertungen.json steht "createdAt": "27.10.2026 18:30"	Der Zeitpunkt folgt nicht ISO 8601 und nennt keine Zeitzone.

Das solltest du jetzt können

- ✓ Gültiges JSON mit den sechs Datentypen schreiben und Syntaxfehler finden
- ✓ Eine Ressource der Beer App nach dem Datenmodell als JSON darstellen
- ✓ null und ein fehlendes Feld begründet unterscheiden
- ✓ Mit Accept und Content-Type arbeiten und 406 und 415 deuten
- ✓ Mit PATCH einzelne Felder ändern und mit ETag eine veraltete Fassung erkennen

Als Nächstes: API, Backend und das erste NestJS-Projekt – die Beer API liefert ihre ersten Biere als JSON.

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 5

API, Backend und das erste NestJS-Projekt · Skript

Worum es in dieser Einheit geht

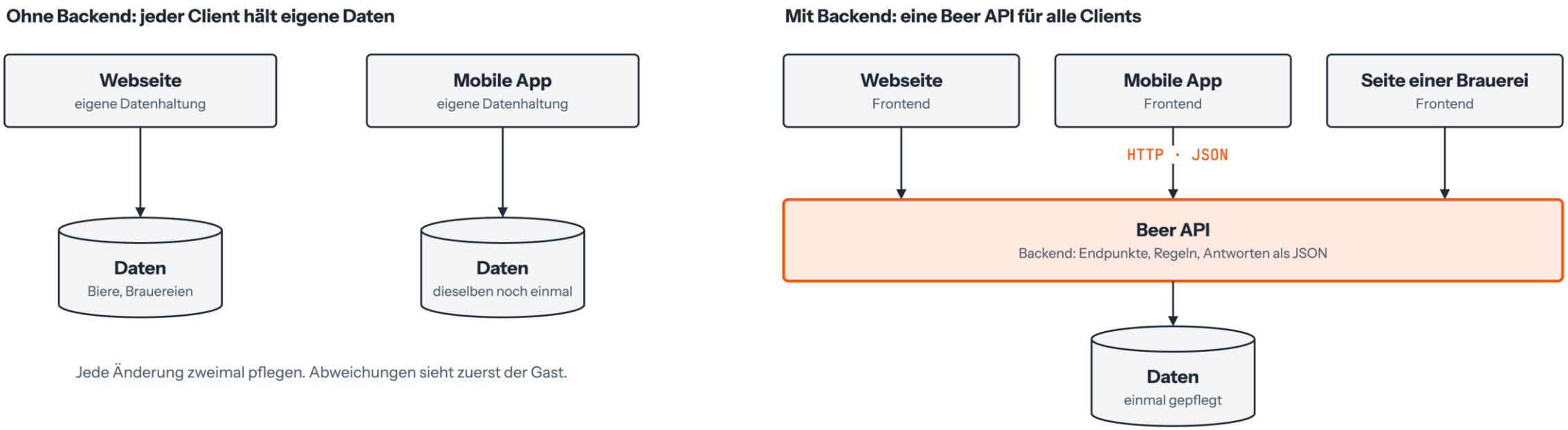
Webseite und App der regionalen Brauereien sollen dieselben Biere zeigen. Dafür bekommen beide ein gemeinsames Backend: die Beer API. In dieser Einheit startest du das Basisprojekt mit NestJS, übernimmst die Endpunkte für die Brauereien und baust die Endpunkte für die Biere. Die Aufgaben setzen diese Story um:

Als Gast möchte ich alle Biere und ein einzelnes Bier abrufen, damit ich sehe, welche Biere es gibt.

Am Ende der Einheit kannst du:

- den Weg einer Anfrage durch NestJS erklären,
- Endpunkte mit einem Controller bauen und einen Fehler systematisch suchen.

Webseite und App fragen dieselbe Beer API



Ohne Backend halten Webseite und Mobile App je eigene Daten, jede Änderung wird zweimal gepflegt. Mit Backend fragen Webseite, Mobile App und die Seite einer Brauerei dieselbe Beer API über HTTP mit JSON; die Daten liegen einmal hinter der API.

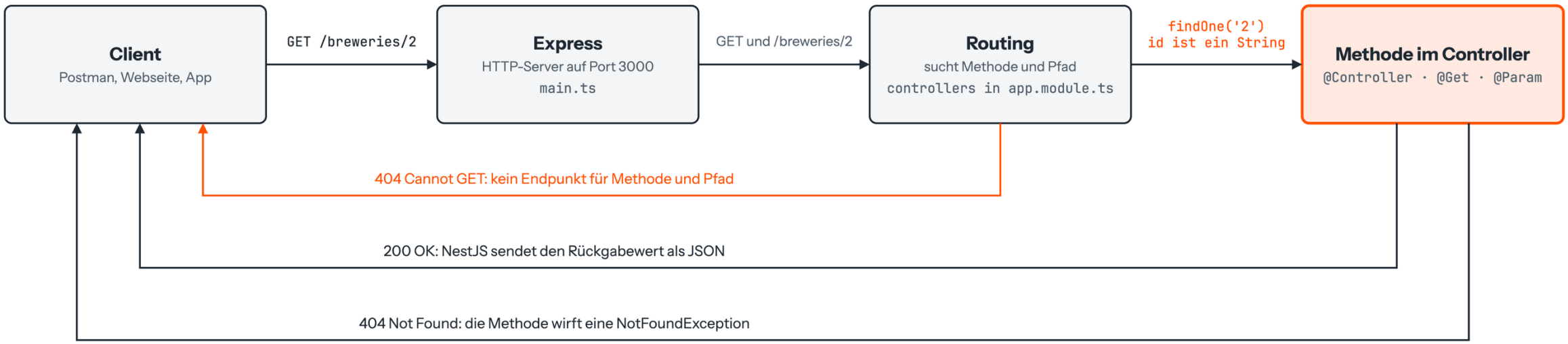
API, Backend und Framework

Die Begriffe API, Backend, Frontend, Endpunkt und Framework stehen mit ihrer Bedeutung auf Folie 5.5 der Vorlesung, die Aufgaben eines Frameworks auf Folie 5.6.

- Frontend und Backend sind Rollen. Postman ist für die Beer API ebenfalls ein Frontend.
- Ein Endpunkt ist Methode und Pfad. GET /beers und POST /beers sind zwei Endpunkte.
- NestJS ist in TypeScript geschrieben und nutzt intern Express als HTTP-Server.
- Das Basisprojekt stammt aus nest new und enthält schon Validierung und Swagger UI. Mit ihm haben alle dieselben Versionen.

Dokumentation: https://docs.nestjs.com/first-steps

Eine Anfrage läuft über Express und das Routing zur Methode im Controller



Der Client sendet GET /breweries/2 an Express auf Port 3000. Das Routing sucht zu Methode und Pfad eine Methode in den Controllern aus app.module.ts und ruft findOne mit der id als String auf. Findet das Routing keinen Endpunkt, antwortet es mit 404 Cannot GET. Gibt die Methode einen Wert zurück, sendet NestJS ihn als JSON mit 200; wirft sie eine NotFoundException, kommt 404 Not Found.

Der Weg einer Anfrage durch NestJS

src/main.ts erzeugt die Anwendung aus AppModule und startet Express auf Port 3000. Das Routing kennt nur die Controller, die src/app.module.ts unter controllers listet.

- Beim Start schreibt NestJS jede Route ins Terminal, zum Beispiel Mapped {/breweries/:id, GET} route.
- Fehlt die Zeile zu einem Endpunkt, kennt das Routing ihn nicht.
- Die beiden Ausgänge mit 404 unterscheidet die Meldung: Cannot GET kommt vom Routing, Brewery 42 not found aus der Methode.
- Ein Rückgabewert braucht kein JSON.stringify; das übernimmt NestJS.

Die übrigen Zeilen in main.ts und app.module.ts richten die Validierung und Swagger UI ein; sie kommen in den Einheiten 7 und 11.

Dokumentation: https://docs.nestjs.com/controllers

Decorators und Parameter

Die Decorators @Controller, @Get, @Param und @Query stehen mit ihrer Bedeutung auf Folie 5.11 der Vorlesung. Ein Decorator ist eine Funktion mit @ vor einer Klasse, einer Methode oder einem Parameter. NestJS liest die Decorators beim Start und baut daraus die Routen.

- Pfadparameter kommen immer als String an, auch 2 in /beers/2. Ein Query-Parameter kommt als String an, mehrfach genannt als Array.
- Ein Typ number am Parameter wandelt nichts um. TypeScript prüft Typen nur beim Übersetzen, nicht zur Laufzeit.
- Number('2') ergibt die Zahl 2, Number('abc') ergibt NaN.
- Fehlt ein Query-Parameter in der Anfrage, ist sein Wert undefined.

Dokumentation: https://docs.nestjs.com/controllers#route-parameters

Fehler finden in vier Schritten

Diese vier Schritte gelten für jede Fehlersuche im Semester.

1. Symptom: Lies Statuscode und Body der Antwort. Notiere die Meldung im Wortlaut.
2. Hypothese: Suche im Weg der Anfrage (Skript 5.5) die Stelle, die genau dieses Symptom erzeugt.
3. Spur: Prüfe die Hypothese, ohne Code zu ändern. Lies Terminal, Antwort und die Ausgabe von npx eslint src.
4. Beheben: Ändere genau eine Stelle. Sende dieselbe Anfrage erneut.

Cannot GET kommt vom Routing, eine eigene Meldung aus deiner Methode. Das Terminal von npm run start:dev zeigt Routen und Fehler beim Übersetzen. Wer zwei Stellen gleichzeitig ändert, weiß danach nicht, welche geholfen hat.

Dokumentation: https://docs.nestjs.com/faq/common-errors

Das Basisprojekt starten

Den Ordner beer-api bekommst du mit den Unterlagen dieser Einheit. Du brauchst Node.js 24 und Postman.

1. Öffne den Ordner beer-api in VS Code und darin ein Terminal.
2. Installiere die Pakete mit npm install.
3. Starte den Server mit npm run start:dev. Das Terminal zeigt Nest application successfully started.
4. Sende in Postman GET http://localhost:3000/. Die Antwort ist {"message":"Hello World"}.
5. Lege mit git init ein Repository an und committe den Ausgangsstand.

npm run start:dev übersetzt bei jedem Speichern neu und startet den Server neu. Committe nach dem Beispiel und nach jeder Aufgabe, zum Beispiel mit feat: Biere auflisten.

Beispiel: die Brauereien in vier Schritten

Das Live-Coding der Vorlesung baut GET /breweries und GET /breweries/:id. Übernimm es in dein Projekt, bevor du mit den Aufgaben beginnst.

1. Erzeuge den Controller mit der Nest CLI; sie trägt ihn in app.module.ts ein (Skript 5.11).
2. Schreibe Import, Typ Brewery und die Konstante breweries in die Datei (Skript 5.12 bis 5.14).
3. Schreibe findAll; der Rückgabewert wird zur Antwort (Skript 5.15).
4. Schreibe findOne; Number wandelt die id aus dem Pfad in eine Zahl (Skript 5.16).

Danach antwortet GET http://localhost:3000/breweries mit vier Brauereien, GET /breweries/2 mit dem Wildbräu Grafing und GET /breweries/42 mit 404.

Beispiel, Schritt 1: Die Nest CLI erzeugt den Controller und trägt ihn ein

BASH

```
npx nest generate controller breweries

CREATE src/breweries/breweries.controller.ts (107 bytes)
UPDATE src/app.module.ts (698 bytes)
```

Beispiel, Schritt 2: Import und Typ der Brauerei

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
import { Controller, Get, NotFoundException, Param } from '@nestjs/common';

interface Brewery {
  id: number;
  name: string;
  city: string;
  country: string;
}
```

Beispiel, Schritt 2: die Brauereien 1 und 2 in der Konstante breweries

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
// Feste Liste; ab Einheit 6 liegen die Daten im Service
const breweries: Brewery[] = [
 {
  id: 1,
  name: 'Privatbrauerei Schlumberger',
  city: 'Nattheim',
  country: 'Deutschland',
 },
 {
  id: 2,
  name: 'Wildbräu Grafing',
  city: 'Grafing bei München',
  country: 'Deutschland',
 },
```

Beispiel, Schritt 2: die Brauereien 3 und 4

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
  { id: 3, name: 'Augustiner-Bräu', city: 'München', country: 'Deutschland' },
  {
    id: 4,
    name: 'Edelweissbrauerei Farny',
    city: 'Kißlegg',
    country: 'Deutschland',
  },
];
```

Beispiel, Schritt 3: findAll gibt alle Brauereien zurück

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Controller('breweries')
export class BreweriesController {
  @Get()
  findAll(): Brewery[] {
    return breweries;
  }
```

Beispiel, Schritt 4: findOne sucht die Brauerei oder antwortet mit 404

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
  @Get(':id')
  findOne(@Param('id') id: string): Brewery {
    // Pfadparameter kommen immer als String an
    const brewery = breweries.find((b) => b.id === Number(id));
    if (!brewery) {
      throw new NotFoundException(`Brewery ${id} not found`);
    }
    return brewery;
  }
}
```

Aufgabe 5.1 · Biere auflisten

ZIEL

Die Beer API liefert die Liste aller Biere.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel der Brauereien.

SCHRITTE

1. Erzeuge den Controller beers mit der Nest CLI.
2. Lege den Typ Beer nach dem Datenmodell an (Skript 5.18).
3. Lege die vier Biere als Konstante an (Skript 5.19).
4. Schreibe die Methode für GET /beers.
5. Sende GET http://localhost:3000/beers in Postman.

FERTIG, WENN

- GET /beers antwortet mit 200 und den vier Bieren in der Reihenfolge der Testdaten.
- Jedes Bier hat id, name, style, abv und breweryId.

Datenmodell zu Aufgabe 5.1: das Bier

Alle Schlüssel sind englisch in camelCase. Die Felder kennst du aus Aufgabe 4.4.

- id: ganze Zahl, Kennung des Biers.
- name: String, Name des Biers, nicht leer.
- style: String, Bierstil: helles, pils, spezial, weizen oder dunkel.
- abv: Zahl, Alkoholgehalt in Volumenprozent (alcohol by volume).
- breweryId: ganze Zahl, id der Brauerei des Biers.
- description: String, Kurzbeschreibung; kein Pflichtfeld, die Testdaten haben keine.

Regel: Ein Bier gehört zu genau einer Brauerei. Eine Brauerei hat beliebig viele Biere.

Testdaten zu Aufgabe 5.1: die vier Biere in dieser Reihenfolge

id	name	style	abv	breweryId
1	Nattheimer Spezial	spezial	5.4	1
2	Wildbräu Grafing Helles	helles	5.0	2
3	Augustiner Lagerbier Hell	helles	5.2	3
4	Farny Kristall-Weizen	weizen	5.3	4

Aufgabe 5.2 · Ein Bier abrufen

ZIEL

Ein Bier lässt sich über seine id abrufen. Gibt es die id nicht, antwortet die API mit 404.

AUSGANGSLAGE

Aufgabe 5.1 ist gelöst.

SCHRITTE

1. Schreibe eine Methode für GET /beers/:id.
2. Lies die id aus dem Pfad.
3. Suche das Bier mit dieser id in der Liste.
4. Wirf eine NotFoundException, wenn kein Bier passt, zum Beispiel mit der Meldung Beer 42 not found.

FERTIG, WENN

- GET /beers/2 antwortet mit 200 und dem Wildbräu Grafing Helles.
- GET /beers/42 antwortet mit 404.

Zusatz 5.3 · Biere filtern

ZIEL

GET /beers?style=helles liefert nur die Biere eines Stils. Der Zusatz ist keine Voraussetzung für Einheit 6.

AUSGANGSLAGE

Aufgabe 5.1 ist gelöst.

SCHRITTE

1. Lies den Query-Parameter style in der Methode für GET /beers.
2. Gib ohne style alle Biere zurück.
3. Gib mit style nur die Biere dieses Stils zurück.

FERTIG, WENN

- GET /beers?style=helles liefert das Wildbräu Grafing Helles und das Augustiner Lagerbier Hell.
- GET /beers liefert weiter alle vier Biere.

Selbstkontrolle mit Postman

Der Ordner der Einheit 5 in der Collection Web-Programmierung prüft das Beispiel und deine Aufgaben. Importiere die Datei web-programmierung.postman_collection.json aus den Unterlagen mit **Import**, falls sie noch nicht in Postman ist. Starte den Server und führe darin den Ordner deiner Aufgabe mit **Run** aus, zum Beispiel „Aufgabe 5.1 – Biere auflisten“. Der Ordner „Beispiel – Brauereien abrufen“ prüft das übernommene Beispiel.

Ein roter Test nennt einen Hinweis auf Ort und Ursache. Lies danach Statuscode und Body der Antwort und das Terminal.

Hinweise zu Aufgabe 5.1

- Stufe 1: Das Beispiel der Brauereien hat denselben Aufbau wie deine Lösung.
- Stufe 2: Typ, Konstante und Klasse stehen wie bei den Brauereien in einer Datei (Skript 5.12 bis 5.15).
- Stufe 3: Erzeuge den Controller mit npx nest generate controller beers. Schreibe findAll nach dem Muster von Skript 5.15.

Hinweise zu Aufgabe 5.2

- Stufe 1: Der Wert aus dem Pfad hat einen anderen Typ als die id in deinen Daten.
- Stufe 2: @Param('id') liefert einen String, auch mit dem Typ number. find liefert undefined, wenn kein Bier passt.
- Stufe 3: Wandle den Wert mit Number um, bevor find vergleicht. Prüfe danach das Ergebnis von find.

Hinweise zu Zusatz 5.3

- Stufe 1: Die Query gehört zur Anfrage GET /beers. Es gibt keinen neuen Endpunkt.
- Stufe 2: @Query('style') übergibt den Wert als Parameter von findAll. Ohne Query ist er undefined.
- Stufe 3: Ergänze in findAll einen Parameter mit @Query('style'). Unterscheide in der Methode die Fälle mit und ohne Wert.

Typische Fehler

Symptom	Ursache
404 mit „Cannot GET /beers“	Der Controller fehlt unter controllers in app.module.ts, oder der Pfad in @Controller weicht ab.
404 mit „Cannot GET /beers/2“	@Get('id') statt @Get(':id'): Ohne Doppelpunkt ist id ein fester Pfad. Das Terminal zeigt Mapped {/beers/id, GET} route.
404 mit „Beer 2 not found“ für ein vorhandenes Bier	find vergleicht die Zahl aus den Daten mit dem String aus dem Pfad. Der Typ number am Parameter wandelt nichts um.
GET /beers/42 antwortet mit 200 und leerem Body	findOne gibt das Ergebnis von find ungeprüft zurück; find liefert undefined.
GET /beers?style=helles liefert alle vier Biere	findAll liest den Query-Parameter nicht, oder der Name in @Query weicht von style ab.

Das solltest du jetzt können

- ✓ Begründen, warum Webseite und App eine gemeinsame API nutzen
- ✓ Den Weg einer Anfrage von Express bis zur Methode im Controller zeigen
- ✓ Einen Controller mit der Nest CLI erzeugen und Endpunkte mit @Get, @Param und @Query bauen
- ✓ Einen Fehler in vier Schritten vom Symptom zur Ursache verfolgen

Als Nächstes: Service und Dependency Injection – wo Daten und Logik hingehören, wenn der Controller nur noch HTTP beantwortet.

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 6

Service und Dependency Injection · Skript

Worum es in dieser Einheit geht

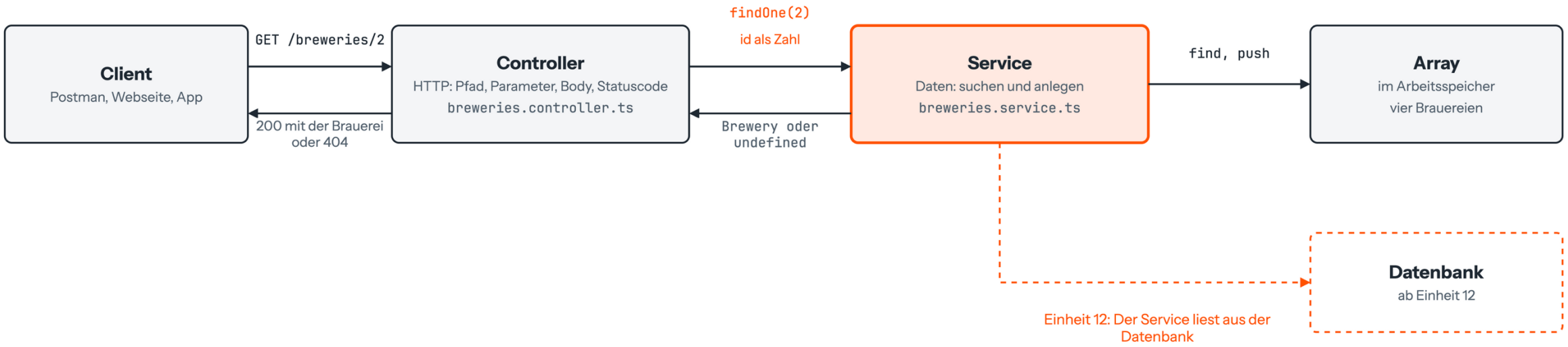
Bisher halten die Controller Daten und Suche selbst. Ab Einheit 12 liegen die Daten in einer Datenbank; dann ändert sich jede Zeile, die auf die Daten zugreift. In dieser Einheit ziehen Daten und Suche in Services, und NestJS übergibt jeden Service per Dependency Injection an seinen Controller. Die Aufgaben setzen diese Story um:

Als Betreiber möchte ich ein Bier anlegen und löschen, damit die Liste dem Angebot der Brauereien entspricht.

Am Ende der Einheit kannst du:

- Code nach Controller und Service trennen und den Service im Constructor anfordern,
- Endpunkte zum Anlegen und Löschen bauen.

Der Service hält die Daten an einer Stelle, der Controller beantwortet HTTP



Der Client sendet GET /breweries/2 an den Controller. Der Controller wandelt die id in eine Zahl und ruft findOne(2) im Service auf. Der Service sucht im Array im Arbeitsspeicher und gibt die Brauerei oder undefined zurück. Der Controller antwortet mit 200 oder 404. Ab Einheit 12 liest der Service aus einer Datenbank statt aus dem Array.

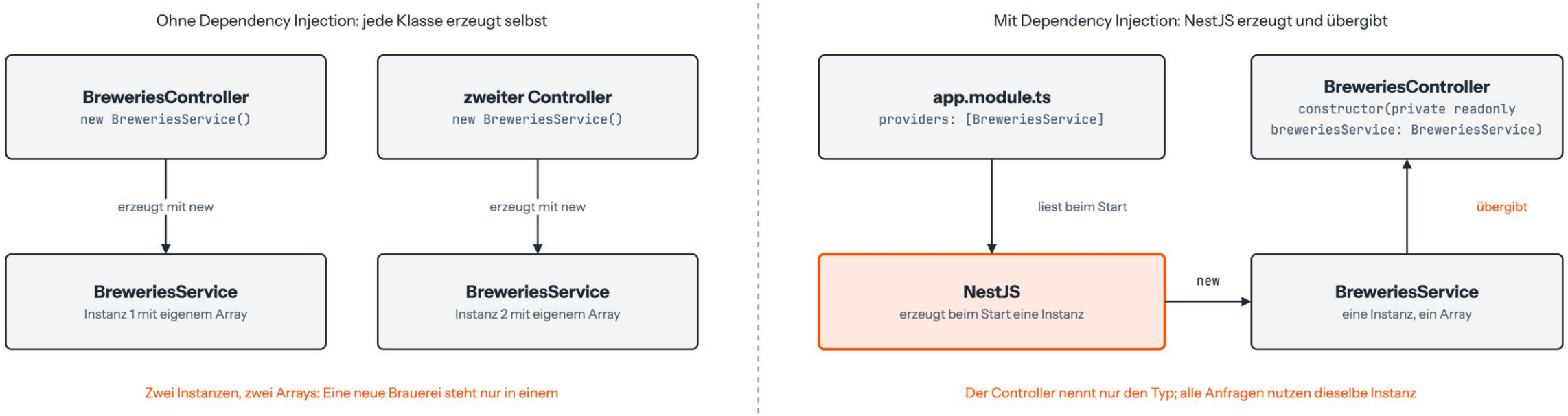
Controller und Service

Welche Aufgabe in den Controller und welche in den Service gehört, zeigt Folie 6.5 der Vorlesung. Der Weg einer Anfrage bis zum Controller steht in Skript 5.6.

- Der Controller liest Pfad, Parameter und Body, wandelt die id in eine Zahl und entscheidet über den Statuscode.
- Der Service bekommt Werte wie die Zahl 2 und gibt Werte zurück, zum Beispiel eine Brauerei oder undefined.
- Vereinfachung: In dieser Einheit kennt der Service kein HTTP.
- Ab Einheit 8 wirft der Service selbst eine NotFoundException, und NestJS antwortet mit 404.

Dokumentation: https://docs.nestjs.com/providers#services

Mit Dependency Injection erzeugt NestJS den Service und übergibt ihn dem Controller



Ohne Dependency Injection erzeugt jeder Controller seinen Service mit new; zwei Controller hätten zwei Instanzen mit je eigenem Array. Mit Dependency Injection liest NestJS beim Start providers in app.module.ts, erzeugt eine Instanz von BreweriesService und übergibt sie dem Constructor von BreweriesController.

Dependency Injection in NestJS

Dependency Injection heißt: Eine Klasse bekommt die Objekte übergeben, die sie braucht. Die drei Stellen dafür stehen auf Folie 6.8 der Vorlesung, die Decorators zum Anlegen und Löschen auf Folie 6.10.

- private readonly vor dem Parameter im Constructor legt das Feld this.breweriesService an.
- Einen Typ aus einer anderen Datei, der in einer Methode mit Decorator vorkommt, importierst du mit import type.
- Vereinfachung: NestJS erzeugt einen Service standardmäßig einmal je Module. Alle Anfragen teilen diese Instanz und ihr Array.
- Das Array liegt im Arbeitsspeicher. Nach jedem Neustart beginnt es wieder mit den Daten aus dem Code.

Dokumentation: https://docs.nestjs.com/fundamentals/injection-scopes

Beispiel: die Brauereien in sechs Schritten

Übernimm das Live-Coding in dein Projekt aus Einheit 5 oder in den Lösungsstand der Einheit 5, bevor du mit den Aufgaben beginnst.

1. Erzeuge den Service mit der Nest CLI (Skript 6.8).
2. Verschiebe Typ und Array in den Service, schreibe dort die Suche (Skript 6.9, 6.10).
3. Fordere den Service im Constructor an (Skript 6.11).
4. Rufe ihn in findAll und findOne auf (Skript 6.12).
5. Schreibe create im Service (Skript 6.13).
6. Schreibe POST /breweries im Controller (Skript 6.14).

Danach antwortet POST /breweries mit 201 und der Brauerei mit der id 5. Starte den Server vor jeder Prüfung neu.

Beispiel, Schritt 1: Die Nest CLI erzeugt den Service und trägt ihn ein

BASH

```
npx nest generate service breweries

CREATE src/breweries/breweries.service.ts (93 bytes)
UPDATE src/app.module.ts (863 bytes)
```

Beispiel, Schritt 2: Typ und Array ziehen in den Service

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
import { Injectable } from '@nestjs/common';

export interface Brewery {
  id: number;
  name: string;
  city: string;
  country: string;
}

@Injectable()
export class BreweriesService {
  // Daten im Arbeitsspeicher: Nach einem Neustart sind es wieder vier
  private readonly breweries: Brewery[] = [
```

Beispiel, Schritt 2: Zähler und Suche im Service

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
private nextId = 5;

findAll(): Brewery[] {
  return this.breweries;
}

findOne(id: number): Brewery | undefined {
  return this.breweries.find((b) => b.id === id);
}
```

Beispiel, Schritt 3: Der Controller fordert den Service im Constructor an

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
import {
  Body,
  Controller,
  Get,
  NotFoundException,
  Param,
  Post,
} from '@nestjs/common';
import { BreweriesService } from './breweries.service';
import type { Brewery } from './breweries.service';

@Controller('breweries')
export class BreweriesController {
  constructor(private readonly breweriesService: BreweriesService) {}
```

Beispiel, Schritt 3 und 4: findAll und findOne rufen den Service auf

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Get()
findAll(): Brewery[] {
  return this.breweriesService.findAll();
}

@Get(':id')
findOne(@Param('id') id: string): Brewery {
  // Pfadparameter kommen immer als String an
  const brewery = this.breweriesService.findOne(Number(id));
  if (!brewery) {
    throw new NotFoundException(`Brewery ${id} not found`);
  }
  return brewery;
}
```

Beispiel, Schritt 5: create vergibt die nächste id und legt die Brauerei ab

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
  create(data: Omit<Brewery, 'id'>): Brewery {
    const brewery = { id: this.nextId++, ...data };
    this.breweries.push(brewery);
    return brewery;
  }
}
```

Beispiel, Schritt 6: POST /breweries reicht den Body an den Service weiter

BEISPIEL

- Ergänze Body und Post im Import aus @nestjs/common.
- Sende in Postman POST http://localhost:3000/breweries mit Body raw und JSON.
- Body: { "name": "Brauerei Gold Ochsen", "city": "Ulm", "country": "Deutschland" }
- Die Antwort ist 201 mit der Brauerei und der id 5.

```
  @Post()
  create(@Body() data: Omit<Brewery, 'id'>): Brewery {
    return this.breweriesService.create(data);
  }
}
```

Aufgabe 6.1 · Ein Bier anlegen

ZIEL

POST /beers legt ein Bier an. Daten und Suche der Biere liegen im BeersService.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel der Brauereien.

SCHRITTE

1. Erzeuge den Service beers mit der Nest CLI.
2. Verschiebe Typ und Biere in den Service.
3. Schreibe dort findAll und findOne.
4. Fordere den Service im BeersController an.
5. Schreibe create im Service.
6. Schreibe die Methode für POST /beers.

FERTIG, WENN

- POST /beers mit dem Body aus Skript 6.16 antwortet mit 201, dem Bier und der id 5.
- Danach liefert GET /beers fünf Biere.

Daten zu Aufgabe 6.1: das neue Bier

BEISPIEL

- Datenmodell und die vier Biere: Skript 5.18 und 5.19.
- Regel: Alle id sind ganze Zahlen, die der Server vergibt.
- Der Body enthält deshalb keine id; die erste neue id ist 5.
- Omit<Beer, 'id'> beschreibt ein Bier ohne id, wie es im Body ankommt.

```
{
  "name": "Wildbräu Grafing Pils",
  "style": "pils",
  "abv": 4.9,
  "breweryId": 2
}
```

Aufgabe 6.2 · Ein Bier löschen

ZIEL

DELETE /beers/:id entfernt ein Bier aus der Liste.

AUSGANGSLAGE

Aufgabe 6.1 ist gelöst. Prüfe im selben Lauf des Servers: Lege zuerst das Pils an, dann lösche es.

SCHRITTE

1. Schreibe im Service eine Methode remove, die ein Bier anhand seiner id entfernt.
2. Schreibe im Controller die Methode für DELETE /beers/:id mit @Delete(':id').
3. Wandle die id aus dem Pfad in eine Zahl und rufe remove auf.

FERTIG, WENN

- DELETE /beers/5 antwortet mit 200.
- Danach antwortet GET /beers/5 mit 404.

Zusatz 6.3 · Biere nach Stil filtern

ZIEL

GET /beers?style=pils findet auch Biere, die POST /beers angelegt hat. Der Zusatz ist keine Voraussetzung für Einheit 7.

AUSGANGSLAGE

Aufgabe 6.1 ist gelöst. Zusatz 5.3 ist nicht nötig.

SCHRITTE

1. Lies im Controller den Query-Parameter style.
2. Gib ihn an findAll im Service weiter.
3. Filtere im Service das Array, in dem create die Biere ablegt.

FERTIG, WENN

- Nach dem Anlegen liefert GET /beers?style=pils das Wildbräu Grafing Pils.
- Die Antwort enthält nur Biere im Stil pils.

Selbstkontrolle mit Postman

Der Ordner der Einheit 6 in der Collection Web-Programmierung prüft das Beispiel und deine Aufgaben. Starte den Server frisch und führe darin den Ordner deiner Aufgabe mit **Run** aus, zum Beispiel „Aufgabe 6.1 – Ein Bier anlegen“.

Die Anfragen ändern die Daten im Arbeitsspeicher. Starte den Server deshalb vor jedem Lauf neu. Aufgabe 6.2 baut auf Aufgabe 6.1 im selben Lauf auf. Ein roter Test nennt einen Hinweis auf Ort und Ursache.

Hinweise zu Aufgabe 6.1

- Stufe 1: Das Beispiel der Brauereien hat denselben Aufbau wie deine Lösung.
- Stufe 2: Der Typ braucht export, der Controller importiert ihn mit import type. create bekommt den Body ohne id.
- Stufe 3: Erzeuge den Service mit npx nest generate service beers. Schreibe create nach dem Muster von Skript 6.13.

Hinweise zu Aufgabe 6.2

- Stufe 1: Das Bier muss aus dem Array verschwinden, das findAll und findOne lesen.
- Stufe 2: filter erzeugt ein neues Array und ändert das Array im Feld nicht.
- Stufe 3: Entferne das Bier aus demselben Array. Die Methoden eines Arrays zeigt https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

Hinweise zu Zusatz 6.3

- Stufe 1: Die Query gehört zur Anfrage GET /beers. Es gibt keinen neuen Endpunkt.
- Stufe 2: @Query('style') liefert den Wert im Controller, ohne Query undefined.
- Stufe 3: Gib den Wert als Parameter an findAll. Filtere im Service nur, wenn ein Wert da ist.

Typische Fehler

Symptom	Ursache
Postman meldet „connect ECONNREFUSED“; das Terminal zeigt „Nest can't resolve dependencies of the BeersController (?)“	Der Service fehlt unter providers in app.module.ts.
Das Terminal meldet „TS1272: A type referenced in a decorated signature must be imported with 'import type'“	Der Controller importiert den Typ Beer ohne import type.
Das Terminal meldet „TS2540: Cannot assign to 'beers' because it is a read-only property“	remove weist das Ergebnis von filter dem Feld zu, das readonly ist.
Nach DELETE /beers/5 antwortet GET /beers/5 weiter mit 200	remove ruft filter auf und verwirft das neue Array; das Array, das findOne liest, bleibt gleich.
POST /beers antwortet mit einer id größer als 5, GET /beers liefert mehr als fünf Biere	Der Server lief schon vor dem Lauf; die Daten der letzten Anfragen sind noch im Array.

Das solltest du jetzt können

- ✓ Begründen, welcher Code in den Controller und welcher in den Service gehört
- ✓ Einen Service mit der Nest CLI erzeugen und im Constructor anfordern
- ✓ Endpunkte mit @Post, @Body und @Delete bauen
- ✓ Erklären, warum angelegte Daten einen Neustart nicht überstehen
- ✓ Die Meldung „can't resolve dependencies“ auf einen fehlenden Provider zurückführen

Als Nächstes: DTOs und Validierung mit Zod – wie die API einen Body prüft, bevor er den Service erreicht.

WEB-PROGRAMMIERUNG · EINHEIT 7

DTOs und Validierung mit Zod · Skript

Worum es in dieser Einheit geht

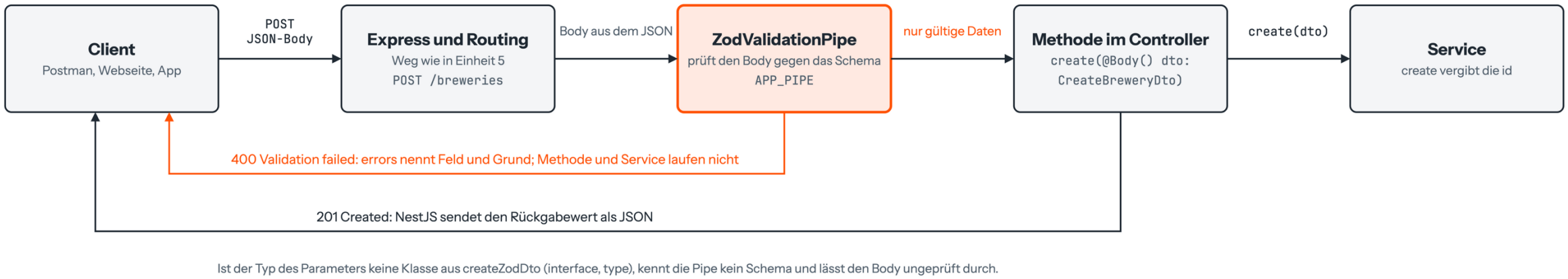
Seit Einheit 6 nimmt die Beer API jeden Body an, auch eine Brauerei ohne Namen. Webseite, App und Berichte der regionalen Brauereien lesen dieselben Daten; ein falscher Datensatz fällt oft erst im Bericht auf. In dieser Einheit prüft ein Zod-Schema jeden Body, bevor er den Service erreicht. Die Aufgaben setzen diese Story um:

Als Betreiber möchte ich, dass die API nur vollständige und gültige Biere annimmt, damit keine fehlerhaften Daten in die App gelangen.

Am Ende der Einheit kannst du:

- ein Zod-Schema schreiben und daraus ein DTO für einen Endpunkt erzeugen,
- eine Antwort 400 lesen und das fehlerhafte Feld benennen.

Eine Pipe prüft Parameter, bevor die Methode im Controller läuft



Ist der Typ des Parameters keine Klasse aus createZodDto (interface, type), kennt die Pipe kein Schema und lässt den Body ungeprüft durch.

Der Client sendet POST /breweries mit einem JSON-Body. Nach Express und Routing prüft die ZodValidationPipe den Body gegen das Schema. Ist er ungültig, antwortet sie mit 400 Validation failed; Methode und Service laufen nicht. Ist er gültig, ruft NestJS die Methode im Controller auf, die den Service aufruft, und antwortet mit 201. Ist der Typ des Parameters keine Klasse aus createZodDto, lässt die Pipe den Body ungeprüft durch.

Die Pipe im Weg der Anfrage

Der Weg einer Anfrage bis zum Controller steht in Skript 5.6. Neu ist die Pipe zwischen Routing und Methode. Warum der Server prüft, obwohl der Client prüfen kann, zeigt Folie 7.4 der Vorlesung.

- Das Basisprojekt registriert die ZodValidationPipe mit APP_PIPE in app.module.ts für alle Endpunkte.
- Sie prüft Body, Query und Pfadparameter, aber nur, wenn ihr Typ ein DTO ist.
- Ein Pfadparameter mit dem Typ string bleibt ungeprüft; GET /breweries/abc antwortet weiter mit 404.
- Bisher hatte die Pipe nichts zu tun, weil kein Parameter ein DTO als Typ hatte.

Dokumentation: https://docs.nestjs.com/pipes

Schema, DTO und Typ

Zod ist eine Bibliothek, die Daten zur Laufzeit gegen ein Schema prüft. Ein DTO (Data Transfer Object) beschreibt, welche Daten ein Endpunkt annimmt. Mit nestjs-zod entsteht es aus einem Zod-Schema. Die Bausteine eines Schemas stehen auf Folie 7.7 der Vorlesung, Typ und Klasse im Vergleich auf Folie 7.8.

- z.infer<typeof Schema> liefert den TypeScript-Typ. Ein zweites interface entfällt.
- Ein Typ fehlt zur Laufzeit. Nur die Klasse aus createZodDto trägt das Schema bis zur Pipe.
- Importiere die DTO-Klasse im Controller ohne import type.
- z.object entfernt Felder, die das Schema nicht kennt, ohne Meldung.

Dokumentation: https://zod.dev/api

Eine Antwort 400 lesen

Die Pipe ruft parse am Schema auf. Wirft parse einen ZodError, antwortet die Pipe mit 400 und listet die Fehler von Zod unter errors. Die Felder der Antwort stehen auf Folie 7.10 der Vorlesung.

- path nennt das Feld. Ein leerer path heißt: Der ganze Body fehlt oder ist kein Objekt.
- code nennt die Art des Fehlers, zum Beispiel invalid_type für einen falschen Typ.
- Ein Body mit mehreren Fehlern ergibt mehrere Einträge.
- Mit Content-Type: text/plain kommt der Body als undefined an (Skript 4.6); dann ist path leer.

Dokumentation: https://github.com/BenLorantfy/nestjs-zod

Beispiel: die Brauereien in vier Schritten

Übernimm das Live-Coding in dein Projekt aus Einheit 6 oder in den Lösungsstand der Einheit 6, bevor du mit den Aufgaben beginnst.

1. Lies in app.module.ts, wo die Pipe registriert ist (Skript 7.8).
2. Lege den Ordner dto und darin Schema und DTO für eine neue Brauerei von Hand an (Skript 7.9).
3. Verwende das DTO in Controller und Service (Skript 7.10, 7.11).
4. Sende einen ungültigen Body und lies die Antwort 400 (Skript 7.12).

Von Hand, weil nest generate class einen zusätzlichen Unterordner anlegt. Danach antwortet POST /breweries mit einem gültigen Body weiter mit 201 und mit einem ungültigen Body mit 400.

Beispiel, Schritt 1: Die Pipe ist seit dem Basisprojekt registriert

TYPESCRIPT · SRC/APP.MODULE.TS

```
providers: [
  // Prüft Body, Query und Pfadparameter gegen die Zod-Schemas der DTOs
  { provide: APP_PIPE, useClass: ZodValidationPipe },
  // Prüft Antworten gegen Zod-Schemas, wenn ein Endpunkt eines angibt
  { provide: APP_INTERCEPTOR, useClass: ZodSerializerInterceptor },
  BreweriesService,
  BeersService,
],
```

Beispiel, Schritt 2: Schema und DTO für eine neue Brauerei

TYPESCRIPT · SRC/BREWERIES/DTO/CREATE-BREWERY.DTO.TS

```typescript
import { createZodDto } from 'nestjs-zod';
import { z } from 'zod';

// Gültige Daten für eine neue Brauerei; die id vergibt der Service
export const CreateBrewerySchema = z.object({
  name: z.string().min(1),
  city: z.string().min(1),
  country: z.string().min(1),
});

// Klasse aus dem Schema: Nur eine Klasse prüft die ZodValidationPipe
export class CreateBreweryDto extends createZodDto(CreateBrewerySchema) {}
```

Beispiel, Schritt 3: Der Controller nimmt das DTO

BEISPIEL

- Importiere die Klasse mit import { CreateBreweryDto } from './dto/create-brewery.dto'.
- Schreibe kein import type: Die Pipe braucht die Klasse zur Laufzeit.
- Ersetze Omit<Brewery, 'id'> durch CreateBreweryDto.

```
@Post()
create(@Body() dto: CreateBreweryDto): Brewery {
  return this.breweriesService.create(dto);
}
```

Beispiel, Schritt 3: Der Service nimmt das DTO

BEISPIEL

- Importiere CreateBreweryDto im Service wie im Controller.
- Ersetze auch hier Omit<Brewery, 'id'> durch CreateBreweryDto.

```
create(dto: CreateBreweryDto): Brewery {
  const brewery = { id: this.nextId++, ...dto };
  this.breweries.push(brewery);
  return brewery;
}
```

Beispiel, Schritt 4: Ein ungültiger Body ergibt 400

BEISPIEL

- Sende POST http://localhost:3000/breweries mit Body raw und JSON.
- Body: { "name": 42, "city": "Ulm", "country": "Deutschland" }
- path nennt name, code nennt invalid_type.
- Ein leeres Objekt ergibt drei Einträge.

```
{
  "statusCode": 400,
  "message": "Validation failed",
  "errors": [{
    "expected": "string",
    "code": "invalid_type",
    "path": ["name"],
    "message":
    "Invalid input: expected string, received number"
  }]
}
```

Aufgabe 7.1 · Pflichtfelder und Typen prüfen

ZIEL

POST /beers nimmt nur Biere mit allen Pflichtfeldern im richtigen Typ an.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel der Brauereien.

SCHRITTE

1. Lege src/beers/dto/create-beer.dto.ts an.
2. Schreibe dort CreateBeerSchema nach dem Datenmodell in Skript 7.14.
3. Erzeuge daraus die Klasse CreateBeerDto.
4. Verwende CreateBeerDto in create von Controller und Service.

FERTIG, WENN

- Das Pils aus Skript 7.15 ergibt 201 und die id 5.
- Die drei ungültigen Bodies dort ergeben 400 mit dem Feld.

Datenmodell zu Aufgabe 7.1: ein neues Bier

Feld	Typ	Pflicht	Bedeutung
name	String	ja	Name des Biers, nicht leer
style	String	ja	Bierstil; die erlaubten Werte stehen in Skript 7.17
abv	Zahl	ja	Alkoholgehalt in Volumenprozent (alcohol by volume)
breweryId	ganze Zahl	ja	id der Brauerei des Biers
description	String	nein	Kurzbeschreibung

Testdaten zu Aufgabe 7.1

BEISPIEL

- Die id fehlt im Body, weil der Service sie vergibt.
- Ohne name: 400, die Antwort nennt name.
- Mit "name": "": 400, die Antwort nennt name.
- Mit "abv": "5.2" als String: 400, die Antwort nennt abv.

```
{
  "name": "Wildbräu Grafing Pils",
  "style": "pils",
  "abv": 4.9,
  "breweryId": 2
}
```

Aufgabe 7.2 · Bierstil und Alkoholgehalt prüfen

ZIEL

POST /beers nimmt nur die fünf Bierstile der Beer App und einen Alkoholgehalt von 0 bis 20 an.

AUSGANGSLAGE

Aufgabe 7.1 ist gelöst.

SCHRITTE

1. Lies die Regeln in Skript 7.17.
2. Beschränke style in CreateBeerSchema auf die fünf Bierstile.
3. Lege für abv die Grenzen aus der Regel fest.

FERTIG, WENN

- Die drei Bodies aus Skript 7.17 antworten mit 400 und nennen das Feld.
- Das Pils aus Skript 7.15 ergibt weiter 201.

Regeln und Testdaten zu Aufgabe 7.2

Bierstile der Beer App: helles, pils, spezial, weizen oder dunkel. Andere Werte sind ungültig.

Regel zum Alkoholgehalt: abv liegt zwischen 0 und 20.

Ob die Brauerei mit der breweryId existiert, prüft erst Einheit 9 nach der Regel „Ein Bier gehört zu genau einer Brauerei; die Brauerei muss existieren.“

Jeder dieser Bodies ist das Pils aus Skript 7.15 mit einem geänderten Feld:

- "style": "lager" ergibt 400, die Antwort nennt style.
- "abv": 25 ergibt 400, die Antwort nennt abv.
- "abv": -1 ergibt 400, die Antwort nennt abv.

Zusatz 7.3 · Ein Bier ändern

ZIEL

PATCH /beers/:id ändert nur die Felder aus dem Body. Der Zusatz ist keine Voraussetzung für die Aufgaben der Einheit 8.

AUSGANGSLAGE

Aufgabe 7.2 ist gelöst.

SCHRITTE

1. Lege src/beers/dto/update-beer.dto.ts mit einem Schema aus partial() an.
2. Schreibe im Service eine Methode update, die nur die Felder aus dem Body überschreibt.
3. Schreibe im Controller die Methode für PATCH /beers/:id. Eine unbekannte id ergibt 404.

FERTIG, WENN

- PATCH /beers/2 mit { "abv": 5.1 } antwortet mit 200; Name und Stil bleiben.
- { "abv": 25 } ergibt 400.

Selbstkontrolle mit Postman

Der Ordner der Einheit 7 in der Collection Web-Programmierung prüft das Beispiel und deine Aufgaben. Starte den Server frisch und führe darin den Ordner deiner Aufgabe mit **Run** aus, zum Beispiel „Aufgabe 7.1 – Pflichtfelder und Typen prüfen“.

Die Anfragen legen Daten im Arbeitsspeicher an. Starte den Server deshalb vor jedem Lauf neu. Ein roter Test nennt einen Hinweis auf Ort und Ursache.

Hinweise zu Aufgabe 7.1

- Stufe 1: Das Schema der Brauereien in Skript 7.9 hat denselben Aufbau.
- Stufe 2: Jede Zeile im Datenmodell wird ein Feld in z.object; die Spalte Typ wählt den Baustein.
- Stufe 3: Die Bausteine für Zahl, ganze Zahl und ein Feld, das fehlen darf, stehen auf Folie 7.7 der Vorlesung.

Hinweise zu Aufgabe 7.2

- Stufe 1: Beide Regeln gehören ins Schema, nicht in den Service.
- Stufe 2: Ein Feld mit fester Liste von Werten braucht einen anderen Baustein als z.string().
- Stufe 3: Hänge an z.number() eine untere und eine obere Grenze an. Prüfe, ob 0 und 20 gültig bleiben.

Hinweise zu Zusatz 7.3

- Stufe 1: Das Schema für PATCH baut auf CreateBeerSchema auf; die Regeln gelten weiter.
- Stufe 2: Der Body von PATCH enthält nur die Felder, die sich ändern. Der Rest des Biers bleibt im Array.
- Stufe 3: Suche das Bier mit findOne. Eine Methode von Object kopiert die Felder eines Objekts in ein anderes: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object

Typische Fehler

Symptom	Ursache
Jeder Body ergibt 201; das Terminal meldet „Found 0 errors“	@Body() hat einen Typ aus z.infer, oder CreateBeerDto ist mit import type importiert.
"style": "lager" ergibt 201	style ist z.string() statt einer Aufzählung der Bierstile.
"abv": "5.2" ergibt 201 mit "abv": 5.2	abv ist z.coerce.number(); das wandelt den String in eine Zahl.
PATCH mit { "abv": 5.1 } ergibt 400 mit „Invalid input: expected string, received undefined“ an name	Der Parameter von PATCH hat den Typ CreateBeerDto statt des Schemas aus partial().
400 mit path [] und „Invalid input: expected object, received undefined“	Postman sendet den Body als Text; unter Body raw ist nicht JSON gewählt.

Das solltest du jetzt können

- ✓ Begründen, warum der Server Eingaben prüft, auch wenn der Client prüft
- ✓ Ein Zod-Schema mit Pflichtfeldern, Aufzählung und Grenzen schreiben
- ✓ Aus einem Schema ein DTO erzeugen und an einem Endpunkt verwenden
- ✓ In einer Antwort 400 das fehlerhafte Feld und den Grund benennen
- ✓ Erklären, warum ein TypeScript-Typ allein nichts prüft

Als Nächstes: Fehlerbehandlung und Statuscodes – wie jede Antwort der API einen passenden Statuscode und eine verständliche Meldung bekommt.

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 8

Fehlerbehandlung und Statuscodes im Backend · Skript

Worum es in dieser Einheit geht

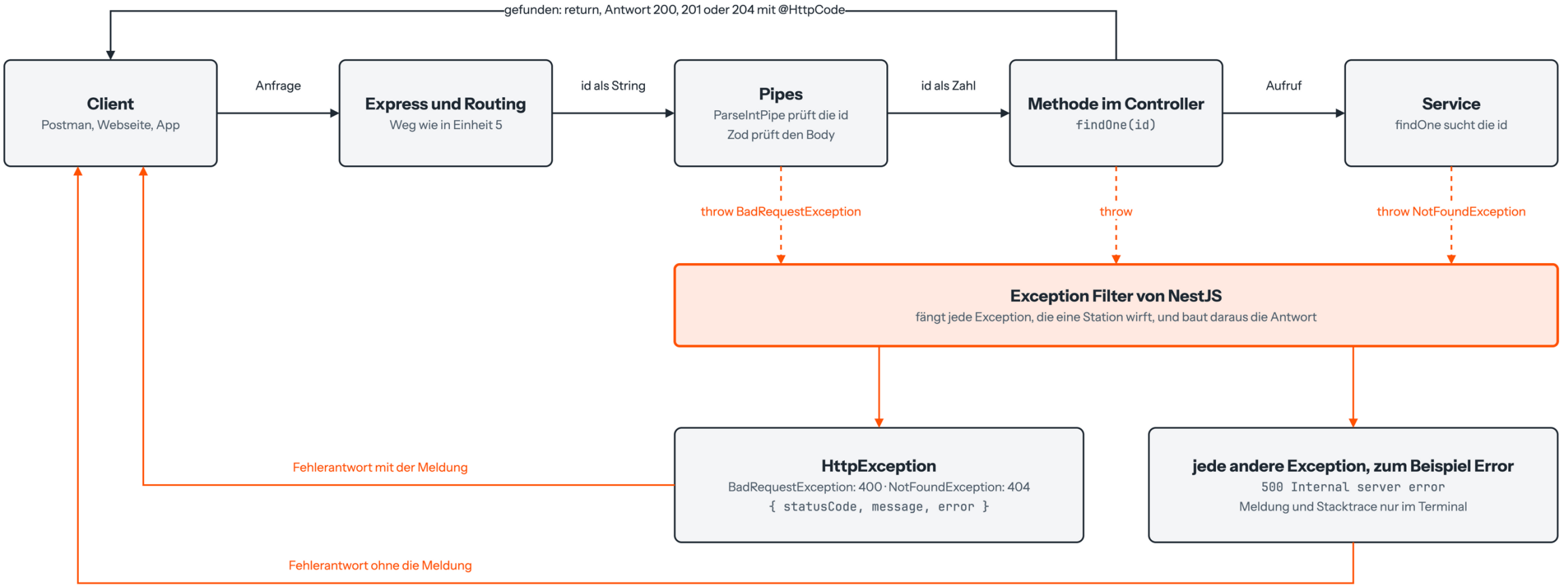
Webseite, App und die Webseiten der Brauereien zeigen an, was die Beer API meldet. Eine Meldung mit dem falschen Statuscode schickt die Frage an die falsche Stelle, oft als Ticket zum Betreiber. In dieser Einheit bekommt jede Anfrage einen passenden Statuscode und jede Fehlerantwort dasselbe Format. Die Aufgaben setzen diese Story um:

Als Betreiber möchte ich, dass jede Anfrage einen passenden Statuscode und eine verständliche Meldung liefert, damit Webseite und App Fehler anzeigen können und Gäste weniger beim Support nachfragen.

Am Ende der Einheit kannst du:

- zu einer Situation den passenden Statuscode bestimmen,
- mit Exceptions, ParseIntPipe und @HttpCode die passende Antwort erzeugen.

Jede Station kann eine Exception werfen; NestJS baut daraus die Antwort



Der Client sendet eine Anfrage. Sie läuft über Express und Routing, die Pipes und die Methode im Controller bis zum Service. Findet der Service die id, gibt die Methode das Ergebnis zurück, und NestJS antwortet mit 200, 201 oder 204. Wirft eine Station eine Exception, fängt sie der Exception Filter von NestJS. Aus einer HttpException wie BadRequestException oder NotFoundException wird eine Antwort mit statusCode, message und error. Aus jeder anderen Exception wird 500 Internal server error; Meldung und Stacktrace stehen nur im Terminal.

Exceptions und Statuscodes

Der Weg einer Anfrage bis zum Controller steht in Skript 5.6. Welcher Code zu welcher Situation gehört, zeigt Folie 8.4 der Vorlesung, die Exceptions von NestJS Folie 8.6.

- throw beendet die Methode sofort. Die Stationen danach laufen nicht.
- Der Exception Filter von NestJS fängt jede Exception. Eine HttpException bestimmt Statuscode und Meldung.
- Für jede andere Exception, etwa Error, antwortet NestJS mit 500 und ohne die Meldung.
- Meldung und Stacktrace einer 500 stehen im Terminal von start:dev. In einer Antwort würden sie den Aufbau des Servers zeigen.

Dokumentation: https://docs.nestjs.com/exception-filters

Pfadparameter, Statuscode für Erfolg und ein Format für alle Fehler

Pfadparameter kommen als String an. Die Zod-Pipe aus Einheit 7 prüft sie nicht, weil ihr Typ keine DTO-Klasse ist.

- ParseIntPipe als zweiter Parameter von @Param() wandelt die id in eine ganze Zahl.
- Ist die id keine ganze Zahl, antwortet die Pipe mit 400. Ob es die id gibt, prüft findOne im Service.
- Ohne Angabe antwortet NestJS auf POST mit 201, sonst mit 200. @HttpCode setzt einen anderen Code, etwa 204.
- Eine Antwort 204 hat nie einen Body, auch wenn die Methode etwas zurückgibt.
- createZodValidationPipe aus nestjs-zod baut eine Pipe, deren Fehler das Format statusCode, message, error haben.

Dokumentation: https://docs.nestjs.com/pipes

Beispiel: die Fehler der Brauereien in vier Schritten

Übernimm das Live-Coding in dein Projekt aus Einheit 7 oder in den Lösungsstand der Einheit 7, bevor du mit den Aufgaben beginnst.

1. findOne im Service wirft die NotFoundException (Skript 8.7).
2. Der Controller prüft die id mit ParseIntPipe; importiere sie, NotFoundException fällt weg (Skript 8.8).
3. Schreibe remove im Service; es ruft findOne. DELETE antwortet mit 204; importiere Delete, HttpCode, HttpStatus (Skript 8.9, 8.10).
4. Schreibe die Pipe über @Module, setze sie unter providers ein und importiere BadRequestException, createZodValidationPipe und ZodError (Skript 8.11).

Danach antwortet ein leerer Body wie in Skript 8.12, GET /breweries/abc mit 400.

Beispiel, Schritt 1: findOne im Service wirft die 404

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
findOne(id: number): Brewery {
  const brewery = this.breweries.find((b) => b.id === id);
  if (!brewery) {
    // Jeder Aufrufer bekommt dieselbe 404
    throw new NotFoundException(`Brewery ${id} not found`);
  }
  return brewery;
}
```

Beispiel, Schritt 2: Der Controller prüft die id mit ParseIntPipe

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number): Brewery {
  // Keine ganze Zahl: ParseIntPipe antwortet mit 400
  return this.breweriesService.findOne(id);
}
```

Beispiel, Schritt 3: remove im Service löscht über findOne

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
remove(id: number): Brewery {
  const brewery = this.findOne(id);
  this.breweries.splice(this.breweries.indexOf(brewery), 1);
  return brewery;
}
```

Beispiel, Schritt 3: DELETE /breweries/:id antwortet mit 204

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
@Delete(':id')
@HttpCode(HttpStatus.NO_CONTENT)
remove(@Param('id', ParseIntPipe) id: number): Brewery {
  return this.breweriesService.remove(id);
}
```

Beispiel, Schritt 4: eine Pipe für alle Endpunkte im Format aller Fehler

TYPESCRIPT · SRC/APP.MODULE.TS

```typescript
// Validierungsfehler im Format aller anderen Fehler
const ValidationPipe = createZodValidationPipe({
  createValidationException: (error) =>
    new BadRequestException(
      (error as ZodError).issues.map(
        (issue) => `${issue.path.join('.')}: ${issue.message}`,
      ),
    ),
});
```

Beispiel, Schritt 4: Ein leerer Body ergibt dieselbe Form wie eine 404

JSON

```
{
  "message": [
    "name: Invalid input: expected string, received undefined",
    "city: Invalid input: expected string, received undefined",
    "country: Invalid input: expected string, received undefined"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

Aufgabe 8.1 · Situation und Statuscode zuordnen

ZIEL

Du weißt vor dem Programmieren, welche Antwort jede Anfrage der Aufgabe 8.2 bekommen soll.

AUSGANGSLAGE

Das Beispiel der Brauereien und die Tabelle der Statuscodes auf Folie 8.4 der Vorlesung. Bier 5 ist das Pils aus Skript 7.15.

SCHRITTE

1. Trage in die Tabellen in Skript 8.14 und 8.15 je Situation den Statuscode ein.
2. Begründe jeden Code in einem Satz: Wer muss handeln?
3. Vergleiche deine Tabelle zu zweit.

FERTIG, WENN

- Jede Zeile hat einen Code und eine Begründung.
- Nach Aufgabe 8.2 und Zusatz 8.3 hast du sie mit der API verglichen.

Vorlage zu Aufgabe 8.1: Biere anlegen und löschen

Situation	Statuscode	Begründung
POST /beers mit dem Pils, das Bier ist gültig		
POST /beers mit dem Pils, aber "abv": 25		
DELETE /beers/5, das Bier existiert		
DELETE /beers/5 ein zweites Mal		

Vorlage zu Aufgabe 8.1: Biere lesen und ändern

Situation	Statuscode	Begründung
GET /beers/5, nachdem das Bier gelöscht ist		
GET /beers/abc		
Zu Zusatz 8.3: PATCH /beers/42 mit { "abv": 5.1 }, Bier 42 fehlt		
Zu Zusatz 8.3: PATCH /beers/abc mit { "abv": 5.1 }		

Aufgabe 8.2 · Fehler der Biere erkennbar machen

ZIEL

Als Betreiber möchte ich, dass jede Anfrage einen passenden Statuscode und eine verständliche Meldung liefert.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel der Brauereien. Das Format der Antwort 400 gilt seit Schritt 4 schon für die Biere.

SCHRITTE

1. findOne im BeersService wirft eine NotFoundException mit der Meldung aus Skript 8.17.
2. GET und DELETE prüfen die id mit ParseIntPipe.
3. remove ruft findOne, bevor es ein Bier entfernt.
4. DELETE /beers/:id antwortet mit 204.

FERTIG, WENN

- DELETE /beers/5 ergibt 204 ohne Body, ein zweites Mal 404.
- GET /beers/abc ergibt 400.

Testdaten und Fehlerformat zu Aufgabe 8.2

BEISPIEL

- Lege Bier 5 mit dem Pils aus Skript 7.15 und POST /beers an.
- Nach einem Neustart bekommt das Pils die id 5.
- Das Datenmodell der Biere steht in Skript 7.14.
- Jede Fehlerantwort hat die Form unten; die Meldung nennt das Bier.
- Keine Fehlerantwort enthält einen Stacktrace.

```
{
  "statusCode": 404,
  "message": "Beer 5 not found",
  "error": "Not Found"
}
```

Zusatz 8.3 · Ein Bier mit Fehlern ändern

ZIEL

PATCH /beers/:id liefert dieselben Fehler wie GET und DELETE.

AUSGANGSLAGE

Aufgabe 8.2 ist gelöst, und PATCH /beers/:id aus Zusatz 7.3 ist vorhanden. Ohne Zusatz 7.3 nimmst du den Lösungsstand der Einheit 7.

SCHRITTE

1. Prüfe die id von PATCH mit ParseIntPipe.
2. Entferne die Prüfung auf undefined aus update in Controller und Service; update gibt den Typ Beer zurück.

FERTIG, WENN

- PATCH /beers/abc ergibt 400, PATCH /beers/42 ergibt 404 mit „Beer 42 not found“.
- PATCH /beers/2 mit { "abv": 5.1 } ergibt weiter 200.

Selbstkontrolle mit Postman

Der Ordner der Einheit 8 in der Collection Web-Programmierung prüft das Beispiel und deine Aufgaben. Führe darin den Ordner deiner Aufgabe mit **Run** aus, zum Beispiel „Aufgabe 8.2 – Fehler der Biere erkennbar machen“.

Die Anfragen legen Biere an und löschen Brauereien und Biere im Arbeitsspeicher. Starte den Server deshalb vor jedem Lauf neu. Ein roter Test nennt einen Hinweis auf Ort und Ursache.

Hinweise zu Aufgabe 8.1

- Stufe 1: Die Tabelle auf Folie 8.4 der Vorlesung nennt je Code, wer handeln muss.
- Stufe 2: Frage bei jeder Zeile: Ist die Anfrage falsch, oder fehlt nur die Ressource?
- Stufe 3: Die Antworten der Brauereien aus dem Beispiel zeigen dieselben Situationen; übertrage sie auf die Biere.

Hinweise zu Aufgabe 8.2

- Stufe 1: Das Beispiel der Brauereien in Skript 8.7 bis 8.10 hat denselben Aufbau.
- Stufe 2: Der Rückgabetyp von findOne bestimmt, ob der Controller noch auf undefined prüfen muss.
- Stufe 3: Sende DELETE /beers/5 zweimal und lies, womit remove im Service das Bier sucht.

Hinweise zu Zusatz 8.3

- Stufe 1: An GET und DELETE der Biere steht seit Aufgabe 8.2 schon, was PATCH fehlt.
- Stufe 2: Lies update im Service: Was liefert findOne, wenn es kein Bier gibt?
- Stufe 3: Lass TypeScript übersetzen und lies, welche Zeile einen Typfehler meldet.

Typische Fehler

Symptom	Ursache
GET /beers/1 ergibt 404 mit „Beer 1 not found“	@Param('id') hat den Typ number, aber keine ParseIntPipe; die id bleibt der String "1".
GET /beers/abc ergibt 404 mit „Beer NaN not found“	Der Controller wandelt mit Number(id) statt mit ParseIntPipe.
DELETE /beers/5 ergibt 200 statt 204	@HttpCode(HttpStatus.NO_CONTENT) fehlt an der Methode für DELETE.
DELETE /beers/5 ergibt auch beim zweiten Mal 204	remove sucht selbst mit findIndex, statt findOne aufzurufen.
GET /beers/9 ergibt 500 mit „Internal server error“	findOne wirft new Error statt einer NotFoundException.

Das solltest du jetzt können

- ✓ Zu einer Situation den passenden Statuscode nennen und begründen
- ✓ Mit NotFoundException, ParseIntPipe und @HttpCode die passende Antwort erzeugen
- ✓ Erklären, warum eine Antwort 500 keinen Stacktrace enthält und wo er steht
- ✓ Begründen, warum findOne im Service die NotFoundException wirft
- ✓ Prüfen, ob zwei Fehlerantworten dasselbe Format haben

Als Nächstes: Module und Projektstruktur – wie Biere und Brauereien in eigene Module ziehen und die Biere die Brauereien prüfen.

WEB-PROGRAMMIERUNG · EINHEIT 9

Module und Projektstruktur · Skript

Worum es in dieser Einheit geht

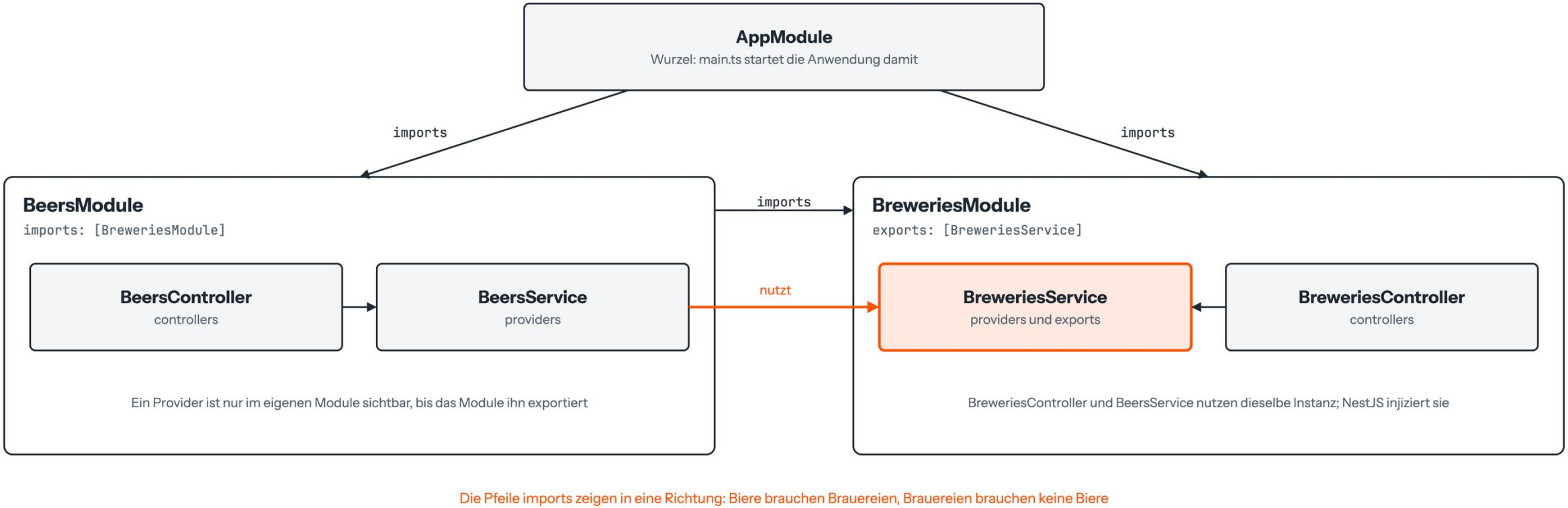
Bisher stehen alle Controller und Services der Beer API in AppModule. In dieser Einheit bekommen Brauereien und Biere je ein eigenes Module. Jedes Module legt fest, was andere Teile nutzen dürfen; so kann ein Team seinen Teil ändern, ohne das andere zu fragen. Die Aufgaben setzen diese Story um:

Als Betreiber möchte ich, dass jedes Bier zu einer existierenden Brauerei gehört, damit Gäste sehen, woher ein Bier kommt.

Am Ende der Einheit kannst du:

- Code in ein eigenes Module verschieben und einen Service exportieren,
- eine Regel umsetzen, die den Service eines anderen Module braucht.

Module importieren die Exporte anderer Module; die Pfeile zeigen in eine Richtung



AppModule ist die Wurzel und importiert BeersModule und BreweriesModule. BeersModule enthält BeersController und BeersService, BreweriesModule enthält BreweriesController und BreweriesService. BreweriesModule exportiert den BreweriesService. BeersModule importiert BreweriesModule; deshalb kann der BeersService den BreweriesService nutzen, NestJS injiziert ihn. BreweriesController und BeersService nutzen dieselbe Instanz. Ein Provider ist nur im eigenen Module sichtbar, bis das Module ihn exportiert. Die Pfeile imports zeigen in eine Richtung: Biere brauchen Brauereien,

Module, imports und exports

Die vier Listen von @Module zeigt Folie 9.6 der Vorlesung. Wer einen Service bekommt, zeigt Folie 9.9. Dependency Injection steht in Skript 6.6.

- main.ts startet die Anwendung mit AppModule. NestJS erzeugt jedes Module, das es von dort über imports erreicht.
- Ein Module, das niemand importiert, gehört nicht zur Anwendung. Seine Endpunkte antworten mit 404.
- Ein Service unter providers ist nur im eigenen Module sichtbar. exports gibt ihn für Module frei, die dieses Module importieren.
- Fehlt ein Service, den ein Constructor verlangt, startet der Server nicht. Die Meldung nennt Klasse und Module.

Dokumentation: https://docs.nestjs.com/modules

Abhängigkeiten zwischen Modulen

Die Biere brauchen die Brauereien, weil ein Bier nur zu einer existierenden Brauerei gehören darf. Die Brauereien brauchen keine Biere.

- Importieren sich zwei Module gegenseitig, bricht NestJS den Start ab. forwardRef() löst das auf; die Beer API braucht es nicht.
- Ein Endpunkt, der zwei Ressourcen braucht, gehört in das Module, das vom anderen abhängt.
- Die Nest CLI überschreibt keine vorhandene Datei: Existiert sie schon, bricht die CLI ab. Module ergänzt sie um Einträge.
- Eine neue Klasse trägt die CLI in das Module ihres Ordners ein, sobald es existiert.

Dokumentation: https://docs.nestjs.com/cli/usages

Beispiel: die Brauereien in drei Schritten

Übernimm das Live-Coding in dein Projekt aus Einheit 8 oder in den Lösungsstand der Einheit 8, bevor du mit den Aufgaben beginnst.

1. Erzeuge das Module mit der Nest CLI (Skript 9.7).
2. Trage Controller und Service der Brauereien im BreweriesModule ein und exportiere den Service (Skript 9.8).
3. Entferne beide aus AppModule, auch ihre Importzeilen oben in der Datei (Skript 9.9).

Danach zeigt das Terminal von start:dev die Zeilen aus Skript 9.10, und GET /breweries liefert weiter die vier Brauereien. Das Verhalten der API ändert sich nicht; als Commit-Typ passt refactor.

Beispiel, Schritt 1: Die Nest CLI erzeugt das Module und trägt es ein

```
$ npx nest generate module breweries
CREATE src/breweries/breweries.module.ts (86 bytes)
UPDATE src/app.module.ts (1369 bytes)
```

Beispiel, Schritt 2: Controller und Service ziehen in das BreweriesModule

TYPESCRIPT · SRC/BREWERIES/BREWERIES.MODULE.TS

```typescript
import { Module } from '@nestjs/common';
import { BreweriesController } from './breweries.controller';
import { BreweriesService } from './breweries.service';

@Module({
  controllers: [BreweriesController],
  providers: [BreweriesService],
  // Andere Module dürfen BreweriesService injizieren
  exports: [BreweriesService],
})
export class BreweriesModule {}
```

Beispiel, Schritt 3: AppModule gibt Controller und Service der Brauereien ab

TYPESCRIPT · SRC/APP.MODULE.TS

```
@Module({
  imports: [BreweriesModule],
  controllers: [AppController, BeersController],
  providers: [
    // Prüft Body, Query und Pfadparameter gegen die Zod-Schemas der DTOs
    { provide: APP_PIPE, useClass: ValidationPipe },
    // Prüft Antworten gegen Zod-Schemas, wenn ein Endpunkt eines angibt
    { provide: APP_INTERCEPTOR, useClass: ZodSerializerInterceptor },
    BeersService,
  ],
})
export class AppModule {}
```

Beispiel, Schritt 3: Das Terminal zeigt eine Zeile je Module und je Controller

```
LOG [InstanceLoader] BreweriesModule dependencies initialized
LOG [InstanceLoader] AppModule dependencies initialized
LOG [RoutesResolver] AppController {/}:
LOG [RoutesResolver] BeersController {/beers}:
LOG [RoutesResolver] BreweriesController {/breweries}:
...
```

Aufgabe 9.1 · Die Biere bekommen ein eigenes Module

ZIEL

Controller und Service der Biere stehen in einem eigenen Module, wie die Brauereien im Beispiel.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel der Brauereien.

ZIELE

- Das BeersModule entsteht mit der Nest CLI im Ordner beers.
- AppModule enthält keine Klasse der Biere mehr.

FERTIG, WENN

- Das Terminal zeigt „BeersModule dependencies initialized“.
- GET /beers liefert weiter die vier Biere.

Aufgabe 9.2 · Nur Biere existierender Brauereien anlegen

ZIEL

Als Betreiber möchte ich, dass jedes Bier zu einer existierenden Brauerei gehört.

AUSGANGSLAGE

Aufgabe 9.1 ist gelöst. Das Datenmodell der Biere steht in Skript 7.14, Testdaten und Regel in Skript 9.13 bis 9.15.

ZIELE

- POST /beers prüft, ob die Brauerei aus breweryId existiert, bevor das Bier entsteht.
- Fehlt die Brauerei, antwortet die API mit 400 oder 404, und die Meldung nennt die Brauerei.
- Du begründest deine Wahl in einem Satz.

FERTIG, WENN

- Das Augustiner Dunkel mit "breweryId": 3 ergibt 201, mit 42 eine Ablehnung.
- Ein Bier einer eben angelegten Brauerei ergibt 201.

Testdaten: Brauereien und Biere nach einem Neustart

id	Brauerei	city	Bier mit dieser breweryId
1	Privatbrauerei Schlumberger	Nattheim	Nattheimer Spezial (Bier 1)
2	Wildbräu Grafing	Grafing bei München	Wildbräu Grafing Helles (Bier 2)
3	Augustiner-Bräu	München	Augustiner Lagerbier Hell (Bier 3)
4	Edelweissbrauerei Farny	Kißlegg	Farny Kristall-Weizen (Bier 4)

Regel und Statuscode zu Aufgabe 9.2

Regel: Ein Bier gehört zu genau einer Brauerei; die Brauerei muss existieren.

Für eine fehlende Brauerei lassen sich zwei Statuscodes vertreten. Die Frage aus Einheit 8 hilft: Wer muss handeln (Folie 8.4 der Vorlesung)?

- 400 Bad Request: Der Client hat einen falschen Wert für breweryId geschickt und korrigiert ihn.
- 404 Not Found: Die Brauerei aus der Anfrage fehlt. Streng nach RFC 9110 meint 404 die Ressource der URL.
- Eine NotFoundException lässt sich fangen und als BadRequestException neu werfen (Einheit 8).

Jede Antwort hat das Fehlerformat aus Einheit 8: statusCode, message und error.

Testdaten zu Aufgabe 9.2

BEISPIEL

- Oben der Body für POST /beers, unten der Body für POST /breweries.
- Das Augustiner Dunkel mit "breweryId": 3 ergibt 201 und die id 5.
- Mit "breweryId": 42 ergibt es 400 oder 404.
- Lege danach mit POST /breweries die Brauerei Gold Ochsen an (id 5).
- Ein Bier mit "breweryId": 5 ergibt dann 201.

```
{
  "name": "Augustiner Dunkel",
  "style": "dunkel",
  "abv": 5.6,
  "breweryId": 3
}

{
  "name": "Brauerei Gold Ochsen",
  "city": "Ulm",
  "country": "Deutschland"
}
```

Zusatz 9.3 · Die Brauerei eines Biers ändern

ZIEL

PATCH /beers/:id prüft dieselbe Regel wie POST. Der Zusatz ist keine Voraussetzung für Einheit 10.

AUSGANGSLAGE

Aufgabe 9.2 ist gelöst, und PATCH /beers/:id aus Zusatz 7.3 ist vorhanden. Ohne Zusatz 7.3 nimmst du den Lösungsstand der Einheit 8 und überträgst Aufgabe 9.1 und 9.2 darauf.

ZIELE

- Enthält der Body eine breweryId, prüft update die Brauerei, bevor ein Feld überschrieben wird.
- Ohne breweryId im Body ändert PATCH weiter nur die übrigen Felder.

FERTIG, WENN

- PATCH /beers/2 mit { "breweryId": 42 } ergibt 400 oder 404.
- GET /beers/2 zeigt danach weiter "breweryId": 2.

Zusatz 9.4 · Die Biere einer Brauerei abrufen

ZIEL

GET /breweries/:breweryId/beers liefert alle Biere einer Brauerei. Der Zusatz ist keine Voraussetzung für Einheit 10.

AUSGANGSLAGE

Aufgabe 9.2 ist gelöst.

ZIELE

- Der Server startet ohne zirkuläre Abhängigkeit zwischen den Modulen.
- Eine id, die keine Zahl ist, ergibt 400.

FERTIG, WENN

- Nach dem Augustiner Dunkel liefert GET /breweries/3/beers es und das Augustiner Lagerbier Hell.
- GET /breweries/abc/beers ergibt 400.

Selbstkontrolle mit Postman

Der Ordner der Einheit 9 in der Collection Web-Programmierung prüft das Beispiel und deine Aufgaben. Führe darin den Ordner deiner Aufgabe mit **Run** aus, zum Beispiel „Aufgabe 9.2 – Nur Biere existierender Brauereien anlegen“.

Die Anfragen legen Brauereien und Biere an und löschen eine Brauerei im Arbeitsspeicher. Starte den Server deshalb vor jedem Lauf neu. Ein roter Test nennt einen Hinweis auf Ort und Ursache. Aufgabe 9.1 ändert kein Verhalten; ihr Ordner prüft, dass die Biere weiter antworten.

Hinweise zu Aufgabe 9.1

- Stufe 1: Das Beispiel der Brauereien in Skript 9.6 bis 9.9 hat denselben Aufbau.
- Stufe 2: Vergleiche die Zeilen InstanceLoader und RoutesResolver im Terminal mit Skript 9.10.
- Stufe 3: Antwortet GET /beers mit 404, suche den BeersController in den Listen controllers beider Module.

Hinweise zu Aufgabe 9.2

- Stufe 1: Ob es eine Brauerei gibt, weiß seit Einheit 8 eine Methode der Brauereien; Skript 9.3 zeigt den Weg.
- Stufe 2: Startet der Server nicht, nennt die erste Zeile mit ERROR die Klasse, den fehlenden Parameter und das Module.
- Stufe 3: Sende das Bier mit 42 und verfolge, welche Methode es speichert und was sie vorher prüft.

Hinweise zu Zusatz 9.3

- Stufe 1: Die Prüfung aus Aufgabe 9.2 gilt auch für update.
- Stufe 2: Im Body von PATCH ist breweryId optional; die Prüfung läuft nur, wenn das Feld da ist.
- Stufe 3: Sende PATCH /beers/2 mit breweryId 42 und lies danach GET /beers/2: Welche Zeile in update hat schon geschrieben?

Hinweise zu Zusatz 9.4

- Stufe 1: Das Bild in Skript 9.3 zeigt, welches Module welches kennt.
- Stufe 2: Überlege, welchen Service der Endpunkt braucht und welche Module ihn bekommen können.
- Stufe 3: Prüfe für jedes Module, ob es dafür ein Module importieren müsste, das schon von ihm abhängt.

Typische Fehler

Symptom	Ursache
Server startet nicht: „Nest can't resolve dependencies of the BeersService (?)“	imports: [BreweriesModule] im BeersModule oder exports im BreweriesModule fehlt.
POST /beers mit "breweryId": 5 ergibt „Brewery 5 not found“, obwohl Brauerei 5 eben angelegt wurde	BreweriesService steht unter providers im BeersModule statt im Import; das ist eine zweite Instanz.
GET /beers ergibt 404 mit „Cannot GET /beers“	BeersController ist aus AppModule entfernt, aber im BeersModule nicht eingetragen.
Server startet nicht: „Nest cannot create the BeersModule instance“	Das BreweriesModule importiert das BeersModule; die beiden Module bilden eine zirkuläre Abhängigkeit.
nest generate service beers meldet „A merge conflicted on path“	beers.service.ts existiert schon; Controller und Service ziehen von Hand um.

Das solltest du jetzt können

- ✓ Ein Module mit der Nest CLI erzeugen und Controller und Service hineinverschieben
- ✓ Einen Service exportieren und in einem anderen Module injizieren
- ✓ Die Meldung „can't resolve dependencies“ auf einen fehlenden Import oder Export zurückführen
- ✓ Eine Regel umsetzen, die den Service eines anderen Module braucht
- ✓ Begründen, warum die Biere von den Brauereien abhängen und nicht umgekehrt

Als Nächstes: Anforderungen in Code übersetzen – an der Regel für Brauereien alle Fälle aufzählen und Bewertungen für Biere bauen.

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 10

Anforderungen in Code übersetzen · Skript

Worum es in dieser Einheit geht

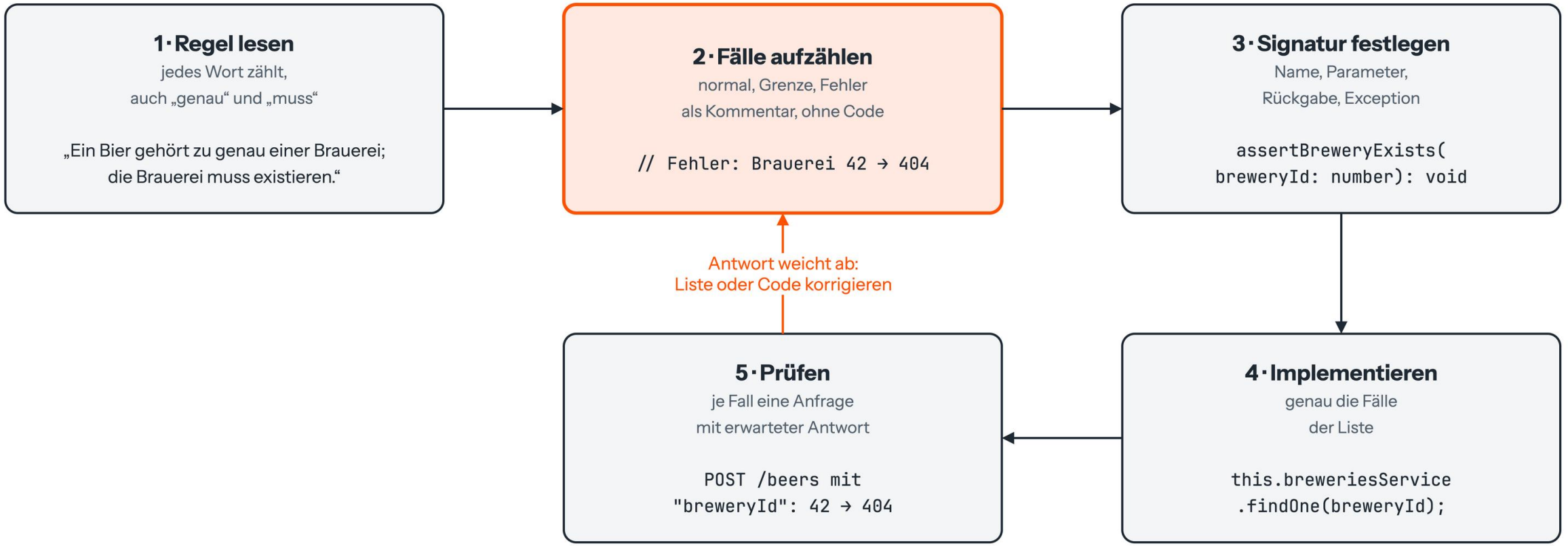
Eine Anforderung ist fertig, wenn ihre Akzeptanzkriterien erfüllt sind. Damit verstehen der Betreiber und das Entwicklungsteam dasselbe unter „fertig“. In dieser Einheit übersetzt du Regeln aus der Anforderung in Fälle und die Fälle in Code. Die Aufgaben setzen diese Story um:

Als Gast möchte ich ein Bier mit 1 bis 5 Sternen bewerten, damit andere Gäste sehen, was gut ankommt.

Am Ende der Einheit kannst du:

- zu einer Regel alle Fälle aufzählen, bevor du Code schreibst,
- Bewertungen für Biere mit drei Regeln umsetzen.

Fünf Schritte führen von der Regel zur geprüften Methode



Fünf Schritte im Kreis, je Schritt ein Beispiel an der Regel für Brauereien. 1 Regel lesen, jedes Wort zählt, auch „genau“ und „muss“: „Ein Bier gehört zu genau einer Brauerei; die Brauerei muss existieren.“ 2 Fälle aufzählen: normal, Grenze, Fehler, als Kommentar, ohne Code: „Fehler: Brauerei 42 → 404“. 3 Signatur festlegen: Name, Parameter, Rückgabe, Exception: assertBreweryExists(breweryId: number): void. 4 Implementieren: genau die Fälle der Liste: this.breweriesService.findOne(breweryId). 5 Prüfen: je Fall eine Anfrage mit erwarteter Antwort: POST /beers mit "breweryId": 42 → 404. Weicht eine

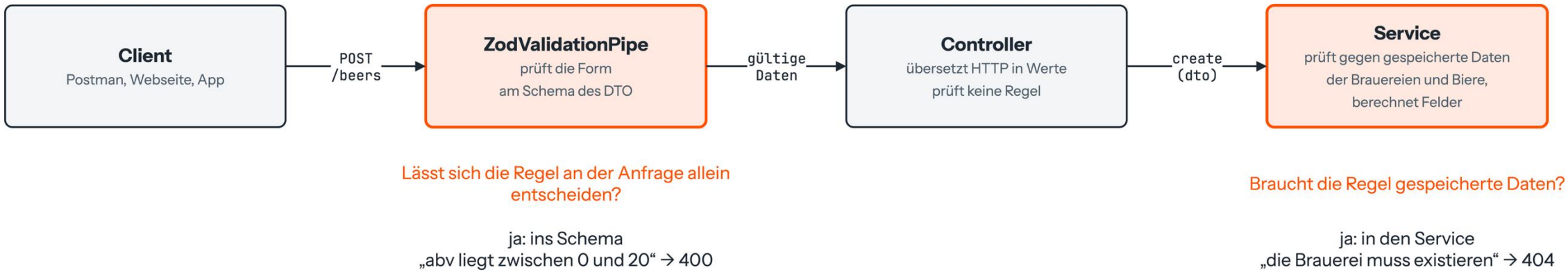
Von der Regel zu den Fällen

Die Teile einer User Story zeigt Folie 10.5 der Vorlesung, die drei Arten von Fällen Folie 10.7.

- Ein Akzeptanzkriterium nennt eine Anfrage mit Werten aus den Beispieldaten und die erwartete Antwort.
- Ein Fall steht als Kommentar in der Form „Art: Eingabe → erwartete Antwort“ über der Methode.
- Grenzfälle findest du am Rand eines Bereichs, beim ersten Eintrag und bei leeren Daten.
- Ein Fall, für den es keine Anfrage gibt, ist nicht geprüft.

Dokumentation: https://www.mountaingoatsoftware.com/agile/user-stories

Das Schema prüft die Form, der Service prüft gegen gespeicherte Daten



Stünde eine Regel im Controller, bräuchte jeder Endpunkt mit derselben Regel eine eigene Kopie, zum Beispiel POST und PATCH.

Weg einer Anfrage POST /beers von links nach rechts: Client, ZodValidationPipe prüft die Form am Schema des DTO, Controller übersetzt HTTP in Werte und prüft keine Regel, Service prüft gegen gespeicherte Daten der Brauereien und Biere und berechnet Felder. Unter der Pipe die Frage „Lässt sich die Regel an der Anfrage allein entscheiden?“, ja: ins Schema, mit dem Beispiel „abv liegt zwischen 0 und 20“ → 400. Unter dem Service die Frage „Braucht die Regel gespeicherte Daten?“, ja: in den Service, mit dem Beispiel „die Brauerei muss existieren“ → 404. Stünde eine Regel im Controller, bräuchte jeder

Ort, Signatur und berechnete Felder

Der Weg einer Anfrage durch NestJS steht in Skript 5.6, die Pipe in Einheit 7.

- Die Pipe prüft vor dem Service. Verletzt eine Anfrage zwei Regeln, nennt die Antwort nur die im Schema.
- Die Signatur legt Name, Parameter, Rückgabe und Exception fest. Eine Methode, die nur prüft, gibt void zurück oder wirft.
- Ein berechnetes Feld speicherst du nicht. Der Service leitet es bei jeder Antwort aus gespeicherten Daten ab.

Dokumentation: https://docs.nestjs.com/pipes

Beispiel: die Regel für Brauereien als Methode mit Fällen

Übernimm das Live-Coding in dein Projekt aus Einheit 9, bevor du mit den Aufgaben beginnst. Fehlt dir update aus Zusatz 7.3 und 9.3, nimm den Lösungsstand der Einheit 9.

1. Schreibe die Fälle der Regel als Kommentar über remove in die Klasse BeersService (Skript 10.8).
2. Schreibe darunter die Methode assertBreweryExists (Skript 10.8).
3. Rufe sie in create als erste Zeile und in update vor Object.assign auf (Skript 10.9).
4. Sende die Anfragen aus Skript 10.10.

Die Antworten sind dieselben wie vorher. Das Verhalten der API ändert sich nicht; als Commit-Typ passt refactor.

Beispiel, Schritt 2 und 3: Fälle und Signatur im BeersService

TYPESCRIPT · SRC/BEERS/BEERS.SERVICE.TS

```
// R2: Ein Bier gehört zu genau einer Brauerei;
// die Brauerei muss existieren.
// Normal: Brauerei 3 existiert → 201
// Grenze: Brauerei eben mit POST /breweries angelegt → 201
// Grenze: breweryId 0 → 404, es gibt keine Brauerei 0
// Fehler: Brauerei 42 → 404
// Fehler: breweryId fehlt bei POST → 400 (Schema)
// Normal: breweryId fehlt bei PATCH → keine Prüfung
// Lücke: Brauerei gelöscht, ihre Biere bleiben (Fremdschlüssel in Einheit 12)
private assertBreweryExists(breweryId: number): void {
  // findOne wirft 404, wenn die Brauerei fehlt. 404 statt 400, weil
  // die Anfrage gültig ist und nur die Brauerei nicht existiert
  this.breweriesService.findOne(breweryId);
}
```

Beispiel, Schritt 4: update prüft nur, wenn breweryId im Body steht

TYPESCRIPT · SRC/BEERS/BEERS.SERVICE.TS

```
update(id: number, dto: UpdateBeerDto): Beer {
  const beer = this.findOne(id);
  // R2 auch beim Ändern, bevor ein Feld überschrieben wird
  if (dto.breweryId !== undefined) {
    this.assertBreweryExists(dto.breweryId);
  }
  // Nur die Felder aus dem Body überschreiben
  Object.assign(beer, dto);
  return beer;
}
```

Beispiel, Schritt 5: Jeder Fall wird eine Anfrage

Fall	Anfrage	Antwort
Normal: Brauerei 3	POST /beers mit "breweryId": 3	201
Grenze: eben angelegt	POST /breweries, dann POST /beers mit "breweryId": 5	201
Grenze: breweryId 0	PATCH /beers/3 mit "breweryId": 0	404, Brewery 0 not found
Fehler: Brauerei 42	POST /beers mit "breweryId": 42	404, Brewery 42 not found
Ohne breweryId	POST /beers ohne das Feld; PATCH /beers/2 mit "abv": 5.1	400 beim Anlegen; 200 ohne Prüfung beim Ändern

Aufgabe 10.1 · Ein Bier bewerten

ZIEL

Als Gast möchte ich ein Bier mit 1 bis 5 Sternen bewerten.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel. Datenmodell und Regeln stehen in Skript 10.12 und 10.13, Testdaten in Skript 10.14.

ZIELE

- Über der Methode, die eine Bewertung anlegt, stehen die Fälle zur Regel für die Sterne.
- Ein RatingsModule mit RatingsService speichert die Bewertungen; der Server startet ohne Fehler.
- POST /beers/:id/ratings legt eine Bewertung an, GET /beers/:id/ratings liefert die Bewertungen eines Biers.

FERTIG, WENN

- 4 Sterne für Bier 1 ergeben 201; 0, 6 und 3.5 ergeben 400.
- Eine Bewertung für Bier 42 ergibt 404.

Datenmodell einer Bewertung

Der Server vergibt id und createdAt. Bis Einheit 13 schickt der Client die userId im Body; danach kommt sie aus dem Login.

- id: Ganzzahl, Kennung der Bewertung
- beerId: Ganzzahl, id des bewerteten Biers; steht im Pfad
- userId: Ganzzahl, id des Gasts; Pflicht
- score: Ganzzahl, Sterne von 1 bis 5; Pflicht
- comment: String, Freitext; optional
- createdAt: String, Zeitpunkt als ISO 8601, zum Beispiel 2026-10-25T17:36:18.858Z

Regeln zu den Aufgaben 10.1 bis 10.3

Für die Bewertungen gelten drei Regeln aus der Anforderung, im Wortlaut:

- Aufgabe 10.1: score ist eine ganze Zahl von 1 bis 5.
- Aufgabe 10.2: Ein User bewertet ein Bier höchstens einmal; eine zweite Bewertung ersetzt die erste.
- Aufgabe 10.3: Die Durchschnittsbewertung eines Biers wird auf eine Nachkommastelle gerundet.

Im Datenmodell eines Biers steht dazu das Feld averageRating: Zahl oder null, Durchschnitt aller Bewertungen; wird berechnet, nicht gespeichert; null ohne Bewertung. Die Regel aus Einheit 9 gilt weiter: Ein Bier gehört zu genau einer Brauerei; die Brauerei muss existieren.

Testdaten zu Aufgabe 10.1

BEISPIEL

- Nach einem Neustart gibt es die Biere 1 bis 4 und keine Bewertung.
- Bier 1 ist das Nattheimer Spezial, Bier 2 das Wildbräu Grafing Helles.
- Der Body rechts für POST /beers/1/ratings ergibt 201.
- Mit "score": 0, 6 oder 3.5 ergibt er 400.
- POST antwortet mit der Bewertung unten rechts; GET /beers/1/ratings liefert sie in einer Liste.

```
{
  "userId": 1,
  "score": 4,
  "comment": "Süffig"
}
```

```
{"id":1,"beerId":1,"userId":1,"score":4,
 "comment":"Süffig",
 "createdAt":"2026-10-25T17:36:18.858Z"}
```

Aufgabe 10.2 · Eine zweite Bewertung ersetzt die erste

ZIEL

Jeder Gast zählt bei einem Bier nur einmal.

AUSGANGSLAGE

Aufgabe 10.1 ist gelöst.

ZIELE

- Die Liste der Fälle nennt die Regel zur zweiten Bewertung mit normalem Fall, Grenze und Fehler.
- Bewertet ein User ein Bier noch einmal, ersetzt die neue Bewertung seine alte.
- Die Bewertungen anderer User bleiben unverändert.

FERTIG, WENN

- Nach 4 und 5 von User 1 zeigt GET /beers/1/ratings von ihm nur die 5.
- Die Bewertung von User 2 bleibt unverändert.

Aufgabe 10.3 · Den Durchschnitt anzeigen

ZIEL

Gäste sehen an jedem Bier, wie gut es ankommt.

AUSGANGSLAGE

Aufgabe 10.2 ist gelöst.

ZIELE

- Die Liste der Fälle nennt die Regel zum Durchschnitt mit normalem Fall und Grenzen.
- GET /beers/:id und GET /beers zeigen bei jedem Bier das Feld averageRating.
- Der Wert wird bei jeder Antwort berechnet und nirgends gespeichert.

FERTIG, WENN

- Nach 5, 4 und 4 von drei Usern zeigt GET /beers/1 "averageRating": 4.3.
- GET /beers/2 zeigt "averageRating": null.
- GET /beers zeigt averageRating bei jedem Bier.

Testdaten zu Aufgabe 10.2 und 10.3

BEISPIEL

- Starte den Server neu und sende die Bewertungen rechts an POST /beers/1/ratings.
- Die zweite Bewertung von User 1 ersetzt seine erste.
- Danach hat Bier 1 die Bewertungen 5, 4 und 4.
- GET /beers/1 zeigt den gerundeten Durchschnitt als Zahl.
- Alle anderen Biere haben null.

```
{ "userId": 1, "score": 4 }
{ "userId": 1, "score": 5 }
{ "userId": 2, "score": 4 }
{ "userId": 3, "score": 4 }

{"id":1,"name":"Nattheimer Spezial",
 "style":"spezial","abv":5.4,"breweryId":1,
 "averageRating":4.3}
```

Zusatz 10.4 · Fälle, die die Story nicht nennt

ZIEL

Die Story nennt sieben Kriterien. Du findest Fälle, die sie offenlässt, und prüfst sie an deinem Server. Der Zusatz braucht keinen Code und ist keine Voraussetzung für Einheit 11.

AUSGANGSLAGE

Aufgabe 10.3 ist gelöst. Vorlage in Skript 10.19.

ZIELE

- Zu jeder der drei Regeln steht mindestens eine Grenze oder ein Fehler, der in Skript 10.11 bis 10.17 fehlt.
- Je Fall stehen Anfrage, erwartete Antwort und gemessene Antwort in der Tabelle.

FERTIG, WENN

- Die Tabelle hat neben der Beispielzeile mindestens vier eigene Fälle mit gemessener Antwort.
- Weicht eine Antwort ab, steht daneben, ob Liste oder Code falsch ist.

Vorlage zu Zusatz 10.4

Regel	Art	Anfrage	Erwartete Antwort	Antwort deines Servers
Die Brauerei muss existieren	Grenze	PATCH /beers/3 mit "breweryId": 0	404	404, Brewery 0 not found
Sterne von 1 bis 5				
Zweite Bewertung ersetzt die erste				
Durchschnitt mit einer Nachkommastelle				

Selbstkontrolle mit Postman

Der Ordner der Einheit 10 in der Collection Web-Programmierung prüft das Beispiel und deine Aufgaben. Führe darin den Ordner deiner Aufgabe mit **Run** aus, zum Beispiel „Aufgabe 10.2 – Eine zweite Bewertung ersetzt die erste“.

Die Anfragen legen Brauereien, Biere und Bewertungen im Arbeitsspeicher an. Starte den Server deshalb vor jedem Lauf neu. Ein roter Test nennt einen Hinweis auf Ort und Ursache. Der Ordner zu Zusatz 10.4 enthält Fälle zum Vergleich mit deiner Tabelle; sieh ihn dir erst an, wenn deine Tabelle fertig ist.

Hinweise zu Aufgabe 10.1

- Stufe 1: Stelle für die Regel zu den Sternen die beiden Fragen aus Skript 10.5.
- Stufe 2: Eine Bewertung hängt an einem Bier. Welches Module bedient den Pfad /beers/:id, und was muss es importieren?
- Stufe 3: Sende die Bewertung für Bier 42 und verfolge, welche Methode sie speichert und was vorher geprüft wird.

Hinweise zu Aufgabe 10.2

- Stufe 1: Lies die Regel Wort für Wort: Was macht zwei Bewertungen zu „derselben“?
- Stufe 2: Zwei Felder einer Bewertung entscheiden, ob es schon eine von diesem User für dieses Bier gibt.
- Stufe 3: Sende zwei Bewertungen von User 1 für Bier 1 und lies GET /beers/1/ratings: Wie viele Einträge siehst du?

Hinweise zu Aufgabe 10.3

- Stufe 1: Folie 10.10 der Vorlesung zeigt, warum der Durchschnitt nicht gespeichert wird.
- Stufe 2: Prüfe den Typ von averageRating in der Antwort: Zahl oder String?
- Stufe 3: Suche alle Methoden im BeersService, die ein Bier zurückgeben, und prüfe, ob jede das Feld ergänzt.

Hinweise zu Zusatz 10.4

- Stufe 1: Die Tabelle auf Folie 10.7 der Vorlesung nennt, wo Grenzen liegen.
- Stufe 2: Welche Werte liegen direkt an den Grenzen einer Regel, und welche haben den falschen Typ?
- Stufe 3: Welche Durchschnitte muss die Rundung entscheiden? Rechne einen aus und sende die Bewertungen dazu.

Typische Fehler

Symptom	Ursache
Server startet nicht: „TS2322: Type 'string' is not assignable to type 'number'“	toFixed liefert einen String; averageRating ist eine Zahl.
GET /beers/1 zeigt "averageRating": 4.333333333333333	Der Durchschnitt ist nicht gerundet.
POST /beers/42/ratings ergibt 201	Niemand prüft, ob es das Bier gibt, bevor die Bewertung gespeichert wird.
GET /beers/1/ratings zeigt zwei Bewertungen von User 1	Die alte Bewertung desselben Users wird nicht entfernt.
Server startet nicht: „Nest cannot create the RatingsModule instance“	RatingsModule und BeersModule importieren sich gegenseitig; die Bewertungen brauchen keine Biere.

Das solltest du jetzt können

- ✓ Ein Akzeptanzkriterium als Anfrage mit erwarteter Antwort formulieren
- ✓ Zu einer Regel normale Fälle, Grenzen und Fehler aufzählen, bevor du Code schreibst
- ✓ Entscheiden, ob eine Regel ins Schema oder in den Service gehört
- ✓ Eine Signatur festlegen, bevor du implementierst
- ✓ Ein berechnetes Feld in jeder Antwort ausgeben, ohne es zu speichern

Als Nächstes: REST-Design und API-Dokumentation – Biere filtern und sortieren und die Beer API mit OpenAPI beschreiben.

WEB-PROGRAMMIERUNG · EINHEIT 11

REST-Design und API-Dokumentation · Skript

Worum es in dieser Einheit geht

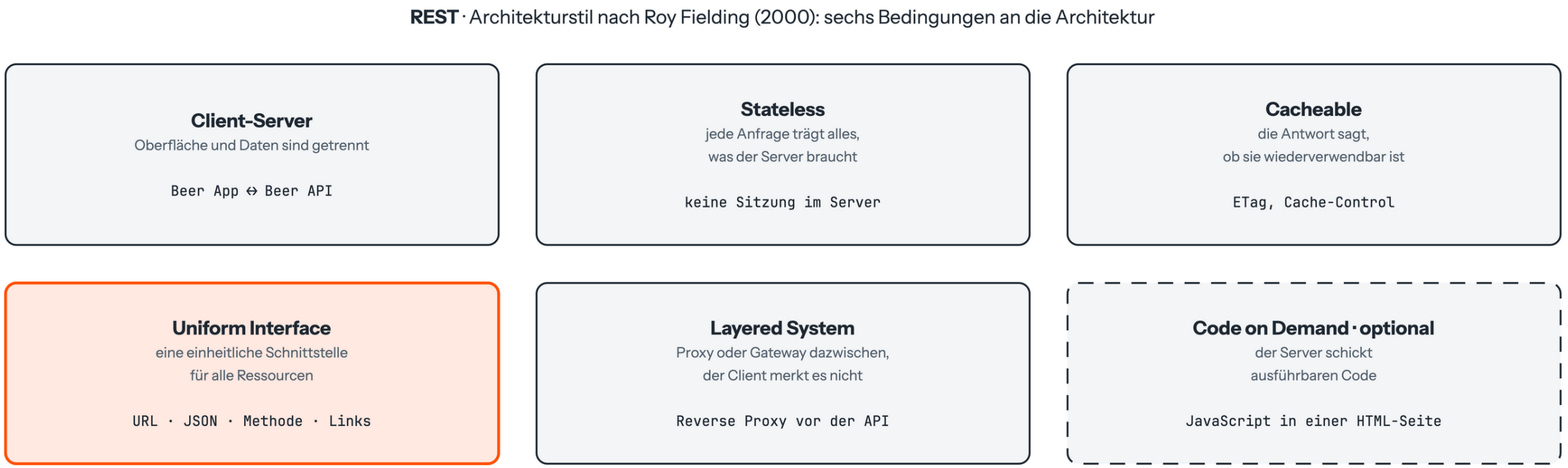
Die Brauereien zeigen Biere und Bewertungen auf ihren eigenen Webseiten. Dafür brauchen sie eine API, deren URLs einem bekannten Stil folgen, und eine Beschreibung, auf die sie sich verlassen können. Die Aufgaben setzen diese Story um:

Als Brauerei möchte ich Biere filtern und sortieren und die API anhand einer Dokumentation anbinden, damit ich die Daten auf meiner eigenen Webseite zeige.

Am Ende der Einheit kannst du:

- eine API nach den Constraints von REST und dem Richardson Maturity Model einordnen,
- Filter und Sortierung als Query-Parameter bauen,
- die Beer API mit OpenAPI beschreiben.

REST ist ein Architekturstil aus sechs Constraints, einer davon optional



Sechs Karten in zwei Reihen unter der Überschrift „REST · Architekturstil nach Roy Fielding (2000): sechs Bedingungen an die Architektur“. Client-Server: Oberfläche und Daten sind getrennt, Beer App und Beer API. Stateless: jede Anfrage trägt alles, was der Server braucht, keine Sitzung im Server. Cacheable: die Antwort sagt, ob sie wiederverwendbar ist, ETag und Cache-Control. Uniform Interface, hervorgehoben: eine einheitliche Schnittstelle für alle Ressourcen, URL, JSON, Methode, Links. Layered System: Proxy oder Gateway dazwischen, der Client merkt es nicht, Reverse Proxy vor der API. Code

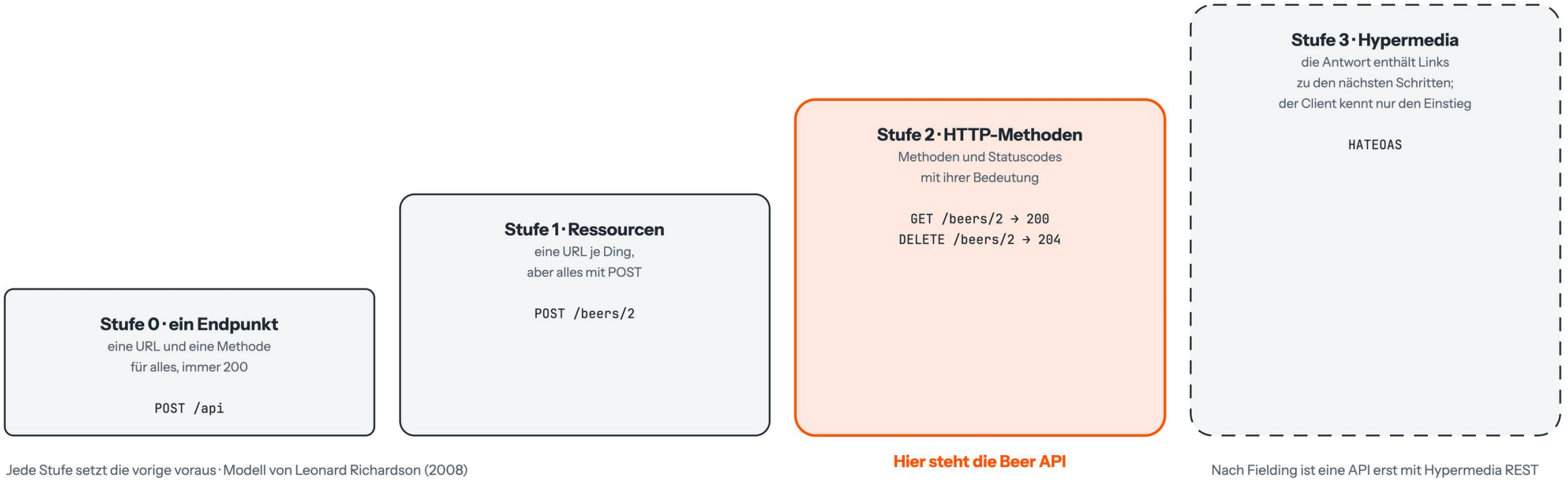
REST legt Bedingungen an die Architektur fest

Roy Fielding hat REST im Jahr 2000 beschrieben. HTTP ist die übliche Umsetzung. Eine API heißt RESTful, wenn sie die Constraints einhält. Die vier Teile des Uniform Interface, URL-Design und Methoden stehen auf den Folien 11.6 bis 11.9 der Vorlesung.

- Eine Ressource ist ein Ding der Fachlichkeit, etwa ein Bier; übertragen wird eine Repräsentation, hier JSON.
- Stateless: Der Server führt keine Sitzung je Client; gespeicherte Biere sind Zustand der Ressourcen.
- Ein Filter wählt aus einer Sammlung aus; findet er nichts, antwortet die API mit 200 und [].

Dokumentation: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Das Richardson Maturity Model hat vier Stufen; die Beer API steht auf Stufe 2



Treppe aus vier Stufen von links nach rechts. Stufe 0, ein Endpunkt: eine URL und eine Methode für alles, immer 200, POST /api. Stufe 1, Ressourcen: eine URL je Ding, aber alles mit POST, POST /beers/2. Stufe 2, HTTP-Methoden, hervorgehoben mit „Hier steht die Beer API“: Methoden und Statuscodes mit ihrer Bedeutung, GET /beers/2 → 200, DELETE /beers/2 → 204. Stufe 3, Hypermedia, gestrichelt: die Antwort enthält Links zu den nächsten Schritten; der Client kennt nur den Einstieg, HATEOAS. Darunter: Jede Stufe setzt die vorige voraus, Modell von Leonard Richardson (2008); nach Fielding ist eine API

JSON-APIs auf Stufe 2 beschreiben ihre URLs mit OpenAPI

HATEOAS (Hypermedia as the Engine of Application State) heißt: Der Client kennt nur den Einstieg und folgt Links aus den Antworten, wie ein Mensch auf einer Webseite. Die Beer API enthält keine Links. Die meisten JSON-APIs stehen wie sie auf Stufe 2.

- Auf Stufe 2 baut der Client die URLs selbst. Welche es gibt, erfährt er aus der Beschreibung.
- OpenAPI beschreibt Pfade, Parameter, Body und Antworten je Statuscode als JSON oder YAML.
- Swagger UI zeigt die Beschreibung an, Postman importiert sie als Collection.
- Ein Breaking Change, etwa ein umbenanntes Feld, bricht Clients, die sich an die Beschreibung halten.

Dokumentation: https://martinfowler.com/articles/richardsonMaturityModel.html

Die Beschreibung entsteht beim Start aus dem Code

Welcher Decorator was in der Beschreibung erzeugt, zeigt Folie 11.13 der Vorlesung. Die Beer API liefert die Beschreibung unter http://localhost:3000/api-json, Swagger UI unter http://localhost:3000/api.

- @ZodResponse trägt das Schema der Antwort ein und prüft beim Kompilieren, ob die Methode diesen Typ zurückgibt.
- Ein TypeScript-Typ oder Interface existiert zur Laufzeit nicht und erscheint deshalb nicht.
- Query-Parameter kommen als Strings an. Ein Schema für Zahlen muss sie umwandeln.
- Zod entfernt unbekannte Query-Parameter ohne Fehler.

Dokumentation: https://docs.nestjs.com/openapi/introduction

Beispiel: Brauereien filtern, sortieren und beschreiben

Übernimm das Live-Coding in dein Projekt aus Einheit 10, bevor du mit den Aufgaben beginnst.

1. Lege das Schema für die Query-Parameter an (Skript 11.9).
2. Ergänze findAll im BreweriesService um Filter und Sortierung (Skript 11.10).
3. Lege die Schemas für Antwort und Fehler an (Skript 11.11 und 11.12).
4. Ergänze die Decorators im BreweriesController (Skript 11.13); create bekommt status: 201.
5. Setze @ApiTags('breweries') über die Klasse; remove bekommt @ApiNoContentResponse(), Rückgabetyp void und kein return.
6. Ergänze die Adresse des Servers in main.ts und importiere die Beschreibung (Skript 11.14).

Swagger UI zeigt danach die Gruppe breweries mit Schemas für Antwort und Fehler.

Beispiel, Schritt 1: Schema für die Query-Parameter

TYPESCRIPT · SRC/BREWERIES/DTO/BREWERY-QUERY.DTO.TS

```typescript
import { createZodDto } from 'nestjs-zod';
import { z } from 'zod';

export const BreweryQuerySchema = z.object({
  city: z.string().optional(),
  sort: z.enum(['name', 'city']).optional(),
  order: z.enum(['asc', 'desc']).default('asc'),
});

export class BreweryQueryDto extends createZodDto(BreweryQuerySchema) {}
```

Beispiel, Schritt 2: Der Service filtert und sortiert eine Kopie

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
findAll(query: BreweryQueryDto): Brewery[] {
  // filter liefert eine neue Liste; sort ändert nur sie
  const breweries = this.breweries.filter(
    (brewery) => !query.city || brewery.city === query.city,
  );
  if (query.sort) {
    const field = query.sort;
    const direction = query.order === 'desc' ? -1 : 1;
    const byField = (a: Brewery, b: Brewery) =>
      a[field].localeCompare(b[field]) * direction;
    breweries.sort(byField);
  }
  return breweries;
}
```

Beispiel, Schritt 3: Schema für die Antwort

TYPESCRIPT · SRC/BREWERIES/DTO/BREWERY.DTO.TS

```typescript
import { createZodDto } from 'nestjs-zod';
import { z } from 'zod';
import { CreateBrewerySchema } from './create-brewery.dto';

// Antwort: die Felder aus dem Body und die id, die der Service vergibt
export const BrewerySchema = z.object({
  id: z.number().int(),
  ...CreateBrewerySchema.shape,
});

export class BreweryDto extends createZodDto(BrewerySchema) {}
```

Beispiel, Schritt 3: Schema für alle Fehler der API

TYPESCRIPT · SRC/ERROR.DTO.TS

```typescript
import { createZodDto } from 'nestjs-zod';
import { z } from 'zod';

// Format aller Fehler: NestJS-Exceptions und die ValidationPipe in AppModule
export const ErrorSchema = z.object({
  message: z.union([z.string(), z.array(z.string())]),
  error: z.string(),
  statusCode: z.number().int(),
});

export class ErrorDto extends createZodDto(ErrorSchema) {}
```

Beispiel, Schritt 4: Decorators an den Endpunkten

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
@Get()
@ZodResponse({ status: 200, type: [BreweryDto] })
@ApiBadRequestResponse({ type: ErrorDto })
findAll(@Query() query: BreweryQueryDto) {
  return this.breweriesService.findAll(query);
}

@Get(':id')
@ZodResponse({ status: 200, type: BreweryDto })
@ApiBadRequestResponse({ type: ErrorDto })
@ApiNotFoundResponse({ type: ErrorDto })
findOne(@Param('id', ParseIntPipe) id: number) {
  // Keine ganze Zahl: ParseIntPipe antwortet mit 400
  return this.breweriesService.findOne(id);
}
```

Beispiel, Schritt 5: Adresse des Servers und Import in Postman

BEISPIEL

- Ohne addServer erreicht keine importierte Anfrage den Server.
- In Postman: **Import**, URL http://localhost:3000/api-json, als Postman Collection.
- Ersetze Platzhalter wie <number> und <string> durch Werte, bevor du sendest.
- GET /breweries mit city=München liefert nur Augustiner-Bräu.

```
const config = new DocumentBuilder()
  .setTitle('Beer API')
  .addServer('http://localhost:3000')
  .build();
```

Aufgabe 11.1 · Biere filtern

ZIEL

Eine Brauerei zeigt auf ihrer Webseite nur ihre eigenen Biere oder nur einen Bierstil.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel. Testdaten und Query-Parameter stehen in Skript 11.16 und 11.17.

ZIELE

- Ein Schema legt die Query-Parameter für GET /beers fest.
- GET /beers filtert nach style, nach breweryId und nach beiden zusammen.
- Ein Stil, den die Beer App nicht kennt, ergibt 400.

FERTIG, WENN

- GET /beers?style=helles&breweryId=3 liefert nur das Augustiner Lagerbier Hell.
- GET /beers?style=lager antwortet mit 400.

Testdaten zu Aufgabe 11.1 und 11.2

id	name	style	abv	breweryId
1	Nattheimer Spezial	spezial	5.4	1 (Privatbrauerei Schlumberger)
2	Wildbräu Grafing Helles	helles	5	2 (Wildbräu Grafing)
3	Augustiner Lagerbier Hell	helles	5.2	3 (Augustiner-Bräu)
4	Farny Kristall-Weizen	weizen	5.3	4 (Edelweissbrauerei Farny)

Query-Parameter für GET /beers

Alle Parameter sind optional. Ohne Parameter liefert GET /beers alle Biere in der Reihenfolge der Beispieldaten.

- style: Bierstil, einer von helles, pils, spezial, weizen, dunkel
- breweryId: Ganzzahl, id einer Brauerei
- sort: Sortierfeld, einer von name, abv, averageRating
- order: asc oder desc; ohne Angabe asc

Mehrere Filter gelten zusammen. Biere ohne Bewertung haben averageRating null und stehen bei jeder Reihenfolge am Ende. Ein unbekannter Stil oder ein unbekanntes Sortierfeld ergibt 400. Die Liste im Service bleibt nach jeder Anfrage in ihrer Reihenfolge.

Aufgabe 11.2 · Biere sortieren

ZIEL

Eine Brauerei zeigt die beliebtesten Biere zuerst.

AUSGANGSLAGE

Aufgabe 11.1 ist gelöst. Testdaten in Skript 11.19.

ZIELE

- GET /beers sortiert nach name, abv oder averageRating, auf- und absteigend.
- Biere ohne Bewertung stehen bei jeder Reihenfolge am Ende.
- Ein unbekanntes Sortierfeld ergibt 400.

FERTIG, WENN

- Nach den Bewertungen aus Skript 11.19 liefert ?sort=averageRating&order=desc die Biere 3, 1, 2, 4.
- GET /beers?sort=price antwortet mit 400.

Testdaten zu Aufgabe 11.2

BEISPIEL

- Starte den Server neu und sende die drei Bewertungen rechts.
- Bier 3 hat danach den Durchschnitt 4.5, Bier 1 den Durchschnitt 2.
- Die Biere 2 und 4 haben keine Bewertung.
- ?sort=averageRating&order=desc liefert 3, 1, dann 2 und 4.
- ?sort=averageRating liefert 1, 3, dann 2 und 4.

```
POST /beers/3/ratings  { "userId": 1, "score": 5 }
POST /beers/3/ratings  { "userId": 2, "score": 4 }
POST /beers/1/ratings  { "userId": 1, "score": 2 }

[{"id":3, …, "averageRating":4.5},
 {"id":1, …, "averageRating":2},
 {"id":2, …, "averageRating":null},
 {"id":4, …, "averageRating":null}]
```

Aufgabe 11.3 · Die API dokumentieren

ZIEL

Eine Brauerei bindet die Beer API ohne Rückfrage an.

AUSGANGSLAGE

Aufgabe 11.2 ist gelöst. Die Bier-Endpunkte zeigen in Swagger UI nur Statuscodes ohne Schema.

ZIELE

- Ein Schema für die Antwort eines Biers enthält id und averageRating; eines für eine Bewertung enthält id, beerId und createdAt.
- Jeder Bier- und Bewertungs-Endpunkt beschreibt seine Antwort und seine Fehler.
- Die Endpunkte stehen in Swagger UI unter einer Gruppe beers.

FERTIG, WENN

- Swagger UI zeigt alle Endpunkte mit Schemas für Antwort und Fehler.
- Die importierte Beschreibung lässt sich in Postman ausführen.

Zusatz 11.4 · Zwei öffentliche APIs einordnen

ZIEL

Du vergleichst zwei öffentliche APIs mit der Beer API und ordnest sie nach dem Richardson Maturity Model ein. Der Zusatz braucht keinen Code.

AUSGANGSLAGE

Postman mit Zugang zum Internet, zu zweit. GitHub: https://api.github.com/repos/nestjs/nest/issues?per_page=1. Open Brewery DB: https://api.openbrewerydb.org/v1/breweries?per_page=1. Vorlage in Skript 11.22.

ZIELE

- Je API sind Filter, Sortierung und die Antworten auf einen ungültigen Wert und auf DELETE ohne Anmeldung gemessen.
- Die Stufe ist mit einer beobachteten Antwort begründet.

FERTIG, WENN

- Die Tabelle ist für beide APIs ausgefüllt.

Vorlage zu Zusatz 11.4

Frage	GitHub	Open Brewery DB
Wie heißen Filter und Sortierung?		
Antwort auf einen ungültigen Wert, etwa ?sort=price		
Antwort auf DELETE ohne Anmeldung an ein Element der Liste		
Enthält die Antwort URLs zu weiteren Ressourcen?		
Stufe im Maturity Model, mit Begründung		

Selbstkontrolle mit Postman

Ab dieser Einheit schreibst du die Tests selbst. Lege in deiner importierten Collection je Punkt aus „Fertig, wenn“ der Aufgaben 11.1 und 11.2 eine Anfrage an. Schreibe im Tab **Scripts** unter **Post-response** einen Test wie in Skript 11.24: zuerst den Statuscode, danach Form und Inhalt. Die Meldung an jedem pm.expect nennt, wo du suchen musst.

Der Ordner der Einheit 11 in der Collection Web-Programmierung zeigt zum Vergleich, welche Fälle eine vollständige Prüfung enthält. Führe darin den Ordner deiner Aufgabe mit **Run** aus, zum Beispiel „Aufgabe 11.2 – Biere sortieren“. Die Anfragen legen Bewertungen an; starte den Server vor jedem Lauf neu.

Dokumentation: https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

Ein Test für GET /breweries?city=München

JAVASCRIPT

```
pm.test('Status 200', () => {
  pm.expect(pm.response.code, 'Kennt das Schema city?').to.eql(200);
});
const weiter = pm.response.code === 200 ? pm.test : pm.test.skip;
weiter('Genau eine Brauerei', () => {
  const list = pm.response.json();
  pm.expect(list.length, 'Filtert findAll nach city?').to.eql(1);
});
weiter('Augustiner-Bräu', () => {
  const name = pm.response.json()[0].name;
  pm.expect(name, 'Filtert findAll nach city?').to.eql('Augustiner-Bräu');
});
```

Hinweise zu Aufgabe 11.1

- Stufe 1: Das Schema aus Skript 11.9 zeigt, wie Query-Parameter in den Service kommen.
- Stufe 2: Wo stehen die erlaubten Stile schon im Code? Welchen Typ hat breweryId im Query-String?
- Stufe 3: Sende ?breweryId=2 und lies die Meldung der Antwort; danach ?style=helles&breweryId=3.

Hinweise zu Aufgabe 11.2

- Stufe 1: Schritt 2 des Beispiels sortiert Strings. Was ändert sich bei Zahlen und bei null?
- Stufe 2: Die Vergleichsfunktion entscheidet für zwei Biere, welches zuerst kommt. Was gibt sie zurück, wenn eines null hat?
- Stufe 3: Sende die Testdaten und danach ?sort=averageRating ohne order: Wo stehen die Biere 2 und 4?

Hinweise zu Aufgabe 11.3

- Stufe 1: Folie 11.13 der Vorlesung zeigt, welcher Decorator was in der Beschreibung erzeugt.
- Stufe 2: Öffne http://localhost:3000/api-json und suche unter paths die Endpunkte ohne schema bei 200 oder 201.
- Stufe 3: Meldet der Compiler TS1241, lies die letzte Zeile: Welches Feld hat einen anderen Typ als im Schema?

Hinweise zu Zusatz 11.4

- Stufe 1: Die Dokumentation nennt Filter und Sortierung: https://docs.github.com/en/rest/issues/issues#list-repository-issues und https://www.openbrewerydb.org/documentation
- Stufe 2: Lies in der Dokumentation die Query-Parameter der Liste, bei jeder Antwort auch Statuscode und Header.
- Stufe 3: Suche in einer Antwort von GitHub nach Feldern, die auf _url enden: Reicht das für Stufe 3?

Typische Fehler

Symptom	Ursache
?breweryId=2 ergibt 400: „breweryId: Invalid input: expected number, received string“	Query-Parameter sind Strings; das Schema wandelt breweryId nicht in eine Zahl um.
?style=lager ergibt 200 mit []	style ist im Query-Schema ein beliebiger String statt einer der Bierstile.
?sort=averageRating: Biere ohne Bewertung stehen vorne	Die Vergleichsfunktion behandelt null wie 0.
Start bricht ab: „TS1241“ mit „Type 'string' is not assignable …“	Ein Typ im Service weicht vom Schema ab, etwa style im Interface Beer als string.
/api-json: GET /beers/{id} hat nur 200 ohne Schema	An der Methode fehlen @ZodResponse und die Decorators für Fehler.

Lösung zu Zusatz 11.4, gemessen am 2026-10-25

Frage	GitHub	Open Brewery DB
Filter und Sortierung	state, labels; sort und direction	by_city, sort=name:desc
?sort=price	200, der Wert wird ignoriert	kein 400 und keine Fehlermeldung, gemessen: 302 auf die Startseite
DELETE ohne Anmeldung	404 für Issue 1, GitHub verrät nicht, ob es existiert	405 mit Allow: GET, HEAD
URLs in der Antwort	viele Felder auf _url, Header Link für weitere Seiten	keine
Stufe	2 mit vielen Links; Methode und Body stehen nur in der Dokumentation	2, nur lesend

Das solltest du jetzt können

- ✓ Die REST-Constraints nennen und an der Beer API zeigen
- ✓ URLs für Ressourcen, Filter und Sortierung entwerfen
- ✓ Eine API nach dem Richardson Maturity Model einordnen
- ✓ Endpunkte so beschreiben, dass Swagger UI Anfrage, Antwort und Fehler zeigt
- ✓ Eine OpenAPI-Beschreibung in Postman importieren und die Anfragen ausführen

Als Nächstes: Datenbank-Anbindung – Biere, Brauereien und Bewertungen überstehen einen Neustart des Servers.

WEB-PROGRAMMIERUNG · EINHEIT 12

Datenbank-Anbindung · Skript

Worum es in dieser Einheit geht

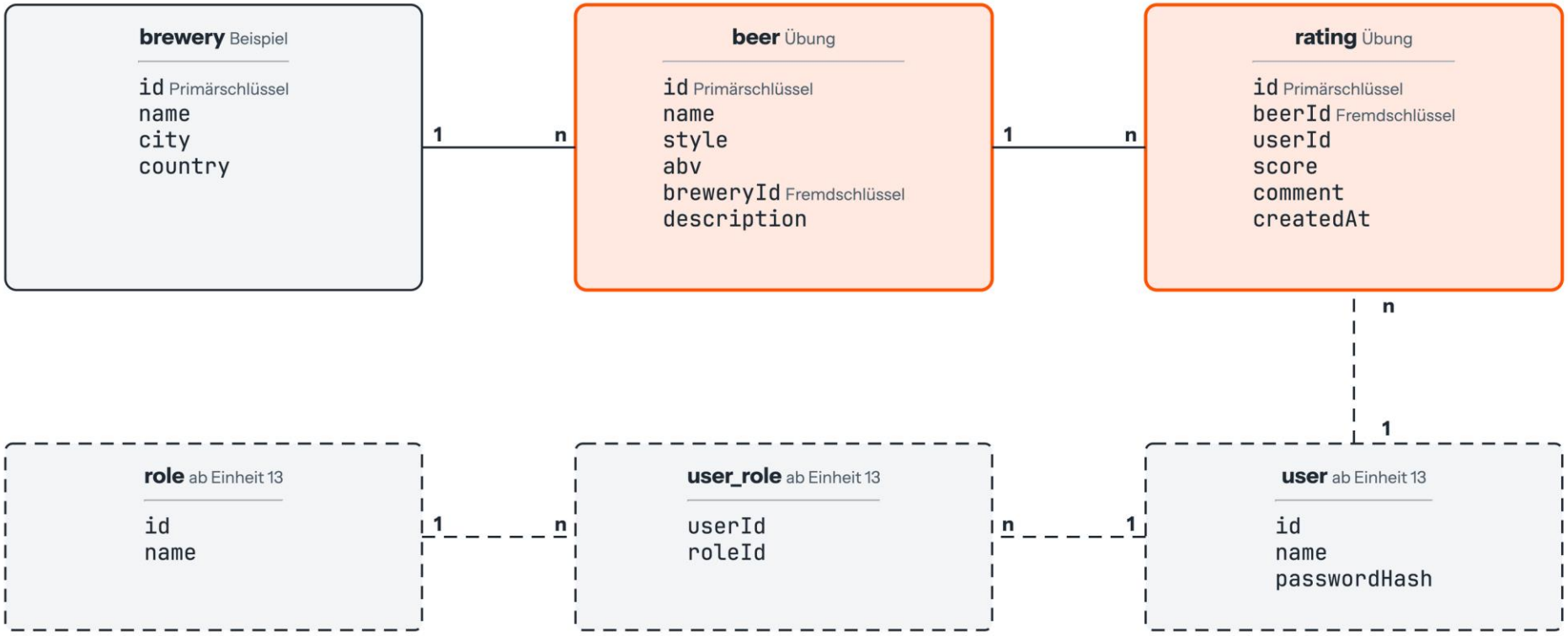
Bis Einheit 11 löscht jeder Neustart des Servers alle neuen Biere, Brauereien und Bewertungen. Für die regionalen Brauereien wären vor allem die Bewertungen verloren, denn nur die App kennt sie. Die Aufgaben setzen diese Story um:

Als Betreiber möchte ich, dass Biere, Brauereien und Bewertungen einen Neustart des Servers überstehen, damit keine Daten verloren gehen.

Am Ende der Einheit kannst du:

- Entities und Migrationen für eine Ressource schreiben,
- einen Service von einem Array auf ein Repository umstellen,
- mit einem Fremdschlüssel eine Regel in der Datenbank absichern.

Das Entity-Relationship-Modell (ER-Modell) der Beer App hat sechs Tabellen



Ein Bier gehört zu genau einer Brauerei, eine Bewertung zu genau einem Bier. Durchgezogen: ab heute in beer.db · gestrichelt: ab Einheit 13

ER-Modell mit sechs Tabellen in zwei Reihen. Oben, durchgezogen: brewery (Beispiel) mit id als Primärschlüssel, name, city, country; beer (Übung, hervorgehoben) mit id als Primärschlüssel, name, style, abv, breweryId als Fremdschlüssel, description; rating (Übung, hervorgehoben) mit id als Primärschlüssel, beerId als Fremdschlüssel, userId, score, comment, createdAt. brewery zu beer 1 zu n, beer zu rating 1 zu n. Unten, gestrichelt, ab Einheit 13: role mit id und name, user_role mit userId und roleId, user mit id, name und passwordHash; role zu user_role 1 zu n, user zu user_role 1 zu n, user zu

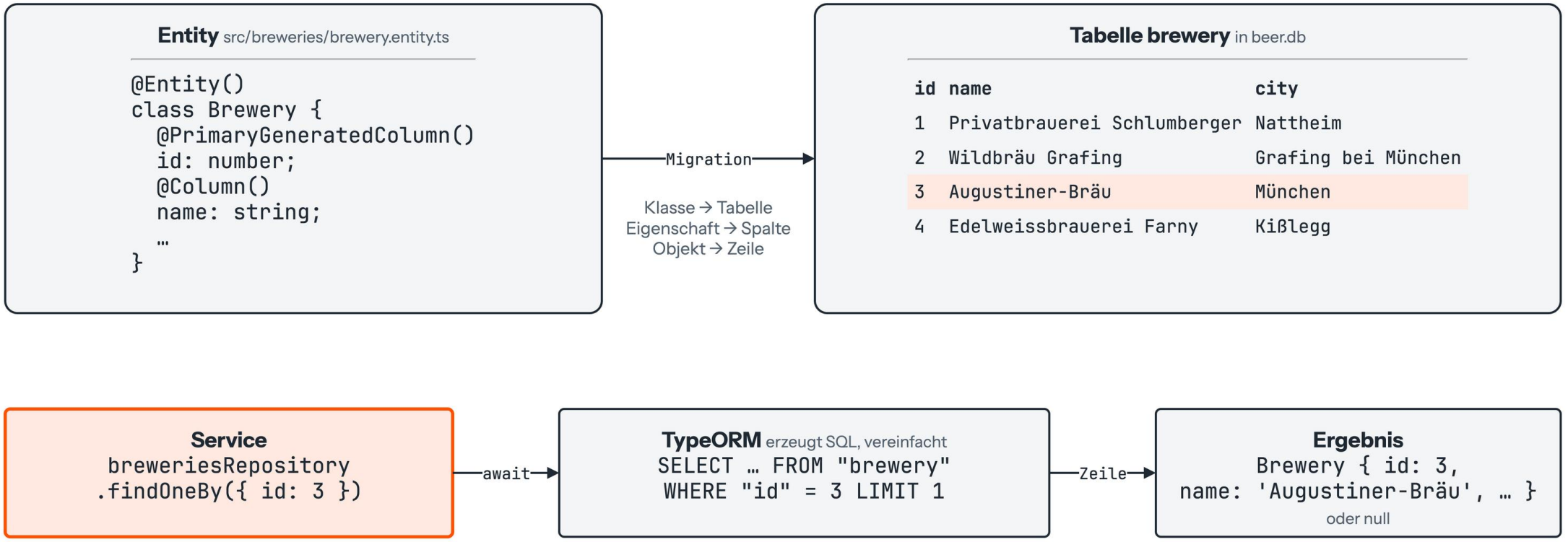
Persistenz heißt: Die Daten überstehen das Ende des Prozesses

Bis Einheit 11 liegen die Daten in Arrays im Arbeitsspeicher. Endet der Prozess, gibt das Betriebssystem den Speicher frei. Welcher Verlust am teuersten ist, zeigt Folie 12.5 der Vorlesung.

- SQLite ist eine Bibliothek im Prozess des Servers. Die ganze Datenbank ist die Datei beer.db im Projektordner.
- beer.db steht in .gitignore. Jede Kopie des Projekts erzeugt ihre Datenbank mit npm run migration:run.
- Ein Backup ist bei SQLite eine Kopie dieser Datei, solange der Server nicht schreibt.
- Primärschlüssel, Fremdschlüssel und Normalisierung erklärt die Datenbank-Vorlesung.

Dokumentation: https://www.sqlite.org/about.html

Ein ORM übersetzt die Klasse Brewery in die Tabelle brewery



Der Code bleibt TypeScript: Das ORM übersetzt Klassen in Tabellen und Methoden des Repository in SQL.

Oben links die Entity in src/breweries/brewery.entity.ts: @Entity() über class Brewery, darin @PrimaryGeneratedColumn() über id: number und @Column() über name: string. Ein Pfeil „Migration legt an“ führt zur Tabelle brewery in beer.db mit den Spalten id, name, city und den vier Brauereien; die Zeile 3, Augustiner-Bräu, München, ist hervorgehoben. Zwischen beiden: Klasse → Tabelle, Eigenschaft → Spalte, Objekt → Zeile. Unten drei Kästen: Service ruft breweriesRepository.findOneBy({ id: 3 }) auf, Pfeil „await“ zu TypeORM, das SQL erzeugt, vereinfacht SELECT … FROM "brewery" WHERE "id" =

Die Entity beschreibt die Tabelle, das Repository liest und schreibt sie

Ein ORM (Object-Relational Mapping) übersetzt zwischen Objekten und Tabellen. Die Methoden des Repository stehen auf Folie 12.8 der Vorlesung.

- TypeORM liest den Typ der Spalte aus dem TypeScript-Typ: string wird varchar, number wird integer.
- Abgeleitete Typen wie CreateBeerDto['style'] und Typen mit null brauchen den Typ in @Column ausdrücklich.
- Für Kommazahlen heißt die Spalte real; andere Datenbanken runden in einer Spalte integer.
- Die Methoden des Repository liefern Promises; die aufrufende Methode wird async.
- SQLite sortiert Strings nach Bytes; bei den Beispieldaten ergibt das dieselbe Reihenfolge wie in Einheit 11.

Dokumentation: https://typeorm.io/docs/entity/entities

Eine Migration bringt jede Kopie der Datenbank auf denselben Stand



migration:run legt beer.db an, wenn die Datei fehlt, und führt nur offene Migrationen aus.

Ablauf in fünf Kästen. Entities (Brewery, Beer, Rating: wie die Tabellen sein sollen) führen zu migration:generate, hervorgehoben: vergleicht die Entities mit den Tabellen in beer.db. Pfeil „erzeugt“ zur Migration im Git-Repository mit up: CREATE TABLE und down: DROP TABLE. Pfeil zu migration:run, hervorgehoben: führt jede Migration aus, die noch nicht gelaufen ist. Pfeil „ändert“ zu beer.db, nicht im Repository: Tabellen brewery, beer, rating und Tabelle migrations, die festhält, was gelaufen ist. Links: Jede Kopie des Projekts ergibt mit npm install und npm run migration:run dieselben Tabellen.

Migrationen ändern das Datenbankschema, Fremdschlüssel sichern Beziehungen

Die Befehle stehen auf Folie 12.10 der Vorlesung. Eine Migration ist eine Datei mit up und down.

- Lies jede erzeugte Migration, bevor du sie ausführst. Sie zeigt das SQL, das die Datenbank ändert.
- Beispieldaten stehen in einer eigenen Migration; npx typeorm migration:create legt sie leer an.
- @ManyToOne mit @JoinColumn auf eine vorhandene Spalte erzeugt in der nächsten Migration einen Fremdschlüssel.
- TypeORM schaltet die Prüfung der Fremdschlüssel in SQLite beim Verbinden ein; danach lehnt SQLite Verstöße ab.

Dokumentation: https://typeorm.io/docs/migrations/why

Beispiel: Brauereien in SQLite speichern

Übernimm das Live-Coding in dein Projekt aus Einheit 11, bevor du mit den Aufgaben beginnst.

1. Installiere die Pakete (Skript 12.10).
2. Lege data-source.ts an und öffne die Verbindung im AppModule (Skript 12.11 und 12.12).
3. Lege die Entity Brewery an und trage die Befehle in package.json ein (Skript 12.13 und 12.14).
4. Erzeuge die Migrationen und führe sie aus (Skript 12.15 bis 12.17).
5. Stelle den BreweriesService auf das Repository um (Skript 12.18 bis 12.20).
6. Passe Module, Controller und BeersService an (Skript 12.21).

Beispiel, Schritt 1: Pakete installieren

BASH

```
npm install @nestjs/typeorm@12.0.2 typeorm@1.1.2
npm install better-sqlite3@12.11.1
```

Beispiel, Schritt 2: Eine Beschreibung der Verbindung für Server und CLI

TYPESCRIPT · SRC/DATA-SOURCE.TS

```typescript
import { DataSource, DataSourceOptions } from 'typeorm';
import { Brewery } from './breweries/brewery.entity';

// Eine Beschreibung der Verbindung für den Server (AppModule) und die CLI
export const dataSourceOptions: DataSourceOptions = {
  type: 'better-sqlite3',
  database: 'beer.db',
  entities: [Brewery],
  // .ts für die CLI im Ordner src, .js für den Server im Ordner dist
  migrations: [__dirname + '/migrations/*.{ts,js}'],
};

// Die CLI liest die Verbindung aus dem Default-Export
export default new DataSource(dataSourceOptions);
```

Beispiel, Schritt 2: Das AppModule öffnet die Verbindung als Erstes

TYPESCRIPT · SRC/APP.MODULE.TS

```
@Module({
  imports: [
    // Verbindung zu beer.db; dieselbe Beschreibung nutzt die CLI
    TypeOrmModule.forRoot(dataSourceOptions),
    BreweriesModule,
    BeersModule,
    RatingsModule,
  ],
```

Beispiel, Schritt 3: Die Entity Brewery

TYPESCRIPT · SRC/BREWERIES/BREWERY.ENTITY.TS

```
// Die Klasse beschreibt die Tabelle brewery, ein Objekt eine Zeile
@Entity()
export class Brewery {
  // Die Datenbank vergibt die id beim Einfügen
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  name: string;

  @Column()
  city: string;

  @Column()
  country: string;
}
```

Beispiel, Schritt 3: Befehle für die CLI unter scripts

JSON · PACKAGE.JSON

```
"typeorm": "typeorm-ts-node-commonjs -d src/data-source.ts",
"migration:generate": "npm run typeorm -- migration:generate",
"migration:run": "npm run typeorm -- migration:run",
"db:reset": "npm run typeorm -- schema:drop && npm run migration:run"
```

Beispiel, Schritt 4: Die erste Migration erzeugen und ausführen

```
$ npm run migration:generate -- src/migrations/CreateBreweries
Migration …/src/migrations/1792672856555-CreateBreweries.ts
  has been generated successfully.

$ npm run migration:run
Migration CreateBreweries1792672856555 has been executed
  successfully.
```

Beispiel, Schritt 4: Die Beispieldaten als zweite Migration

TYPESCRIPT · SRC/MIGRATIONS/1792672866842-SEEDBREWERIES.TS

```
public async up(queryRunner: QueryRunner): Promise<void> {
  await queryRunner.manager.insert(Brewery, [
    {
      id: 1,
      name: 'Privatbrauerei Schlumberger',
      city: 'Nattheim',
      country: 'Deutschland',
    },
```

Beispiel, Schritt 4: Die Migration mit den Beispieldaten vervollständigen

- Lege die leere Migration mit npx typeorm migration:create src/migrations/SeedBreweries an.
- Importiere Brewery aus ../breweries/brewery.entity.
- up legt die vier Brauereien mit fester id an: 1 Privatbrauerei Schlumberger, Nattheim; 2 Wildbräu Grafing, Grafing bei München.
- Dazu 3 Augustiner-Bräu, München; 4 Edelweissbrauerei Farny, Kißlegg. country ist bei allen Deutschland.
- down löscht sie wieder mit await queryRunner.manager.delete(Brewery, [1, 2, 3, 4]).
- Führe danach npm run migration:run aus. Der Viewer zeigt vier Zeilen.

Beispiel, Schritt 5: Der Service bekommt das Repository

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
// Daten in beer.db: Ein Neustart ändert nichts
constructor(
  @InjectRepository(Brewery)
  private readonly breweriesRepository: Repository<Brewery>,
) {}

findAll(query: BreweryQueryDto): Promise<Brewery[]> {
  // undefined im where wirft in TypeORM 1: nur gesetzte Filter übergeben
  return this.breweriesRepository.find({
    where: query.city ? { city: query.city } : {},
    order: query.sort ? { [query.sort]: query.order } : {},
  });
}
```

Beispiel, Schritt 5: Lesen und Anlegen mit dem Repository

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```typescript
async findOne(id: number): Promise<Brewery> {
  const brewery = await this.breweriesRepository.findOneBy({ id });
  if (!brewery) {
    // Jeder Aufrufer bekommt dieselbe 404
    throw new NotFoundException(`Brewery ${id} not found`);
  }
  return brewery;
}

create(dto: CreateBreweryDto): Promise<Brewery> {
  // Ohne id fügt save eine Zeile ein; die Datenbank vergibt die id
  return this.breweriesRepository.save(dto);
}
```

Beispiel, Schritt 5: Ändern und Löschen mit dem Repository

TYPESCRIPT · SRC/BREWERIES/BREWERIES.SERVICE.TS

```
async update(id: number, dto: UpdateBreweryDto): Promise<Brewery> {
  const brewery = await this.findOne(id);
  // Nur die Felder aus dem Body überschreiben
  Object.assign(brewery, dto);
  // Mit id ändert save die vorhandene Zeile
  return this.breweriesRepository.save(brewery);
}

async remove(id: number): Promise<void> {
  const brewery = await this.findOne(id);
  await this.breweriesRepository.remove(brewery);
}
```

Beispiel, Schritt 6: Module, Controller und BeersService anpassen

- BreweriesService: Interface Brewery, Array und nextId entfallen. Neue Imports: InjectRepository aus @nestjs/typeorm, Repository aus typeorm.
- BreweriesModule: imports enthält TypeOrmModule.forFeature([Brewery]); damit kann NestJS das Repository injizieren.
- BreweriesController: remove gibt das Promise mit return zurück, sonst erreicht die 404 den Client nicht.
- PATCH /breweries/:id ruft update auf und trägt dieselben Decorators wie GET /breweries/:id; UpdateBreweryDto entsteht aus CreateBrewerySchema.partial().
- BeersService: assertBreweryExists wird async und wartet mit await auf findOne; create und update warten darauf.

Danach antwortet GET /breweries wie in Einheit 11, auch nach einem Neustart.

Aufgabe 12.1 · Biere in der Datenbank speichern

ZIEL

Biere überstehen einen Neustart des Servers.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel. Daten und Anfragen in Skript 12.23 bis 12.25. Dokumentation: https://typeorm.io/docs/relations/many-to-one-one-to-many-relations

ZIELE

- Migrationen legen die Tabelle beer mit Fremdschlüssel und die vier Beispielbiere an.
- Der BeersService liest und schreibt nur über das Repository.
- Das Antwortschema erlaubt null für description.
- Filter und Sortierung antworten wie in Einheit 11.

FERTIG, WENN

- Nach einem Neustart liefert GET /beers/5 das Pils, und Bier 6 bleibt gelöscht.
- GET /beers?style=pils&breweryId=2 liefert danach nur das Pils.

Datenmodell eines Biers in der Tabelle beer; die id vergibt die Datenbank

Feld	Typ in der Antwort	Pflicht	Bedeutung
name	String	ja	Name des Biers, nicht leer
style	String	ja	Bierstil: helles, pils, spezial, weizen oder dunkel
abv	Kommazahl	ja	Alkoholgehalt in Volumenprozent, zum Beispiel 5.2
breweryId	Ganzzahl	ja	id einer existierenden Brauerei, Fremdschlüssel auf brewery
description	String oder null	nein	Kurzbeschreibung; ohne Angabe null

Beispieldaten der Tabelle beer

id	name	style	abv	breweryId
1	Nattheimer Spezial	spezial	5.4	1 (Privatbrauerei Schlumberger)
2	Wildbräu Grafing Helles	helles	5.0	2 (Wildbräu Grafing)
3	Augustiner Lagerbier Hell	helles	5.2	3 (Augustiner-Bräu)
4	Farny Kristall-Weizen	weizen	5.3	4 (Edelweissbrauerei Farny)

Anfragen zu Aufgabe 12.1

BEISPIEL

- Setze die Datenbank mit npm run db:reset zurück.
- Sende die drei Anfragen rechts.
- Starte den Server neu.
- GET /beers/5 liefert das Pils.
- GET /beers liefert die Biere 1 bis 5.

```
POST /beers
{ "name": "Wildbräu Grafing Pils",
  "style": "pils", "abv": 4.9, "breweryId": 2 }
→ 201 mit id 5

POST /beers
{ "name": "Augustiner Dunkel",
  "style": "dunkel", "abv": 5.6, "breweryId": 3 }
→ 201 mit id 6

DELETE /beers/6  → 204
```

Regeln zu Aufgabe 12.1 und 12.2

Diese Regeln aus den Einheiten 7 bis 10 gelten nach der Umstellung weiter:

- abv liegt zwischen 0 und 20.
- Ein Bier gehört zu genau einer Brauerei; die Brauerei muss existieren.
- score ist eine ganze Zahl von 1 bis 5.
- Ein User bewertet ein Bier höchstens einmal; eine zweite Bewertung ersetzt die erste.
- Die Durchschnittsbewertung eines Biers wird auf eine Nachkommastelle gerundet.

averageRating steht in keiner Spalte; der Service berechnet es für jede Antwort.

Alles, was die Beer API bis Einheit 11 konnte, funktioniert nach der Umstellung unverändert: Filter, Sortierung, Durchschnitt und die Prüfung der Regeln. Eine Änderung ist erlaubt: Eine leere description oder ein leerer comment steht als null in der Antwort.

Aufgabe 12.2 · Bewertungen in der Datenbank speichern

ZIEL

Bewertungen und der Durchschnitt überstehen einen Neustart des Servers.

AUSGANGSLAGE

Aufgabe 12.1 ist gelöst. Daten und Anfragen in Skript 12.28 und 12.29, Regeln in Skript 12.26. Dokumentation: https://typeorm.io/docs/relations/many-to-one-one-to-many-relations

ZIELE

- Eine Migration legt die Tabelle rating mit Fremdschlüssel auf beer an.
- Der RatingsService liest und schreibt nur über das Repository; comment darf null sein.
- Ein bewertetes Bier lässt sich löschen; seine Bewertungen fallen mit weg.

FERTIG, WENN

- Nach einem Neustart liefert GET /beers/1/ratings die Bewertung, und averageRating ist 4.
- DELETE /beers/1 antwortet danach mit 204.

Datenmodell einer Bewertung in der Tabelle rating; die id vergibt die Datenbank

Feld	Typ in der Antwort	Pflicht	Bedeutung
beerId	Ganzzahl	ja	id des Biers aus dem Pfad, Fremdschlüssel auf beer
userId	Ganzzahl	ja	id des Gasts, noch ohne Tabelle user
score	Ganzzahl	ja	Sterne von 1 bis 5
comment	String oder null	nein	Freitext; ohne Angabe null
createdAt	String	ja	Zeitpunkt als ISO 8601, vom Server gesetzt

Anfragen zu Aufgabe 12.2

BEISPIEL

- Setze die Datenbank mit npm run db:reset zurück.
- Sende die Bewertung und lege Brauhaus Riegele an.
- Starte den Server neu.
- Die Bewertung, der Durchschnitt 4 und die Brauerei sind erhalten.
- Lösche danach Bier 1.

```
POST /beers/1/ratings
{ "userId": 1, "score": 4, "comment": "Süffig" }
→ 201

POST /breweries
{ "name": "Brauhaus Riegele",
  "city": "Augsburg", "country": "Deutschland" }
→ 201

Neustart, dann:
GET /beers/1/ratings  → 200, eine Bewertung
GET /beers/1          → 200, averageRating 4
DELETE /beers/1       → 204
```

Zusatz 12.3 · Eine Brauerei mit Bieren behalten

ZIEL

Eine Brauerei, zu der Biere gehören, lässt sich nicht löschen. Die Antwort nennt den Grund.

AUSGANGSLAGE

Aufgabe 12.1 ist gelöst. DELETE /breweries/2 antwortet mit 500, weil der Fremdschlüssel das Löschen ablehnt. Dokumentation: https://docs.nestjs.com/exception-filters#built-in-http-exceptions

ZIELE

- Der BreweriesService übersetzt nur die Ablehnung durch den Fremdschlüssel in 409 Conflict.
- Jeder andere Fehler der Datenbank bleibt 500.

FERTIG, WENN

- DELETE /breweries/2 antwortet mit 409 im Format der anderen Fehler; Brauerei und Bier bleiben.
- Eine neu angelegte Brauerei ohne Biere lässt sich mit 204 löschen.

Selbstkontrolle mit Postman

Schreibe die Tests wie in Einheit 11 selbst (Skript 11.24). Einen Neustart des Servers kann Postman nicht auslösen. Lege deshalb je Aufgabe zwei Ordner an: vor dem Neustart Daten anlegen und löschen, nach dem Neustart nur lesen. Setze die Datenbank vor jedem Lauf zurück; dann bekommen neue Biere die id 5 und 6.

Der Ordner der Einheit 12 in der Collection Web-Programmierung zeigt zum Vergleich, welche Fälle eine vollständige Prüfung enthält, je Aufgabe in zwei Ordnern.

Ob alles, was bis Einheit 11 ging, unverändert funktioniert, zeigen die Ordner der Einheiten 5 bis 11 in der Collection Web-Programmierung. Setze die Datenbank vor jedem dieser Ordner mit npm run db:reset zurück und führe seine Aufgaben aus; kein Test darf rot sein.

Ein Lauf mit Neustart

1. Setze die Datenbank mit npm run db:reset zurück; der Server darf dabei laufen.
2. Führe die Ordner vor dem Neustart aus, zum Beispiel mit **Run folder** in Postman.
3. Beende npm run start:dev mit Strg+C (macOS: Ctrl+C) und starte es neu.
4. Führe die Ordner nach dem Neustart aus.
5. Zusatz 12.3 läuft nach npm run db:reset ohne Neustart.

Hinweise zu Aufgabe 12.1

- Stufe 1: Das Beispiel zeigt jeden Schritt an den Brauereien. Die Reihenfolge Entity, Migration, Service gilt auch für die Biere.
- Stufe 2: Bricht migration:generate ab, nennt die Meldung Entity und Eigenschaft. Sonst lies die Migration: Gibt es einen FOREIGN KEY?
- Stufe 3: Sende GET /beers ohne Filter und lies das Terminal von start:dev, bevor du die Filter prüfst.

Hinweise zu Aufgabe 12.2

- Stufe 1: Antwortet DELETE /beers/1 mit 500, lies das Terminal von start:dev: Wer lehnt das Löschen ab?
- Stufe 2: Welche Zeilen zeigen nach dem Löschen auf Bier 1, und was soll mit ihnen geschehen?
- Stufe 3: Suche unter den Optionen von @ManyToOne die Option für das Löschen und erzeuge danach eine neue Migration.

Hinweise zu Zusatz 12.3

- Stufe 1: Lies im Terminal von start:dev den Namen des Fehlers und seine Meldung beim Löschen.
- Stufe 2: Woran erkennst du an Klasse und Meldung des Fehlers, dass der Fremdschlüssel abgelehnt hat?
- Stufe 3: Fange in remove nur diesen einen Fehler ab und wirf eine Exception mit eigener Meldung; alle anderen weiter.

Typische Fehler zu Aufgabe 12.1

Symptom	Ursache
migration:generate bricht ab: „Data type "Object" in "Beer.style" is not supported by "better-sqlite3" database.“	style hat den Typ CreateBeerDto['style']; TypeScript übergibt dafür Object.
Start bricht ab mit „error TS1241: Unable to resolve signature of method decorator“ an @ZodResponse	Die Entity erlaubt null für description; das Antwortschema erlaubt es noch nicht.
500, im Terminal „SqliteError: no such table: beer“	Die Migration für beer ist nicht erzeugt oder nicht ausgeführt.
GET /beers ergibt 500, im Terminal „Undefined value encountered in property 'Beer.style' of a where condition.“	findAll übergibt einen fehlenden Filter als undefined an find.

Typische Fehler zu Aufgabe 12.2

Symptom	Ursache
Start bricht ab: „Entity metadata for Rating#beer was not found.“	Beer fehlt unter entities in data-source.ts.
DELETE /beers/1 ergibt 500, im Terminal „FOREIGN KEY constraint failed“	Bewertungen zeigen auf das Bier; die Relation an Rating legt nicht fest, was beim Löschen geschieht.
Zusatz 12.3: DELETE /breweries/2 ergibt 500, im Terminal „FOREIGN KEY constraint failed“	remove fängt die Ablehnung durch den Fremdschlüssel nicht ab.

Das solltest du jetzt können

- ✓ Eine Entity für eine Ressource schreiben
- ✓ Eine Migration erzeugen, lesen und ausführen
- ✓ Einen Service von einem Array auf ein Repository umstellen
- ✓ Mit @ManyToOne einen Fremdschlüssel anlegen und erklären, was er verhindert
- ✓ Die Datenbank mit einem Befehl auf die Beispieldaten zurücksetzen

Als Nächstes: Authentifizierung – Gäste registrieren sich, melden sich an und bewerten mit einem Token.

WEB-PROGRAMMIERUNG · EINHEIT 13

Authentifizierung · Skript

Worum es in dieser Einheit geht

Bis Einheit 12 übernimmt die Beer API jede userId aus dem Body. Ein Gast kann deshalb unter beliebig vielen userIds bewerten und den Durchschnitt eines Biers verschieben. Die regionalen Brauereien brauchen Bewertungen, die echten Gästen gehören. Die Aufgaben setzen diese Story um:

Als Gast möchte ich mich registrieren und anmelden, damit nur ich in meinem Namen bewerte.

Am Ende der Einheit kannst du:

- Passwörter als Hash speichern und beim Login prüfen,
- ein JWT ausstellen und lesen,
- einen Endpunkt mit einem Guard schützen und das Token in Postman senden.

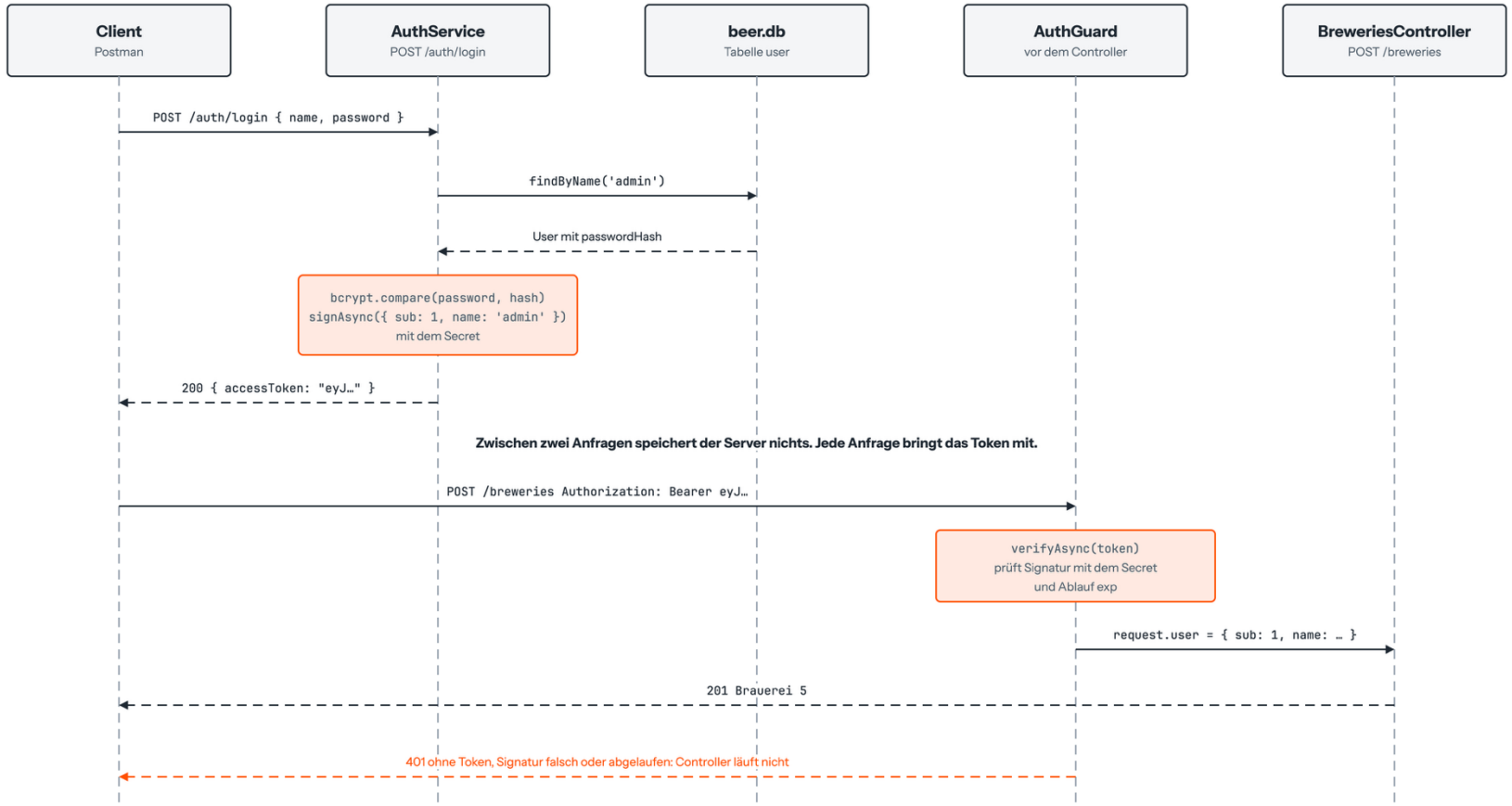
Authentifizierung stellt fest, wer eine Anfrage stellt

Die Unterscheidung zur Autorisierung und die Statuscodes 401 und 403 stehen auf Folie 13.5 der Vorlesung, die drei Wege der Anmeldung auf Folie 13.6.

- Ein Identity Provider übernimmt Anmeldung und Benutzerkonten für viele Anwendungen, zum Beispiel Microsoft Entra ID, Auth0 oder Keycloak.
- Was er zusätzlich leistet, zeigt Folie 13.4: Passwort zurücksetzen, zweiter Faktor, Sperre nach Fehlversuchen.
- Die Beer API baut den Login selbst: Passwort als Hash, Token als JWT, Prüfung in einem Guard.
- 401 heißt: Das Token fehlt oder ist ungültig. Der Name „Unauthorized“ ist historisch; gemeint ist fehlende Authentifizierung.

Dokumentation: https://datatracker.ietf.org/doc/html/rfc9110#section-15.5.2

Nach dem Login trägt jede Anfrage ein signiertes Token, das der Guard prüft



Sequenz mit fünf Beteiligten: Client (Postman), AuthService (POST /auth/login), beer.db (Tabelle user), AuthGuard (vor dem Controller), BreweriesController (POST /breweries). Der Client sendet POST /auth/login mit name und password. Der AuthService ruft findByName('admin') und bekommt den User mit passwordHash. Hervorgehoben: bcrypt.compare(password, hash) und signAsync({ sub: 1, name: 'admin' }) mit dem Secret. Antwort 200 mit accessToken. Darunter: Zwischen zwei Anfragen speichert der Server nichts. Jede Anfrage bringt das Token mit. Der Client sendet POST

Passwörter stehen nur als Hash in der Datenbank

Aufbau eines Hashes und die gemessene Rechenzeit stehen auf den Folien 13.8 und 13.9 der Vorlesung.

- Ein Hash ist das Ergebnis einer Einwegfunktion: Aus ihm lässt sich das Passwort nicht zurückrechnen.
- Das Salt ist eine Zufallszahl je Passwort. Zwei gleiche Passwörter ergeben deshalb verschiedene Hashes.
- bcrypt.hash erzeugt Salt und Hash, bcrypt.compare prüft ein Passwort gegen einen gespeicherten Hash.
- Die Beer API nutzt das Paket bcryptjs. Es ist reines JavaScript und installiert ohne Compiler.
- passwordHash erscheint in keiner Antwort der API.

Dokumentation: https://github.com/dcodeIO/bcrypt.js

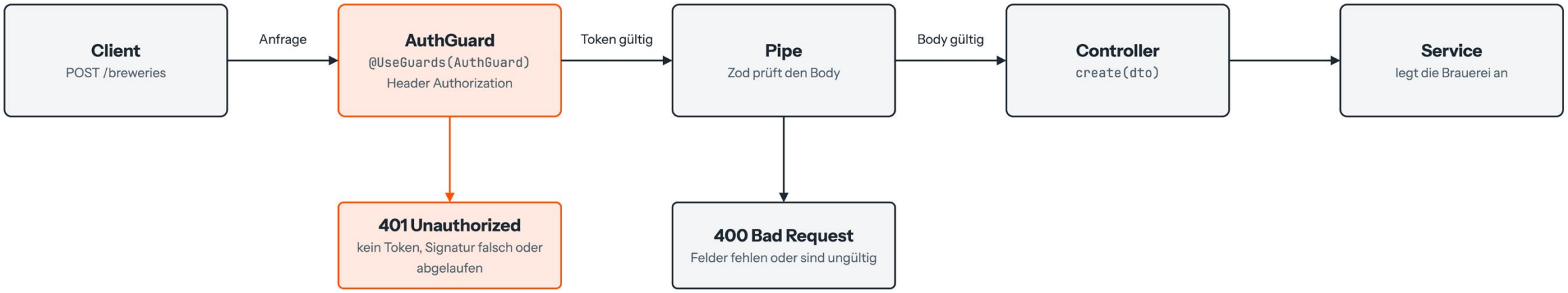
Ein JWT nennt den User; die Signatur schützt vor Änderung

Aufbau und Teile eines JWT (JSON Web Token) stehen auf den Folien 13.10 und 13.11 der Vorlesung.

- Header und Payload sind Base64URL-kodiert. Jeder kann sie lesen, zum Beispiel mit https://jwt.io. Ein Passwort gehört nie in den Payload.
- Die Signatur berechnet der Server mit einem Secret. Ein geänderter Payload passt nicht mehr zur Signatur.
- Der Client sendet das Token im Header Authorization: Bearer <token>. Im Betrieb geht das nur über HTTPS.
- Die Beer API speichert keine Tokens. Ein Token gilt deshalb bis exp; der Server kann es vorher nicht sperren.

Dokumentation: https://datatracker.ietf.org/doc/html/rfc7519

Ein Guard entscheidet vor der Pipe, ob die Anfrage den Controller erreicht



Der Guard läuft vor der Pipe: Ohne Token und mit leerem Body antwortet der Server mit 401.

Weg einer Anfrage in fünf Kästen von links nach rechts: Client mit POST /breweries, Pfeil „Anfrage“ zum hervorgehobenen AuthGuard mit @UseGuards(AuthGuard), Header Authorization; Pfeil „Token gültig“ zur Pipe, Zod prüft den Body; Pfeil „Body gültig“ zum Controller mit create(dto); Pfeil zum Service, legt die Brauerei an. Vom AuthGuard nach unten in Orange: 401 Unauthorized, kein Token, Signatur falsch oder abgelaufen. Von der Pipe nach unten: 400 Bad Request, Felder fehlen oder sind ungültig. Darunter: Der Guard läuft vor der Pipe: Ohne Token und mit leerem Body antwortet der

Ein Guard ist eine Klasse mit der Methode canActivate

Den Weg einer Anfrage kennst du aus Einheit 5, die Pipe aus Einheit 7. Der Guard steht davor.

- canActivate gibt true zurück, dann läuft die Anfrage weiter. Wirft sie eine UnauthorizedException, antwortet NestJS mit 401.
- Der AuthGuard legt den Payload unter request.user ab. Der Controller liest dort, wer angemeldet ist.
- @UseGuards(AuthGuard) über einer Methode schützt nur diesen Endpunkt, über der Klasse alle Endpunkte des Controllers.
- @ApiBearerAuth() markiert den Endpunkt in Swagger UI als geschützt.

Dokumentation: https://docs.nestjs.com/guards

Beispiel: Login und AuthGuard an den Brauereien

Übernimm das Live-Coding in dein Projekt aus Einheit 12, bevor du mit den Aufgaben beginnst.

1. Installiere die Pakete (Skript 13.10).
2. Lege die Entities User, Role und UserRole im UsersModule an (Skript 13.11 bis 13.13).
3. Erzeuge die Migrationen mit Tabellen und Beispiel-Usern (Skript 13.14 bis 13.16).
4. Lege AuthModule, AuthService und AuthController mit dem Login an (Skript 13.17 bis 13.19).
5. Schreibe den AuthGuard und schütze POST /breweries (Skript 13.20 bis 13.22).
6. Ergänze Swagger und prüfe in Postman (Skript 13.23).

Beispiel, Schritt 1: Pakete installieren

BASH

```
npm install @nestjs/jwt@12.0.3 bcryptjs@3.0.3
```

Beispiel, Schritt 2: Die Entity User

TYPESCRIPT · SRC/USERS/USER.ENTITY.TS

```typescript
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  // Anmeldename; die Datenbank lehnt einen zweiten gleichen Namen ab
  @Column({ unique: true })
  name: string;

  // Nur der Hash, nie das Passwort; erscheint in keiner Antwort
  @Column()
  passwordHash: string;
}
```

Beispiel, Schritt 2: UserRole ordnet Rollen in einer eigenen Tabelle zu

TYPESCRIPT · SRC/USERS/USER-ROLE.ENTITY.TS

```typescript
// RBAC: Die Zuordnung steht in einer eigenen Tabelle, nicht am User
@Entity()
export class UserRole {
  @PrimaryColumn()
  userId: number;

  @PrimaryColumn()
  roleId: number;

  // Fremdschlüssel userId → user.id; mit dem User fallen seine Rollen weg
  @ManyToOne(() => User, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'userId' })
  user: User;
```

Beispiel, Schritt 2: Rollen und UsersModule vervollständigen

RBAC (Role-Based Access Control) heißt: Rechte hängen an Rollen, und die Zuordnung steht in einer eigenen Tabelle. Eine Anfrage, die einen User ändert, ändert so nie seine Rechte. Einheit 14 prüft die Rollen.

- Lege zuerst UsersModule und UsersService mit npx nest generate an.
- role.entity.ts: id mit @PrimaryGeneratedColumn, name mit @Column({ unique: true }), Werte admin oder user.
- UserRole endet mit einer zweiten Relation role zu Role über roleId, ohne onDelete.
- UsersModule: TypeOrmModule.forFeature([User, Role, UserRole]) unter imports, UsersService unter exports.
- UsersService: findByName(name) liefert this.usersRepository.findOneBy({ name }), also User oder null.
- data-source.ts: User, Role und UserRole unter entities.

Beispiel, Schritt 3: Migrationen für Tabellen und Beispiel-User anlegen

```
$ npm run migration:generate -- src/migrations/CreateUsers
Migration …/src/migrations/1792675018866-CreateUsers.ts
  has been generated successfully.

$ npx typeorm migration:create src/migrations/SeedUsers
Migration …/src/migrations/1792675025666-SeedUsers.ts
  has been generated successfully.
```

Beispiel, Schritt 3: Die Migration legt Rollen und Beispiel-User an

TYPESCRIPT · SRC/MIGRATIONS/1792675025666-SEEDUSERS.TS

```typescript
public async up(queryRunner: QueryRunner): Promise<void> {
  await queryRunner.manager.insert(Role, [
    { id: 1, name: 'admin' },
    { id: 2, name: 'user' },
  ]);
  await queryRunner.manager.insert(User, [
    {
      id: 1,
      name: 'admin',
      passwordHash:
        '$2b$10$o6W5KznaQaytXGZ1itv87ODpeic6pzT91PoVsjnx5FyGIMimNJW/q',
    },
```

Beispiel, Schritt 3: Die Migration mit den Beispiel-Usern vervollständigen

Die Zugangsdaten stehen in users.txt im Lösungsstand: admin mit admin123 und der Rolle admin, gast mit gast123 und der Rolle user.

- Erzeuge den Hash für gast einmal mit node -e "console.log(require('bcryptjs').hashSync('gast123', 10))".
- Jeder Aufruf ergibt wegen des Salts einen anderen Hash; alle passen zum Passwort.
- Lege gast mit id 2 an. Danach ordnet insert(UserRole, …) admin die Rolle 1 und gast die Rolle 2 zu.
- down löscht in umgekehrter Reihenfolge: UserRole, dann User 1 und 2, dann Role 1 und 2.
- Führe npm run db:reset aus. Der Viewer zeigt je zwei Zeilen in user, role und user_role.

Beispiel, Schritt 4: Das AuthModule stellt JwtService bereit

TYPESCRIPT · SRC/AUTH/AUTH.MODULE.TS

```
@Module({
  imports: [
    UsersModule,
    JwtModule.register({
      // JwtService in jedem Module, ohne Import
      global: true,
      // Bis Einheit 14 im Code; danach aus einer Umgebungsvariable
      secret: 'beer-api-secret-einheit-13',
      signOptions: { expiresIn: '1h' },
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService],
})
export class AuthModule {}
```

Beispiel, Schritt 4: Der Login vergleicht den Hash und signiert ein Token

TYPESCRIPT · SRC/AUTH/AUTH.SERVICE.TS

```
constructor(
  private readonly usersService: UsersService,
  private readonly jwtService: JwtService,
) {}

async login(dto: LoginDto): Promise<{ accessToken: string }> {
  const user = await this.usersService.findByName(dto.name);
  // compare hasht das Passwort mit dem Salt aus dem Hash und vergleicht
  if (!user || !(await bcrypt.compare(dto.password, user.passwordHash))) {
    throw new UnauthorizedException('Invalid name or password');
  }
  const payload: JwtPayload = { sub: user.id, name: user.name };
  return { accessToken: await this.jwtService.signAsync(payload) };
}
```

Beispiel, Schritt 4: POST /auth/login antwortet mit 200

TYPESCRIPT · SRC/AUTH/AUTH.CONTROLLER.TS

```
@ApiTags('auth')
@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  // status 200: Der Login legt keine Ressource an
  @Post('login')
  @ZodResponse({ status: 200, type: AccessTokenDto })
  @ApiBadRequestResponse({ type: ErrorDto })
  @ApiUnauthorizedResponse({ type: ErrorDto })
  login(@Body() dto: LoginDto) {
    return this.authService.login(dto);
  }
}
```

Beispiel, Schritt 5: Zwei Typen beschreiben Payload und Anfrage

TYPESCRIPT · SRC/AUTH/AUTH.GUARD.TS

```
// Inhalt des Tokens: sub ist die id des Users
export type JwtPayload = { sub: number; name: string };

// Anfrage nach dem Guard: request.user enthält den Payload
export type AuthRequest = Request & { user: JwtPayload };

@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly jwtService: JwtService) {}
```

Beispiel, Schritt 5: Der AuthGuard liest das Token und prüft es

TYPESCRIPT · SRC/AUTH/AUTH.GUARD.TS

```typescript
// Läuft vor dem Controller; eine Exception beendet die Anfrage
async canActivate(context: ExecutionContext): Promise<boolean> {
  const request = context.switchToHttp().getRequest<AuthRequest>();
  // Header "Authorization: Bearer <token>"
  const header = request.headers.authorization ?? '';
  const [type, token] = header.split(' ');
  if (type !== 'Bearer' || !token) {
    throw new UnauthorizedException();
  }
  try {
    // Prüft Signatur und Ablauf (exp)
    request.user = await this.jwtService.verifyAsync(token);
  } catch {
    throw new UnauthorizedException();
  }
  return true;
```

Beispiel, Schritt 6: Ein Decorator schützt POST /breweries

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
// Nur mit gültigem Token; ohne Token antwortet der Guard mit 401
@Post()
@UseGuards(AuthGuard)
@ApiBearerAuth()
@ZodResponse({ status: 201, type: BreweryDto })
@ApiBadRequestResponse({ type: ErrorDto })
@ApiUnauthorizedResponse({ type: ErrorDto })
create(@Body() dto: CreateBreweryDto) {
  return this.breweriesService.create(dto);
}
```

Beispiel, Schritt 4 bis 7: Dateien und Werkzeuge vervollständigen

- LoginDto: name und password, beide z.string().min(1). AccessTokenDto: accessToken als z.string().
- Lege den Guard mit npx nest generate guard auth an. Request importierst du mit import type aus express.
- Schritt 7: Ergänze im DocumentBuilder in main.ts .addBearerAuth(). Swagger UI zeigt dann den Button **Authorize**.
- Unter **Authorize** fügst du das Token ohne das Wort Bearer ein.
- In Postman setzt du im Tab **Authorization** den **Auth Type Bearer Token** oder den Header Authorization: Bearer {{tokenAdmin}}.
- Prüfe: POST /breweries ohne Token ergibt 401, mit dem Token von admin 201, GET /breweries ohne Token 200.

Aufgabe 13.1 · Registrieren und anmelden

ZIEL

Gäste legen ein eigenes Konto an und melden sich damit an.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel. Datenmodell in Skript 13.25, Anfragen in 13.26, Regeln in 13.29. Dokumentation für NestJS 11: https://github.com/nestjs/docs.nestjs.com/blob/11.0.0/content/security/authentication.md

ZIELE

- POST /auth/register speichert den Namen und einen Hash des Passworts.
- Jeder neue User bekommt die Rolle user.
- Ein vergebener Name ergibt 409 im Format der anderen Fehler.
- Die Antwort enthält id und name, kein Passwort und keinen passwordHash.

FERTIG, WENN

- POST /auth/register mit lena antwortet mit 201, ein zweites Mal mit 409.
- Der Login mit lena liefert ein Token.

Datenmodell: User, Role und UserRole; die ids vergibt die Datenbank

Tabelle	Feld	Typ	Bedeutung
user	name	String	Anmeldename, eindeutig
user	passwordHash	String	Hash des Passworts; erscheint in keiner Antwort
role	name	String	admin (id 1) oder user (id 2), eindeutig
user_role	userId, roleId	Ganzzahl	Zuordnung eines Users zu einer Rolle; beide zusammen Primärschlüssel
Antwort	id, name	Ganzzahl, String	Was POST /auth/register zurückgibt

Anfragen zu Aufgabe 13.1

BEISPIEL

- Setze die Datenbank mit npm run db:reset zurück.
- Sende die Anfragen in dieser Reihenfolge.
- lena bekommt die id 3.
- Die Rolle von lena zeigt der Viewer in user_role.

```
POST /auth/register
{ "name": "lena",
  "password": "lena-passwort" }
→ 201 {"id":3,"name":"lena"}

POST /auth/register  (noch einmal)
→ 409 "User lena already exists"

POST /auth/login  mit lena
→ 200 {"accessToken":"eyJ…"}

POST /auth/login  falsches Passwort
→ 401
```

Aufgabe 13.2 · Bewerten nur mit Token

ZIEL

Nur angemeldete Gäste bewerten, und jede Bewertung gehört dem User aus dem Token.

AUSGANGSLAGE

Aufgabe 13.1 ist gelöst. Anfragen in Skript 13.28, Regeln in Skript 13.29. Dokumentation: https://docs.nestjs.com/controllers#request-object und https://zod.dev/api#strict

ZIELE

- POST /beers/:id/ratings ist mit dem AuthGuard geschützt.
- Der Controller nimmt die userId aus request.user.sub.
- Das Schema einer Bewertung kennt keine userId mehr und lehnt zusätzliche Felder ab.
- Die Antwort enthält weiter die userId.
- Alle GET-Anfragen bleiben ohne Token möglich.

FERTIG, WENN

- Ohne, mit ungültigem und mit abgelaufenem Token antwortet POST /beers/1/ratings mit 401.
- Mit dem Token von lena gehört die Bewertung lena.

Anfragen zu Aufgabe 13.2

BEISPIEL

- Melde lena an und speichere das Token.
- Sende die Bewertungen mit Authorization: Bearer und dem Token.
- Prüfe danach ohne Token, dass Lesen geht.
- Ein abgelaufenes Token erzeugst du wie in Skript 13.29.

```
POST /beers/1/ratings  ohne Token     → 401
POST /beers/1/ratings  Bearer abc     → 401

POST /beers/1/ratings  Token von lena
{ "score": 4, "comment": "Süffig" }
→ 201, userId 3

POST /beers/1/ratings  Token von lena
{ "userId": 1, "score": 5 }   → 400

GET /beers/1/ratings  → 200, eine Bewertung
GET /beers/1          → 200, averageRating 4
```

Regeln zu Aufgabe 13.1 und 13.2

- Eine Registrierung vergibt immer die Rolle user. Weitere Rollen vergibt nur ein admin; das baut Einheit 14.
- „Ohne gültiges Token“ heißt: ohne Token, mit ungültigem oder mit abgelaufenem Token.
- score ist eine ganze Zahl von 1 bis 5.
- Ein User bewertet ein Bier höchstens einmal; eine zweite Bewertung ersetzt die erste.
- Die userId einer Bewertung stammt aus dem Token. Eine userId im Body lehnt der Server mit 400 ab.

Ein abgelaufenes Token erzeugst du so: Setze expiresIn im AuthModule auf '30s', melde dich an, warte 30 Sekunden und sende die Bewertung. Setze den Wert danach auf '1h' zurück. Die Collection enthält dafür ein abgelaufenes Token von lena.

Zusatz 13.3 · Das Tor im Capture the Flag

ZIEL

Du öffnest Level 10 des Capture the Flag mit einem Bearer-Token.

AUSGANGSLAGE

Der CTF-Server aus Einheit 3 läuft auf http://localhost:3001. Du hast dein Team-Token und das Feld token aus Level 8 (Einheit 4). GET http://localhost:3001/ctf/10 mit dem Header X-Team antwortet mit 401.

ZIELE

- Lies, welches Verfahren der Anmeldung der Server in der Antwort 401 erwartet.
- Sende das Token aus Level 8 so, wie er es verlangt.

FERTIG, WENN

- GET /ctf/10 antwortet mit 200 und einem Flag.

Selbstkontrolle mit Postman

Schreibe die Tests wie seit Einheit 11 selbst (Skript 11.24). Neu ist, dass Anfragen ein Token aus einer früheren Antwort brauchen. Speichere das Token im Test des Logins als Collection-Variable wie in Skript 13.32. Setze es in den folgenden Anfragen im Header Authorization: Bearer {{tokenLena}}. Die Reihenfolge der Anfragen ist dann fest. Setze die Datenbank vor jedem Lauf mit npm run db:reset zurück.

Der Ordner der Einheit 13 in der Collection Web-Programmierung zeigt zum Vergleich, welche Fälle eine vollständige Prüfung enthält, je Aufgabe in einem Ordner. Zusatz 13.3 steht im Ordner der Einheit 4 unter „Zusatz 13.3 – Tor (Level 10)“.

Dokumentation: https://learning.postman.com/docs/tests-and-scripts/write-scripts/variables-list/

Ein Test speichert das Token aus dem Login für die folgenden Anfragen

JAVASCRIPT

```
pm.test('Status 200', () => {
  pm.expect(pm.response.code, 'Stimmen Name und Passwort?').to.eql(200);
});
const weiter = pm.response.code === 200 ? pm.test : pm.test.skip;
weiter('Feld accessToken', () => {
  const token = pm.response.json().accessToken;
  pm.expect(typeof token, 'Was gibt login zurück?').to.eql('string');
  // Für die folgenden Anfragen: Bearer {{tokenLena}}
  pm.collectionVariables.set('tokenLena', token);
});
```

Hinweise zu Aufgabe 13.1

- Stufe 1: Der Login im Beispiel liest einen User und vergleicht mit bcrypt. Die Registrierung macht den umgekehrten Weg.
- Stufe 2: Welche Methode von bcrypt erzeugt einen Hash? Welche Tabellen braucht ein neuer User außer user?
- Stufe 3: Prüfe vor dem Speichern mit findByName, ob es den Namen gibt. Die Antwort braucht ein eigenes Schema.

Hinweise zu Aufgabe 13.2

- Stufe 1: Das Beispiel schützt POST /breweries. Derselbe Decorator gehört an genau einen Endpunkt der Biere.
- Stufe 2: Wo legt der Guard den Payload ab? Folge der userId vom Controller über die Services bis zum Speichern.
- Stufe 3: Gib die userId als eigenen Parameter an rate und create weiter, statt sie aus dem Schema zu lesen.

Hinweise zu Zusatz 13.3

- Stufe 1: Lies alle Header der Antwort 401, nicht nur den Body.
- Stufe 2: Welcher Header nennt das Verfahren, und welcher Header der Anfrage trägt das Token?
- Stufe 3: Skript 13.6 nennt den Header für Tokens. Welches Level hat dir ein Token gegeben?

Typische Fehler zu Aufgabe 13.1

Symptom	Ursache
Antwort von register enthält "passwordHash":"$2b$10$…"	Der Endpunkt gibt die Entity ohne Antwortschema zurück; @ZodResponse mit einem Schema aus id und name fehlt.
Zweites register mit lena ergibt 500, im Terminal „SqliteError: UNIQUE constraint failed: user.name“	register prüft nicht, ob es den Namen gibt; die Datenbank lehnt den zweiten Namen ab.
register ergibt 201, der Login mit dem richtigen Passwort 401 „Invalid name or password“	register speichert das Passwort statt des Hashes; compare findet keinen Hash.

Typische Fehler zu Aufgabe 13.2

Symptom	Ursache
Bewerten ohne Token ergibt 500, im Terminal „TypeError: Cannot read properties of undefined (reading 'sub')“	Der Guard fehlt am Endpunkt; request.user ist leer.
Bewerten mit userId im Body ergibt 201	Das Schema der Bewertung lehnt zusätzliche Felder nicht ab.
Bewerten ergibt 201, die Antwort enthält keine userId	Das Antwortschema übernimmt seine Felder aus dem Schema der Bewertung; dort fehlt userId jetzt.
GET /beers/1 ohne Token ergibt 401	@UseGuards steht über dem ganzen BeersController.

Das solltest du jetzt können

- ✓ Erklären, warum Passwörter nur als Hash mit Salt gespeichert werden
- ✓ Ein JWT ausstellen und Header, Payload und Signatur lesen
- ✓ Einen Endpunkt mit einem Guard schützen
- ✓ In Postman ein Token holen und als Bearer senden
- ✓ 401 begründen und von 403 abgrenzen

Als Nächstes: Autorisierung – nur der Betreiber ändert Brauereien, Biere und Rollen.

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 14

Autorisierung und sichere Grundeinstellungen · Skript

Worum es in dieser Einheit geht

Seit Einheit 13 weiß die Beer API, wer eine Anfrage stellt. Was diese Person darf, prüft sie noch nicht: Ohne Anmeldung benennt jeder eine Brauerei um, und jeder Gast legt Biere an. Die regionalen Brauereien brauchen Stammdaten, die nur der Betreiber pflegt. Die Aufgaben setzen diese Story um:

Als Betreiber möchte ich, dass nur der Betreiber Stammdaten und Rollen ändert, damit niemand die App manipuliert oder sich selbst Rechte gibt.

Am Ende der Einheit kannst du:

- einen Endpunkt auf eine Rolle beschränken,
- 401 und 403 begründen,
- Secrets und Beispiel-User aus dem Betrieb heraushalten.

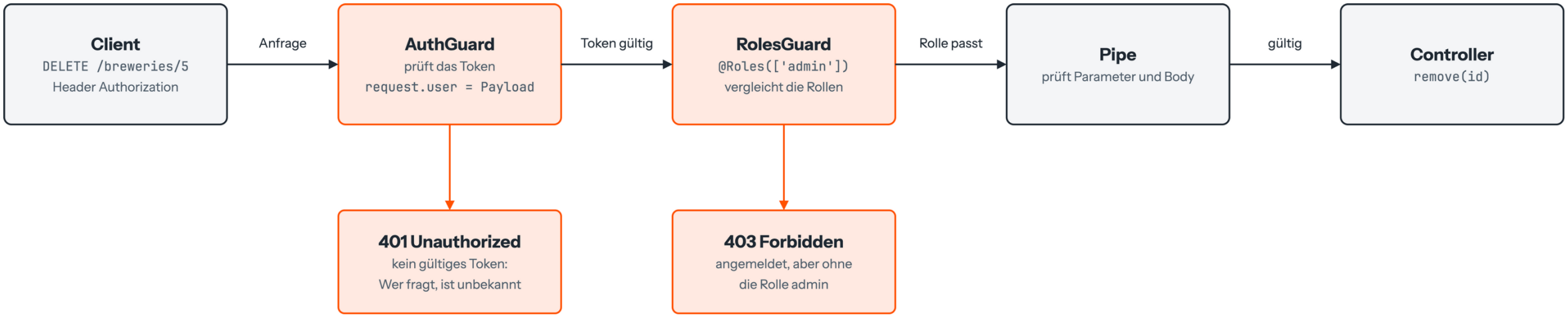
Autorisierung prüft, was eine angemeldete Person darf

RBAC steht in Skript 13.13, die Begriffe der Autorisierung auf Folie 14.7 der Vorlesung, 401 und 403 auf Folie 13.5.

- 403 Forbidden heißt: Der Server kennt die Person, und ihr fehlt das Recht.
- Grundsatz nach OWASP: Was nicht öffentlich ist, ist verboten, bis eine Regel es erlaubt.
- Vereinfachung: Die Beer API setzt Guards je Endpunkt. Ein Endpunkt ohne Guard bleibt offen.
- Diesen Grundsatz setzte erst ein globaler Guard vollständig um.

Dokumentation: https://datatracker.ietf.org/doc/html/rfc9110#section-15.5.4

Erst stellt der AuthGuard fest, wer fragt; dann prüft der RolesGuard die Rolle



@UseGuards(AuthGuard, RolesGuard): NestJS ruft die Guards von links nach rechts auf. Der erste, der ablehnt, beendet die Anfrage.

Weg einer Anfrage in fünf Kästen von links nach rechts: Client mit DELETE /breweries/5 und Header Authorization; Pfeil „Anfrage“ zum hervorgehobenen AuthGuard, prüft das Token, request.user = Payload; Pfeil „Token gültig“ zum hervorgehobenen RolesGuard mit @Roles und der Rolle admin, vergleicht die Rollen; Pfeil „Rolle passt“ zur Pipe, prüft Parameter und Body; Pfeil „gültig“ zum Controller mit remove(id). Vom AuthGuard nach unten in Orange: 401 Unauthorized, kein gültiges Token, wer fragt, ist unbekannt. Vom RolesGuard nach unten in Orange: 403 Forbidden, angemeldet, aber ohne die Rolle

Der RolesGuard vergleicht die Rollen im Token mit der Rolle am Endpunkt

Den Guard kennst du aus Einheit 13 (Skript 13.7 und 13.8).

- @Roles(['admin']) speichert nur Daten an der Methode. Der Decorator prüft nichts.
- Der RolesGuard liest diese Daten mit dem Reflector von NestJS und vergleicht sie mit request.user.roles.
- Gibt canActivate false zurück, antwortet NestJS mit 403.
- Der Login schreibt die Rollen in den Payload. Der Guard fragt deshalb nicht bei jeder Anfrage die Datenbank.
- Folge: Eine neue oder entzogene Rolle gilt erst mit dem nächsten Token.

Dokumentation für NestJS 11: https://github.com/nestjs/docs.nestjs.com/blob/11.0.0/content/guards.md

Ein strenges Schema und Parameter schützen vor Mass Assignment und Injection

Alle zehn Kategorien mit ihrem Bezug zur Beer API stehen auf den Folien 14.9 und 14.10 der Vorlesung.

- Mass Assignment: Der Client sendet ein Feld, das er nicht setzen darf, etwa roles bei der Registrierung.
- Ein Schema mit .strict() lehnt ein solches Feld mit 400 ab.
- Injection: Code setzt SQL aus Text und Eingabe zusammen. TypeORM sendet Werte bei find und findBy getrennt vom Befehl.
- Zod prüft Typ und Form einer Eingabe. Ob ein String gefährlich ist, erkennt es nicht.
- CORS regelt, ob eine Seite einer anderen Origin die Antwort im Browser lesen darf.

Dokumentation: https://owasp.org/Top10/2025/

Secrets und Beispiel-User gehören nicht in den Betrieb

Welche Werte in Entwicklung und Betrieb gelten, zeigt Folie 14.13 der Vorlesung.

- Wer das Secret kennt, stellt Tokens mit beliebigen Rollen aus.
- Ein Secret im Code steht in jeder Kopie des Repositorys, auch in alten Commits. .env bekommt deshalb einen neuen Wert.
- Eine Umgebungsvariable ist ein Wert, den das Betriebssystem einem Prozess mitgibt. Node.js liest sie unter process.env.
- .env sammelt solche Werte lokal und steht in .gitignore. .env.example nennt die Schlüssel ohne echte Werte.
- Der Betreiber setzt im Betrieb NODE_ENV=production. Fehlt die Variable, legt die Migration die Beispiel-User doch an.

Dokumentation: https://docs.nestjs.com/application/configuration

Beispiel: Secret aus .env und Rollen an den Brauereien

Übernimm das Live-Coding in dein Projekt aus Einheit 13, bevor du mit den Aufgaben beginnst.

1. Installiere das Paket und lege .env.example und .env an (Skript 14.9).
2. Lies das Secret über den ConfigService (Skript 14.10).
3. Lege Beispiel-User nur in der Entwicklung an (Skript 14.11).
4. Schreibe die Rollen beim Login in den Payload (Skript 14.12 und 14.13).
5. Schreibe den Decorator @Roles und den RolesGuard (Skript 14.14 und 14.15).
6. Schütze POST, PATCH und DELETE /breweries und prüfe in Postman (Skript 14.16 und 14.17).

Beispiel, Schritt 1: Paket installieren und Vorlage für .env anlegen

BASH

```
npm install @nestjs/config@12.0.1
```

Beispiel, Schritt 2: Das JwtModule liest das Secret aus .env

TYPESCRIPT · SRC/AUTH/AUTH.MODULE.TS

```
@Module({
  imports: [
    UsersModule,
    // Async: Das Secret steht erst fest, wenn ConfigService .env gelesen hat
    JwtModule.registerAsync({
      // JwtService in jedem Module, ohne Import
      global: true,
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        // Fehlt JWT_SECRET, startet der Server nicht
        secret: config.getOrThrow<string>('JWT_SECRET'),
        signOptions: { expiresIn: '1h' },
      }),
    }),
  ],
```

Beispiel, Schritt 3: Die Migration legt Beispiel-User nur in der Entwicklung an

TYPESCRIPT · SRC/MIGRATIONS/1792675025666-SEEDUSERS.TS

```
public async up(queryRunner: QueryRunner): Promise<void> {
  await queryRunner.manager.insert(Role, [
    { id: 1, name: 'admin' },
    { id: 2, name: 'user' },
  ]);
  // Bekannte Passwörter nur in der Entwicklung; im Betrieb gilt
  // NODE_ENV=production, dort gibt es keine Beispiel-User
  if (process.env.NODE_ENV === 'production') {
    return;
  }
  await queryRunner.manager.insert(User, [
```

Beispiel, Schritt 4: Der UsersService liefert die Rollen eines Users

TYPESCRIPT · SRC/USERS/USERS.SERVICE.TS

```typescript
// Namen der Rollen eines Users, zum Beispiel ['admin']
async findRoles(userId: number): Promise<string[]> {
  const userRoles = await this.userRolesRepository.find({
    where: { userId },
    relations: { role: true },
  });
  return userRoles.map((userRole) => userRole.role.name);
}
```

Beispiel, Schritt 4: Der Login schreibt die Rollen in den Payload

TYPESCRIPT · SRC/AUTH/AUTH.SERVICE.TS

```
// Rollen ins Token; der RolesGuard fragt die Datenbank nicht
const roles = await this.usersService.findRoles(user.id);
const payload: JwtPayload = {
  sub: user.id,
  name: user.name,
  roles,
};
return { accessToken: await this.jwtService.signAsync(payload) };
```

Beispiel, Schritt 5: Der Decorator @Roles speichert die Rollen am Endpunkt

TYPESCRIPT · SRC/AUTH/ROLES.DECORATOR.TS

```typescript
import { Reflector } from '@nestjs/core';

// @Roles(['admin']): Rollen, von denen der User eine braucht
export const Roles = Reflector.createDecorator<string[]>();
```

Beispiel, Schritt 5: Der RolesGuard vergleicht Endpunkt und Token

TYPESCRIPT · SRC/AUTH/ROLES.GUARD.TS

```
// Läuft nach dem AuthGuard: request.user ist gesetzt
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    // Rollen aus @Roles; ohne Decorator ist alles erlaubt
    const roles = this.reflector.get(Roles, context.getHandler());
    if (!roles) {
      return true;
    }
    const request = context.switchToHttp().getRequest<AuthRequest>();
    // false: NestJS antwortet mit 403
    return roles.some((role) => request.user.roles.includes(role));
  }
}
```

Beispiel, Schritt 6: Zwei Guards und eine Rolle schützen DELETE /breweries/:id

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```
@Delete(':id')
@UseGuards(AuthGuard, RolesGuard)
@Roles(['admin'])
@ApiBearerAuth()
@HttpCode(HttpStatus.NO_CONTENT)
@ApiNoContentResponse()
@ApiBadRequestResponse({ type: ErrorDto })
@ApiUnauthorizedResponse({ type: ErrorDto })
@ApiForbiddenResponse({ type: ErrorDto })
@ApiNotFoundResponse({ type: ErrorDto })
@ApiConflictResponse({ type: ErrorDto })
remove(@Param('id', ParseIntPipe) id: number): Promise<void> {
```

Beispiel, Schritt 1 bis 6: Dateien vervollständigen und prüfen

- Schritt 1: Kopiere .env.example nach .env und ersetze den Wert durch eine lange, zufällige Zeichenkette.
- Schritt 2: Trage ConfigModule.forRoot({ isGlobal: true }) als ersten Eintrag unter imports in AppModule ein.
- Schritt 5: Lege die Dateien mit npx nest generate decorator auth/roles --flat und npx nest generate guard auth/roles --flat an.
- Schritt 6: Dieselben drei Decorators gehören an POST und PATCH /breweries, dazu @ApiUnauthorizedResponse und @ApiForbiddenResponse.
- Prüfe nach npm run db:reset: Lege mit admin über POST /breweries die Brauerei 5 an.
- DELETE /breweries/5 ergibt dann ohne Token 401, mit gast 403, mit admin 204.

Aufgabe 14.1 · Biere schützen

ZIEL

Nur der Betreiber legt Biere an, ändert und löscht sie; Gäste bewerten weiter.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel. Vorlage in Skript 14.19, Anfragen in 14.20, Regeln in 14.21. Dokumentation für NestJS 11: https://github.com/nestjs/docs.nestjs.com/blob/11.0.0/content/guards.md

ZIELE

- Trage in die Vorlage je Endpunkt offen, angemeldet, admin oder einen Strich ein.
- POST, PATCH und DELETE /beers verlangen die Rolle admin.
- Bewerten bleibt für jeden angemeldeten User erlaubt.
- Alle GET-Anfragen bleiben ohne Token möglich.

FERTIG, WENN

- POST /beers antwortet ohne Token mit 401, mit gast mit 403, mit admin mit 201.
- gast bewertet ein Bier mit 201.

Vorlage Endpunkt und Rolle: Wer darf welchen Endpunkt aufrufen?

Pfad	GET	POST	PATCH	DELETE
/breweries, /breweries/:id	offen	admin	admin	admin
/beers, /beers/:id				
/beers/:id/ratings				
/auth/register, /auth/login				
/users/me, /users/:id/roles				

Anfragen zu Aufgabe 14.1

BEISPIEL

- Setze die Datenbank mit npm run db:reset zurück.
- Melde admin mit admin123 und gast mit gast123 an.
- Sende jede Anfrage ohne Token, mit gast und mit admin.
- PATCH und DELETE nutzen die id aus dem POST.

```
POST /beers
{ "name": "Wildbräu Grafing Pils",
  "style": "pils", "abv": 4.9,
  "breweryId": 2 }
→ ohne 401, gast 403, admin 201

PATCH /beers/5  { "abv": 5.1 }
→ ohne 401, gast 403, admin 200

DELETE /beers/5
→ ohne 401, gast 403, admin 204

POST /beers/1/ratings  { "score": 4 }
→ gast 201, admin 201
```

Regeln zu Aufgabe 14.1 und 14.2

- Nur ein User mit der Rolle admin legt Biere und Brauereien an, ändert und löscht sie; jeder angemeldete User bewertet.
- Eine Registrierung vergibt immer die Rolle user. Weitere Rollen vergibt nur ein admin.
- Ohne gültiges Token antwortet ein geschützter Endpunkt mit 401, mit Token ohne passende Rolle mit 403.
- Das Secret für die Tokens steht nicht im Code, sondern in einer Umgebungsvariable.
- Rollen sind admin und user; andere Werte lehnt der Server mit 400 ab.

Beispiel-User aus users.txt: admin mit admin123 und der Rolle admin, gast mit gast123 und der Rolle user.

Aufgabe 14.2 · Rollen vergibt nur der admin

ZIEL

Niemand gibt sich selbst Rechte; der admin vergibt Rollen, jeder User sieht seine eigenen.

AUSGANGSLAGE

Aufgabe 14.1 ist gelöst. Endpunkte und Antworten in Skript 14.23, Anfragen in 14.24, Regeln in 14.21. Dokumentation: https://zod.dev/api#strict

ZIELE

- Die Registrierung lehnt ein Feld roles mit 400 ab.
- POST /users/:id/roles vergibt eine Rolle, nur für admin.
- GET /users/me zeigt den angemeldeten User mit seinen Rollen aus der Datenbank.
- Keine Antwort enthält passwordHash.

FERTIG, WENN

- POST /users/2/roles antwortet mit gast mit 403, mit admin mit 201.
- GET /users/me zeigt danach die neue Rolle.

Endpunkte zu Aufgabe 14.2 im Überblick

Endpunkt	Body	Antwort	Fehler
POST /auth/register	name, password	201 wie bisher	400, wenn ein weiteres Feld wie roles im Body steht
GET /users/me	–	200 mit id, name und roles	401 ohne Token
POST /users/:id/roles	role: admin oder user	201 mit id, name und roles des Users	401, 403, 404 für eine unbekannte id, 400 für eine unbekannte Rolle
Rolle doppelt vergeben	role: eine Rolle, die der User schon hat	201, die Rolle steht einmal in roles	–

Anfragen zu Aufgabe 14.2

BEISPIEL

- Setze die Datenbank mit npm run db:reset zurück.
- Melde admin und gast an.
- Sende die Anfragen in dieser Reihenfolge.
- gast hat die id 2.

```
POST /auth/register
{ "name": "mallory", "password":
  "mallory123", "roles": ["admin"] }
→ 400

GET /users/me  Token von gast
→ 200 {"id":2,"name":"gast",
      "roles":["user"]}

POST /users/2/roles { "role": "admin" }
→ gast 403, admin 201

GET /users/me  Token von gast
→ 200, roles ["admin","user"]
```

Selbstkontrolle mit Postman

Schreibe die Tests wie seit Einheit 11 selbst (Skript 11.24). Hole die Tokens wie in Skript 13.32 und speichere sie als Collection-Variablen. Sende jede geschützte Anfrage dreimal: ohne Token, mit gast und mit admin. Speichere die id eines neuen Biers als Collection-Variable und nutze sie in PATCH und DELETE. Lege .env an und setze die Datenbank vor jedem Lauf mit npm run db:reset zurück.

Der Ordner der Einheit 14 in der Collection Web-Programmierung zeigt zum Vergleich, welche Fälle eine vollständige Prüfung enthält, je Aufgabe in einem Ordner.

Dokumentation: https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

Ein Test prüft 403 und das Fehlerformat

JAVASCRIPT

```
pm.test('Status 403', () => {
  pm.expect(pm.response.code, 'Prüft ein RolesGuard die Rolle?').to.eql(403);
});
const weiter = pm.response.code === 403 ? pm.test : pm.test.skip;
weiter('Fehlerformat mit Forbidden', () => {
  pm.expect(pm.response.json().error).to.eql('Forbidden');
});
```

Hinweise zu Aufgabe 14.1

- Stufe 1: Das Beispiel schützt die Brauereien. Vergleiche die Decorators an POST /breweries mit denen an POST /beers.
- Stufe 2: Welche Endpunkte der Biere ändern Daten? Welcher davon braucht ein Token, aber keine Rolle?
- Stufe 3: Setze Guards und Rolle an jede schreibende Methode einzeln.

Hinweise zu Aufgabe 14.2

- Stufe 1: Einheit 13 lehnt eine userId im Body einer Bewertung ab. Das Beispiel schützt POST /breweries mit einer Rolle.
- Stufe 2: Welche Methode von Zod lehnt unbekannte Felder ab? Woher kennt GET /users/me die id des angemeldeten Users?
- Stufe 3: Lege einen UsersController mit zwei Endpunkten an. Lies die Rollen für die Antwort aus der Tabelle user_role.

Typische Fehler zu Aufgabe 14.1

Symptom	Ursache
POST /beers ohne Token ergibt 500, im Terminal „TypeError: Cannot read properties of undefined (reading 'roles')“	Der AuthGuard fehlt oder steht nach dem RolesGuard; request.user ist leer.
POST /beers mit gast ergibt 201	@Roles(['admin']) oder der RolesGuard fehlt an der Methode.
POST /beers/1/ratings mit gast ergibt 403 „Forbidden resource“	@Roles(['admin']) steht auch am Bewerten.
GET /beers ohne Token ergibt 401	@UseGuards steht über dem ganzen BeersController.

Typische Fehler zu Aufgabe 14.2

Symptom	Ursache
Registrierung mit roles ergibt 201	Das RegisterSchema lehnt zusätzliche Felder nicht ab; .strict() fehlt.
GET /users/me enthält "passwordHash":"$2b$10$…"	Der Endpunkt gibt die Entity ohne Antwortschema zurück.
Zweites POST /users/2/roles mit admin ergibt 500, im Terminal „UNIQUE constraint failed: user_role.userId, user_role.roleId“	insert legt die vorhandene Zuordnung ein zweites Mal an.
GET /users/me zeigt nach POST /users/2/roles die alte Rolle	Der Endpunkt liest die Rollen aus dem Token statt aus der Datenbank.

Das solltest du jetzt können

- ✓ Einen Endpunkt mit AuthGuard, RolesGuard und @Roles auf eine Rolle beschränken
- ✓ 401 und 403 an einem Endpunkt unterscheiden und begründen
- ✓ Mass Assignment erkennen und mit .strict() verhindern
- ✓ Ein Secret aus dem Code in eine Umgebungsvariable verschieben
- ✓ Die zehn Kategorien der OWASP Top 10 der Beer API zuordnen

Als Nächstes: WebSockets – neue Bewertungen erscheinen ohne Neuladen auf einem Dashboard.

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 15

WebSockets · Skript

Worum es in dieser Einheit geht

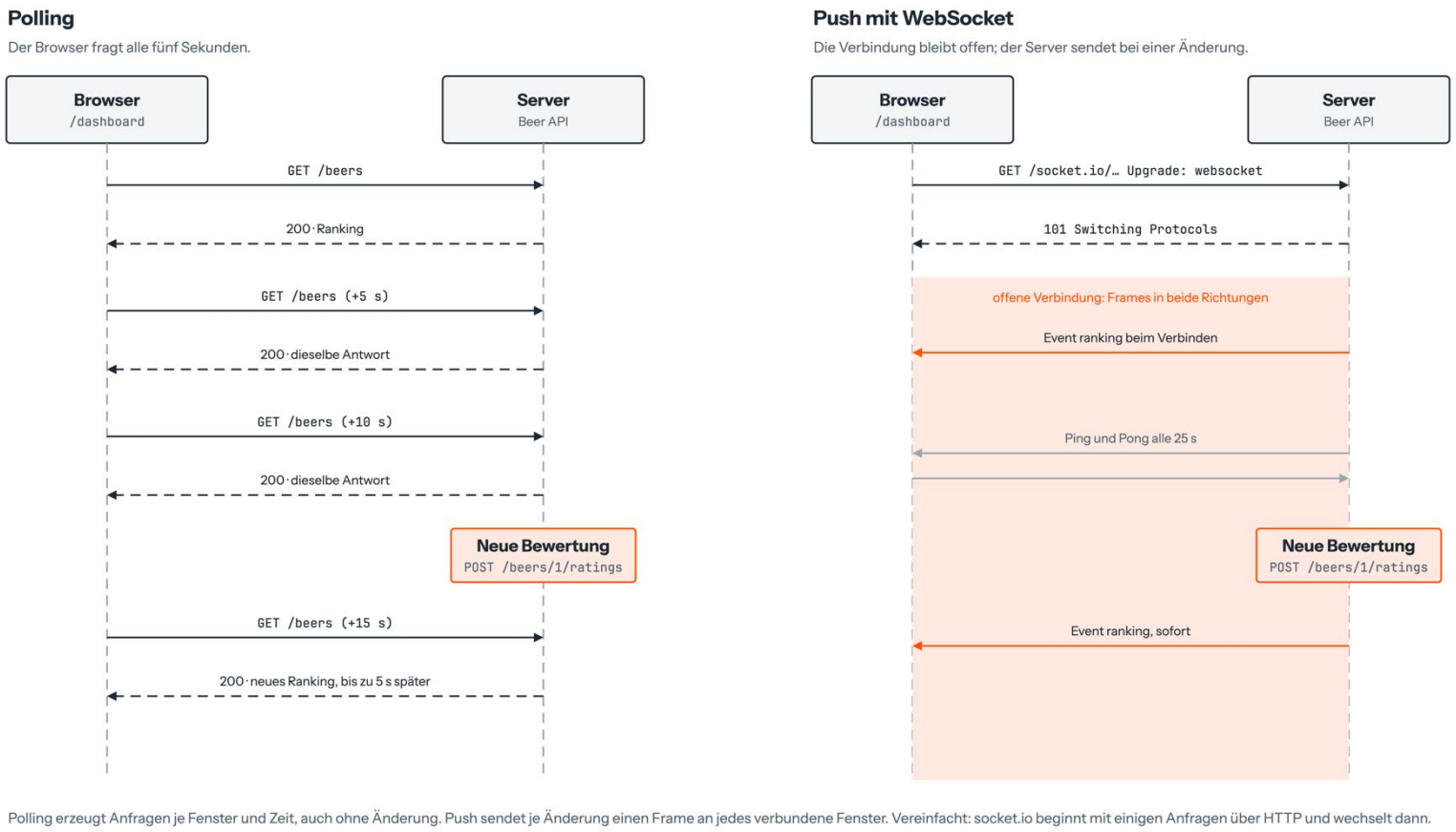
Auf Veranstaltungen zeigt ein Bildschirm die beliebtesten Biere. Heute fragt die Seite dafür alle fünf Sekunden beim Server nach, auch wenn niemand bewertet. Mit WebSockets meldet sich der Server selbst, sobald eine Bewertung eintrifft. Die Aufgaben setzen diese Story um:

Als Betreiber möchte ich die fünf bestbewerteten Biere live sehen, damit ich auf Veranstaltungen zeige, was gerade gut ankommt.

Am Ende der Einheit kannst du:

- ein Gateway bauen, das Events an verbundene Browser sendet,
- begründen, wann sich Push gegenüber Polling lohnt.

Bei Push hält der Browser eine Verbindung offen, und der Server meldet sich



Polling erzeugt Anfragen je Fenster und Zeit, auch ohne Änderung. Push sendet je Änderung einen Frame an jedes verbundene Fenster. Vereinfacht: socket.io beginnt mit einigen Anfragen über HTTP und wechselt dann.

Zwei Sequenzen nebeneinander zwischen Browser und Server. Links Polling: Der Browser sendet GET /beers sofort, nach 5 und nach 10 Sekunden und bekommt jedes Mal 200, zweimal dieselbe Antwort; nach einer neuen Bewertung kommt das neue Ranking erst mit der Anfrage nach 15 Sekunden. Rechts Push mit WebSocket: GET /socket.io/… mit Upgrade: websocket, Antwort 101 Switching Protocols. Danach ist die Verbindung offen: das Event ranking beim Verbinden, Ping und Pong alle 25 Sekunden, nach einer neuen Bewertung sofort das Event ranking. Vereinfacht: socket.io beginnt mit

Polling und Push

Bei Request und Response fragt immer der Client; der Server kann nicht von sich aus senden. Polling umgeht das mit Anfragen in festen Abständen. Push dreht die Richtung um: Der Browser öffnet einmal eine Verbindung, und der Server sendet über sie, sobald sich etwas ändert. Die Rechnung für 200 Bildschirme steht auf Folie 15.6 der Vorlesung, die Abwägung auf Folie 15.12.

- Polling kostet je Fenster und Zeit, auch ohne Änderung.
- Push kostet je Änderung und hält jede Verbindung offen.

Dokumentation: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

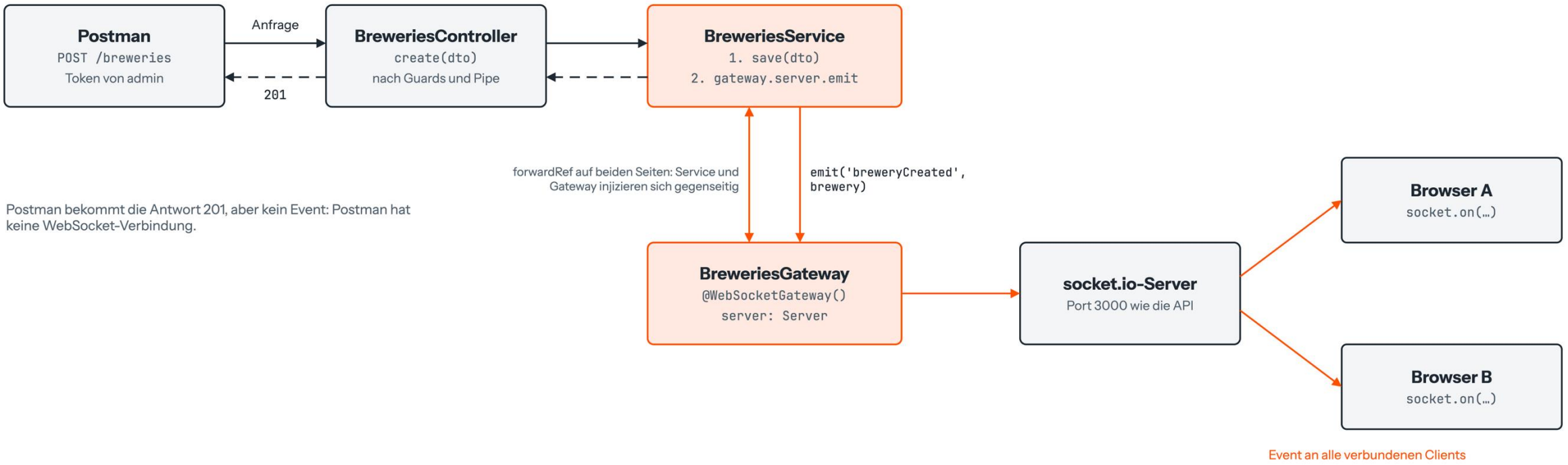
Die Verbindung im Network-Tab

Eine WebSocket-Verbindung beginnt als HTTP-Anfrage mit dem Header Upgrade: websocket. Der Server stimmt mit 101 Switching Protocols zu; danach laufen über dieselbe Verbindung Frames statt Anfragen. Die Bibliothek socket.io legt Events mit Namen darüber. Ihre Frames erklärt Folie 15.8 der Vorlesung.

- Network-Tab, Filter Socket: die Verbindung; der Reiter Messages zeigt jeden Frame.
- Filter Fetch/XHR: Anfragen an /socket.io/ mit transport=polling. So beginnt socket.io, bevor es auf WebSocket wechselt.
- Nach einer Trennung verbindet sich der Client selbst neu. Events aus der Zeit dazwischen gehen verloren.

Dokumentation: https://socket.io/docs/v4/client-api/

Der Service sendet das Event nach dem Speichern an alle verbundenen Clients



Das Event entsteht im Service nach dem Speichern. Lehnt ein Guard oder die Pipe die Anfrage ab, läuft create nicht, und es gibt kein Event.

Weg eines Events an den Brauereien: Postman sendet POST /breweries an den BreweriesController. Der BreweriesService führt 1. save(dto) und 2. gateway.server.emit aus; Postman bekommt 201, aber kein Event. Service und BreweriesGateway injizieren sich gegenseitig, mit forwardRef auf beiden Seiten. Das Gateway sendet emit('breweryCreated', brewery) über den socket.io-Server an Browser A und Browser B. Lehnt ein Guard oder die Pipe die Anfrage ab, läuft create nicht, und es gibt kein Event.

Ein Gateway in NestJS

Die Bausteine eines Gateways stehen auf Folie 15.10 der Vorlesung. Für die Aufgaben braucht es dazu:

- Ein Gateway ist ein Provider. Die Nest CLI trägt es unter providers im Module ein.
- server.emit sendet an alle verbundenen Clients, client.emit nur an einen.
- Mit dem Interface OnGatewayConnection ruft NestJS bei jeder neuen Verbindung handleConnection(client) auf.
- Braucht der Service das Gateway und das Gateway den Service, steht forwardRef an beiden Constructor-Parametern.
- socket.io wandelt die Daten eines Events in JSON um; ein Promise wird zu {}.

Dokumentation: https://docs.nestjs.com/websockets/gateways

Beispiel: ein Gateway an den Brauereien

Übernimm das Live-Coding in dein Projekt aus Einheit 14, bevor du mit den Aufgaben beginnst. Kopiere zuerst die Datei dashboard.html aus dem Ordner websockets im Material dieser Einheit in einen neuen Ordner public im Projekt.

- Schritt 1: Liefere die Seite aus dem Ordner public aus (Skript 15.9).
- Schritt 2 und 3: Installiere zwei Pakete und erzeuge das Gateway (Skript 15.10).
- Schritt 4: Sende nach dem Speichern einer Brauerei ein Event (Skript 15.11).
- Schritt 5: Empfange das Event in der Console (Skript 15.12).
- Schritt 6: Beantworte ein Event vom Client, mit forwardRef auf beiden Seiten (Skript 15.13).
- Schritt 7: Vervollständige die Dateien und prüfe mit zwei Fenstern (Skript 15.14).

Beispiel, Schritt 1: Der Server liefert Dateien aus dem Ordner public

TYPESCRIPT · SRC/MAIN.TS

```
import { NestFactory } from '@nestjs/core';
import { NestExpressApplication } from '@nestjs/platform-express';
import { join } from 'node:path';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { cleanupOpenApiDoc } from 'nestjs-zod';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  // Dateien aus public/: /dashboard liefert public/dashboard.html
  app.useStaticAssets(join(__dirname, '..', 'public'), {
    extensions: ['html'],
  });
```

Beispiel, Schritt 2 und 3: Pakete installieren, die Nest CLI erzeugt das Gateway

BASH

```
npm install @nestjs/websockets@11.2.7 \
  @nestjs/platform-socket.io@11.2.7

npx nest generate gateway breweries
CREATE src/breweries/breweries.gateway.ts (243 bytes)
UPDATE src/breweries/breweries.module.ts (640 bytes)
```

Beispiel, Schritt 4: Der Service sendet nach dem Speichern ein Event

TYPESCRIPT · SRC/BREWERIES/BREWERIES.GATEWAY.TS

```
@WebSocketGateway()
export class BreweriesGateway {
  // Der socket.io-Server; emit sendet an alle verbundenen Clients
  @WebSocketServer()
  server: Server;
```

Beispiel, Schritt 5: Die Console empfängt das Event

BEISPIEL

- Öffne http://localhost:3000/dashboard und die Console.
- Gib die zwei Zeilen ein.
- Sende POST /breweries mit dem Token von admin.
- Die Console zeigt die neue Brauerei.

```
const socket = io();
socket.on('breweryCreated', (b) => console.log(b));

POST /breweries
{ "name": "Brauerei Gold Ochsen",
  "city": "Ulm", "country": "Deutschland" }
→ 201, Console:
{name: 'Brauerei Gold Ochsen', city: 'Ulm',
 country: 'Deutschland', id: 5}
```

Beispiel, Schritt 6: Gateway und Service injizieren sich gegenseitig

TYPESCRIPT · SRC/BREWERIES/BREWERIES.GATEWAY.TS

```
constructor(
  @Inject(forwardRef(() => BreweriesService))
  private readonly breweriesService: BreweriesService,
) {}

// Event vom Client; der Rückgabewert geht als Antwort zurück
@SubscribeMessage('breweries')
findAll() {
  return this.breweriesService.findAll({ order: 'asc' });
}
```

Beispiel, Schritt 1 bis 7: Dateien vervollständigen und prüfen

- Schritt 1: Ohne extensions: ['html'] liefert nur /dashboard.html die Seite, /dashboard antwortet mit 404.
- Schritt 4: WebSocketGateway und WebSocketServer kommen aus @nestjs/websockets, Server aus socket.io.
- Schritt 4: Der Service bekommt im Constructor den Parameter private readonly breweriesGateway: BreweriesGateway.
- Schritt 6: Ersetze die Methode handleMessage aus der Nest CLI. Inject und forwardRef kommen aus @nestjs/common.
- Prüfe Schritt 6 in der Console mit socket.emit('breweries', (list) => console.log(list)).
- Schritt 7: Öffne ein zweites Fenster mit denselben zwei Zeilen. Nach POST /breweries zeigen beide Consoles die Brauerei.

Aufgabe 15.1 · Ranking beim Verbinden

ZIEL

Ein Fenster, das /dashboard öffnet, zeigt sofort das aktuelle Ranking, ohne Anfrage an /beers.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel. Daten und Regeln in Skript 15.16, Anfragen in 15.17. Dokumentation: https://docs.nestjs.com/websockets/gateways

ZIELE

- Ein Gateway für die Bewertungen sendet jedem neuen Client das Ranking.
- Das Ranking entsteht im RatingsService aus den gespeicherten Bewertungen.
- dashboard.html empfängt das Ranking als Event; setInterval entfällt.

FERTIG, WENN

- Nach den Anfragen zeigt ein neues Fenster Nattheimer Spezial · 5 und Augustiner Lagerbier Hell · 3.5.
- Der Network-Tab zeigt keine Anfrage an /beers.

Daten und Regeln zu Aufgabe 15.1 und 15.2

Das Event heißt ranking. Es trägt eine Liste von Bieren im Format von GET /beers; die Funktion render in dashboard.html zeigt sie.

- Die Seite GET /dashboard zeigt die fünf Biere mit dem höchsten averageRating; Biere ohne Bewertung fehlen.
- Zwei geöffnete Browserfenster zeigen nach derselben Bewertung dasselbe Ranking.
- Der Server sendet das Ranking nur nach einer Bewertung, nicht in festen Abständen.
- Die Durchschnittsbewertung wird auf eine Nachkommastelle gerundet; bei Gleichstand steht die kleinere id zuerst.
- Ein User bewertet ein Bier höchstens einmal; eine zweite Bewertung ersetzt die erste.
- Ein Fenster, das sich verbindet, bekommt das aktuelle Ranking sofort.

Anfragen zu Aufgabe 15.1

BEISPIEL

- Setze die Datenbank mit npm run db:reset zurück.
- Melde gast mit gast123 und admin mit admin123 an.
- Sende die drei Bewertungen.
- Öffne erst danach ein Fenster mit /dashboard.

```
POST /beers/1/ratings  gast   { "score": 5 }
POST /beers/3/ratings  gast   { "score": 4 }
POST /beers/3/ratings  admin  { "score": 3 }
→ je 201

Neues Fenster mit /dashboard:
1. Nattheimer Spezial · 5
2. Augustiner Lagerbier Hell · 3.5
```

Aufgabe 15.2 · Neues Ranking nach jeder Bewertung

ZIEL

Alle offenen Fenster zeigen nach einer Bewertung sofort dasselbe neue Ranking, ohne Neuladen.

AUSGANGSLAGE

Aufgabe 15.1 ist gelöst, die Bewertungen aus Skript 15.17 sind gespeichert. Anfragen in 15.19, Regeln in 15.16. Dokumentation: https://docs.nestjs.com/fundamentals/circular-dependency

ZIELE

- Der RatingsService sendet nach dem Speichern einer Bewertung das neue Ranking an alle Clients.
- Das Ranking zeigt höchstens fünf Biere.
- Ohne Bewertung sendet der Server nichts außer Ping und Pong.

FERTIG, WENN

- Zwei Fenster zeigen nach jeder Anfrage aus Skript 15.19 dasselbe Ranking.
- Am Ende fehlt das Augustiner Dunkel.

Anfragen zu Aufgabe 15.2

BEISPIEL

- Öffne zwei Fenster mit /dashboard.
- Sende die Anfragen der Reihe nach.
- Vergleiche nach jeder Bewertung beide Fenster.
- Die neuen Biere bekommen die ids 5 und 6.

```
POST /beers/2/ratings admin { "score": 5 }
→ Nattheimer 5, Wildbräu Helles 5, Augustiner 3.5
POST /beers/1/ratings gast { "score": 2 }
→ Wildbräu Helles 5, Augustiner 3.5, Nattheimer 2
POST /beers admin { "name": "Wildbräu Grafing Pils",
  "style": "pils", "abv": 4.9, "breweryId": 2 }
POST /beers admin { "name": "Augustiner Dunkel",
  "style": "dunkel", "abv": 5.6, "breweryId": 3 }
POST /beers/4/ratings gast { "score": 4 }
POST /beers/5/ratings gast { "score": 3 }
POST /beers/6/ratings gast { "score": 1 }
→ Wildbräu Helles 5, Farny 4, Augustiner 3.5,
  Wildbräu Pils 3, Nattheimer 2
```

Selbstkontrolle im Browser und mit Postman

Die Kriterien dieser Einheit prüfst du im Browser, weil Postman keine Events sieht. Der Ordner der Einheit 15 in der Collection Web-Programmierung legt die Bewertungen aus Skript 15.17 und 15.19 an, je Aufgabe in einem Ordner, und vergleicht das Ranking mit GET /beers. Setze die Datenbank vorher mit npm run db:reset zurück und führe die Ordner der Reihe nach aus.

- Die Fenster zeigen nach jeder Bewertung das Ranking aus dem Vergleich.
- Network-Tab, Filter Fetch/XHR: keine Anfrage an /beers.
- Reiter Messages: ein Frame ranking beim Verbinden und je Bewertung, sonst nur 2 und 3.

Hinweise zu Aufgabe 15.1

- Stufe 1: Das Beispiel sendet ein Event, wenn eine Brauerei entsteht. Wann braucht ein Fenster hier das Ranking?
- Stufe 2: Welche Methode ruft NestJS bei einer neuen Verbindung auf? Woher kennt der RatingsService die Biere zu seinen Bewertungen?
- Stufe 3: Erzeuge das Gateway im Ordner der Bewertungen. Lies die Bewertungen mit ihrem Bier und gruppiere sie je Bier.

Hinweise zu Aufgabe 15.2

- Stufe 1: Das Beispiel sendet aus BreweriesService.create. Welche Methode speichert eine Bewertung?
- Stufe 2: Welcher Teil von emit erreicht alle Clients? Was steht im Frame, wenn der Wert noch nicht berechnet ist?
- Stufe 3: Sende in create nach dem Speichern das berechnete Ranking. Setze forwardRef wie im Beispiel.

Typische Fehler zu Aufgabe 15.1

Symptom	Ursache
Ein neues Fenster bleibt leer; das Ranking erscheint erst nach der nächsten Bewertung.	handleConnection fehlt oder sendet nichts.
Das Ranking zeigt „Wildbräu Grafing Helles · null“.	Das Ranking entsteht aus allen Bieren statt aus den Bewertungen.
Der Network-Tab zeigt weiter alle fünf Sekunden GET /beers.	setInterval steht noch in dashboard.html.
Server startet nicht: „The module at index [1] of the RatingsModule "imports" array is undefined“	Das RatingsModule importiert das BeersModule, das schon das RatingsModule importiert.

Typische Fehler zu Aufgabe 15.2

Symptom	Ursache
Server startet nicht: „Nest can't resolve dependencies of the RatingsGateway (?)“	Gateway und Service injizieren sich ohne forwardRef.
Console: „Uncaught TypeError: beers.map is not a function“, Messages: 42["ranking",{}]	emit bekommt das Promise von ranking(); await fehlt.
Nach einer Bewertung fehlt sie im Ranking; erst die nächste Bewertung oder ein neues Fenster zeigt sie.	create sendet das Ranking vor dem Speichern der Bewertung.
Mit sechs bewerteten Bieren zeigt die Seite sechs.	Das Ranking wird nicht auf fünf gekürzt.

Das solltest du jetzt können

- ✓ Polling und Push an einer Seite im Network-Tab unterscheiden
- ✓ Ein Gateway mit der Nest CLI erzeugen und Events vom Client annehmen
- ✓ Aus einem Service nach dem Speichern ein Event an alle Clients senden
- ✓ Einem neuen Client beim Verbinden den aktuellen Stand senden
- ✓ Begründen, wann sich Push für eine Funktion lohnt

Als Nächstes: Templates – die Beer API liefert Biere als fertige HTML-Seiten.

PlanB. Trusted Agentic Operator

DHBW Duale Hochschule Baden-Württemberg Heidenheim

WEB-PROGRAMMIERUNG · EINHEIT 16

Dynamische serverseitige Erzeugung · Skript

Worum es in dieser Einheit geht

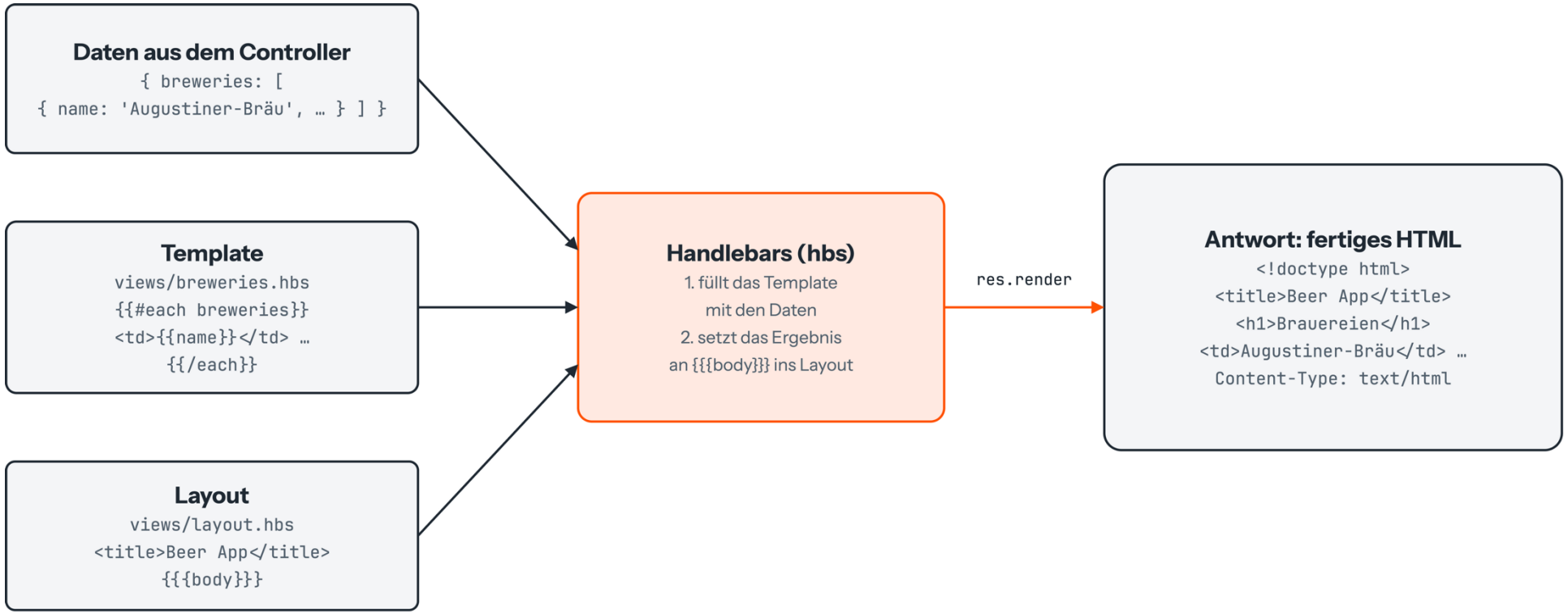
Bisher liefert die Beer API ihre Ressourcen nur als JSON. Wer die Biere auf einer Website zeigen will, braucht dafür ein Frontend. Eine kleine Brauerei hat niemanden, der es baut. Mit einer Template-Engine erzeugt der Server die fertige HTML-Seite selbst, unter demselben Pfad wie das JSON. Die Aufgaben setzen diese Story um:

Als Brauerei möchte ich die Bierliste und die Detailseite eines Biers als fertige HTML-Seite abrufen, damit ich sie ohne eigene Programmierung verlinke.

Am Ende der Einheit kannst du:

- einen Endpunkt je nach Accept JSON oder HTML liefern lassen,
- begründen, wann HTML vom Server mehr Leser erreicht als ein Frontend.

Die Template-Engine setzt Daten in das Template und das Ergebnis in das Layout



{{name}} maskiert Zeichen mit Bedeutung in HTML: aus < wird <. {{{body}}} setzt HTML unverändert ein; das Layout bekommt so das fertige Template.

Fehlt ein Feld in den Daten, bleibt die Stelle leer. Die Engine meldet keinen Fehler.

Drei Eingaben: Daten aus dem Controller mit der Liste breweries, das Template views/breweries.hbs mit {{#each breweries}} und <td>{{name}}</td>, das Layout views/layout.hbs mit {{{body}}}. Handlebars (hbs) füllt 1. das Template mit den Daten und setzt 2. das Ergebnis an {{{body}}} ins Layout. Über res.render geht fertiges HTML mit Content-Type: text/html hinaus. {{name}} maskiert Zeichen mit Bedeutung in HTML, {{{body}}} setzt HTML unverändert ein. Fehlt ein Feld in den Daten, bleibt die Stelle leer.

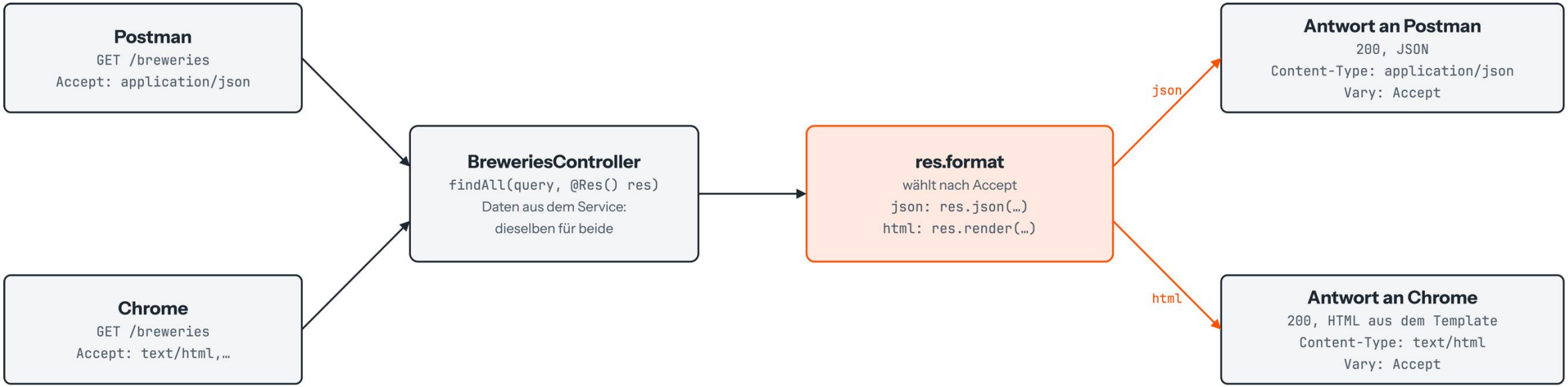
Templates mit Handlebars

Die Bausteine von Handlebars stehen auf Folie 16.6 der Vorlesung, das Maskieren auf Folie 16.7. Für die Aufgaben braucht es dazu:

- Der Name der Datei in views/ ohne .hbs ist der Name in res.render.
- Die Namen im Template passen zu den Namen im Objekt aus res.render.
- Daten aus einer Anfrage stehen immer in zwei Klammern.
- Drei Klammern nur für HTML, das der Server selbst erzeugt, wie {{{body}}} im Layout.
- Ohne NODE_ENV=production liest der Server die Templates bei jeder Anfrage; eine Änderung wirkt ohne Neustart.

Dokumentation: https://handlebarsjs.com/guide/

res.format ruft je nach Accept den Zweig für JSON oder für HTML auf



Unter einem Pfad liefert dieselbe Ressource zwei Repräsentationen. Service und Regeln bleiben gleich; neu ist nur die Form der Antwort.

Vary: Accept sagt jedem Cache, dass die Antwort von Accept abhängt. Er speichert JSON und HTML getrennt.

Postman sendet GET /breweries mit Accept: application/json, Chrome mit Accept: text/html,…. Der BreweriesController holt die Daten aus dem Service, für beide dieselben, und ruft res.format auf. Der Zweig json liefert Postman 200 mit JSON, der Zweig html liefert Chrome 200 mit HTML aus dem Template; beide Antworten tragen Vary: Accept. Unter einem Pfad liefert dieselbe Ressource zwei Repräsentationen; neu ist nur die Form der Antwort.

Ein Endpunkt liefert je nach Accept JSON oder HTML

Accept und Content-Type stehen in Skript 4.5 und 4.6, die Header dieser Einheit auf Folie 16.8 der Vorlesung. Mit @Res() bekommt die Methode das Response-Objekt von Express und sendet die Antwort selbst (Folie 16.10 der Vorlesung).

- res.format wählt den Eintrag mit der höchsten Gewichtung in Accept. Bei gleicher Gewichtung, etwa bei Accept: */*, gewinnt der erste.
- Passt kein Eintrag, antwortet Express mit 406 Not Acceptable.
- Pipes, Guards und Exceptions wirken wie bisher. Ein Fehler kommt als JSON, auch bei Accept: text/html.

Dokumentation: https://expressjs.com/en/5x/api.html#res.format

Beispiel: die Brauereien als HTML-Seite

Übernimm das Live-Coding in dein Projekt aus Einheit 15, bevor du mit den Aufgaben beginnst.

- Schritt 1 und 2: Installiere hbs und richte die Engine in main.ts ein (Skript 16.8).
- Schritt 3: Lege das Layout an (Skript 16.9).
- Schritt 4: Lege das Template für die Brauereien an (Skript 16.10).
- Schritt 5: Wähle in findAll mit res.format zwischen JSON und HTML (Skript 16.11).
- Vervollständige die Dateien und prüfe in Postman und Chrome (Skript 16.12).

Dokumentation für NestJS 11: https://github.com/nestjs/docs.nestjs.com/blob/11.0.0/content/techniques/mvc.md

Beispiel, Schritt 1 und 2: Paket installieren, Engine in main.ts einrichten

BASH

```
npm install hbs@4.4.0
```

Beispiel, Schritt 3: Das Layout rahmt jede Seite

HTML · VIEWS/LAYOUT.HBS

```
<!doctype html>
<html lang="de">
 <head>
  <meta charset="utf-8">
  <title>Beer App</title>
 </head>
 <body>
  {{! Hier setzt hbs das Template der Seite ein }}
  {{{body}}}
 </body>
</html>
```

Beispiel, Schritt 4: Das Template zeigt eine Zeile je Brauerei

HTML · VIEWS/BREWERIES.HBS

```
<h1>Brauereien</h1>
<table>
 <tr><th>Name</th><th>Stadt</th><th>Land</th></tr>
 {{! Eine Zeile je Brauerei aus der Liste breweries }}
 {{#each breweries}}
  <tr><td>{{name}}</td><td>{{city}}</td><td>{{country}}</td></tr>
 {{/each}}
</table>
```

Beispiel, Schritt 5: res.format wählt nach Accept zwischen JSON und Template

TYPESCRIPT · SRC/BREWERIES/BREWERIES.CONTROLLER.TS

```typescript
@Get()
// Ohne @ZodResponse: Mit @Res() sendet die Methode selbst
@ApiOkResponse({ type: [BreweryDto.Output] })
@ApiBadRequestResponse({ type: ErrorDto })
async findAll(
  @Query() query: BreweryQueryDto,
  // Response-Objekt von Express für res.format und res.render
  @Res() res: Response,
) {
  const breweries = await this.breweriesService.findAll(query);
  // Accept entscheidet: JSON wie bisher oder HTML aus dem Template
  res.format({
    json: () => res.json(breweries),
    html: () => res.render('breweries', { breweries }),
  });
}
```

Beispiel, Schritt 1 bis 5: Dateien vervollständigen und prüfen

- Schritt 2: Lege den Ordner views neben src und public an.
- Schritt 5: Res kommt aus @nestjs/common, ApiOkResponse aus @nestjs/swagger, Response mit import type aus express.
- Prüfe nach npm run db:reset: GET /breweries mit Accept: text/html ergibt 200 und eine Tabelle mit vier Brauereien.
- Dieselbe Anfrage mit Accept: application/json ergibt das JSON wie bisher. Beide Antworten tragen Vary: Accept.
- Öffne http://localhost:3000/breweries in Chrome. View Page Source zeigt das HTML mit <title>Beer App</title>.

Aufgabe 16.1 · Die Bierliste als HTML-Seite

ZIEL

GET /beers liefert dem Browser eine Tabelle der Biere und JSON-Clients das JSON wie bisher.

AUSGANGSLAGE

Dein Projekt mit dem Beispiel. Daten und Regeln in Skript 16.14, Anfragen in 16.15. Dokumentation: https://typeorm.io/docs/working-with-entity-manager/find-options/

ZIELE

- Ein Template zeigt je Bier Name, Stil, Name der Brauerei und Durchschnitt.
- Ohne Bewertung steht ein Strich statt einer leeren Zelle.
- Die Seite nutzt das Layout aus dem Beispiel.
- Das JSON behält genau die Felder von BeerDto.

FERTIG, WENN

- Accept: text/html zeigt nach den Anfragen die Tabelle aus Skript 16.15.
- Accept: application/json ergibt vier Biere ohne Feld brewery.

Daten und Regeln zu Aufgabe 16.1 und 16.2

Ein Bier hat die Felder id, name, style, abv, breweryId und description; averageRating wird bei jeder Antwort berechnet. Über die Relation brewery aus Einheit 12 erreichst du die Brauerei mit ihrem name. Eine Bewertung hat score von 1 bis 5, einen optionalen comment und createdAt, vom Server gesetzt.

- Die Durchschnittsbewertung eines Biers wird auf eine Nachkommastelle gerundet.
- Ohne Bewertung ist averageRating null.
- Ein User bewertet ein Bier höchstens einmal; eine zweite Bewertung ersetzt die erste.
- GET /beers und GET /beers/:id mit Accept: application/json antworten weiterhin mit JSON.
- GET /beers/42 mit Accept: text/html antwortet mit 404.

Anfragen zu Aufgabe 16.1

BEISPIEL

- Setze die Datenbank mit npm run db:reset zurück.
- Melde gast mit gast123 und admin mit admin123 an.
- Sende die zwei Bewertungen für das Nattheimer Spezial.
- Rufe GET /beers mit Accept: text/html ab.

```
POST /beers/1/ratings  gast
  { "score": 4, "comment": "Malzig und mild" }
POST /beers/1/ratings  admin
  { "score": 5, "comment": "Passt zum Schweinebraten" }
→ je 201

GET /beers, Accept: text/html → 200
Nattheimer Spezial · spezial
  Privatbrauerei Schlumberger · 4.5
Wildbräu Grafing Helles · helles
  Wildbräu Grafing · –
Augustiner Lagerbier Hell · helles
  Augustiner-Bräu · –
Farny Kristall-Weizen · weizen
  Edelweissbrauerei Farny · –
```

Aufgabe 16.2 · Die Detailseite eines Biers

ZIEL

GET /beers/:id liefert dem Browser eine Seite mit dem Bier und allen seinen Bewertungen.

AUSGANGSLAGE

Aufgabe 16.1 ist gelöst, die Bewertungen aus Skript 16.15 sind gespeichert. Anfragen in Skript 16.17. Dokumentation: https://handlebarsjs.com/guide/builtin-helpers.html

ZIELE

- Ein zweites Template zeigt Name, Stil, Alkoholgehalt und Durchschnitt des Biers.
- Darunter steht jede Bewertung mit Punkten, Kommentar und Datum.
- Ohne Bewertung zeigt die Tabelle die Zeile „Noch keine Bewertung“.
- Für ein unbekanntes Bier entsteht keine Seite.

FERTIG, WENN

- GET /beers/1 mit Accept: text/html zeigt das Nattheimer Spezial mit beiden Bewertungen.
- GET /beers/42 mit Accept: text/html ergibt 404.

Anfragen zu Aufgabe 16.2

BEISPIEL

- Sende die Anfragen nach denen aus Skript 16.15.
- GET /beers/1 mit Accept: text/html zeigt die Seite.
- Dieselbe Anfrage mit Accept: application/json ergibt das JSON wie bisher.
- GET /beers/2 zeigt ein Bier ohne Bewertung, GET /beers/42 ergibt 404.

```
GET /beers/1, Accept: text/html → 200
Nattheimer Spezial
Stil spezial, 5.4 % vol
Durchschnitt: 4.5
4 | Malzig und mild | 2026-10-25T…
5 | Passt zum Schweinebraten | 2026-10-25T…

GET /beers/2, Accept: text/html → 200
Wildbräu Grafing Helles · Durchschnitt: –
Noch keine Bewertung

GET /beers/42, Accept: text/html → 404
{"message":"Beer 42 not found",…}
```

Selbstkontrolle mit Postman

Schreibe die Tests wie seit Einheit 11 selbst (Skript 11.24). Hole die Tokens wie in Skript 13.32. Setze in jeder Anfrage den Header Accept. Ein Test auf HTML liest den Body mit pm.response.text() und prüft, ob ein Text darin vorkommt. Prüfe keine ganzen Zeilen HTML: Leerzeichen und Reihenfolge der Spalten sind Sache deiner Lösung. Setze die Datenbank vor jedem Lauf mit npm run db:reset zurück.

Der Ordner der Einheit 16 in der Collection Web-Programmierung zeigt zum Vergleich, welche Fälle eine vollständige Prüfung enthält, je Aufgabe in einem Ordner.

Dokumentation: https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

Ein Test für GET /breweries mit Accept: text/html

JAVASCRIPT

```
pm.test('Status 200', () => {
  pm.expect(pm.response.code).to.eql(200);
});
const weiter = pm.response.code === 200 ? pm.test : pm.test.skip;
weiter('Content-Type text/html', () => {
  const typ = pm.response.headers.get('Content-Type');
  pm.expect(typ, 'Liest der Controller Accept?').to.include('text/html');
});
weiter('Tabelle mit der Stadt München', () => {
  pm.expect(pm.response.text()).to.include('München');
});
```

Hinweise zu Aufgabe 16.1

- Stufe 1: Das Beispiel liefert die Brauereien als HTML. Vergleiche findAll im BreweriesController mit findAll im BeersController.
- Stufe 2: Ein Bier kennt nur breweryId. Welche Felder hat das JSON, sobald die Brauerei mitkommt und @ZodResponse fehlt?
- Stufe 3: Lade die Brauerei als Relation mit. Bringe die Liste im Zweig json wieder in die Form von BeerDto.

Hinweise zu Aufgabe 16.2

- Stufe 1: Welche Daten braucht die Detailseite, die GET /beers/:id bisher nicht liefert?
- Stufe 2: Welche Methode im BeersService liest die Bewertungen eines Biers? An welcher Stelle entsteht die 404?
- Stufe 3: Lade Bier und Bewertungen vor res.format. Übergib beide unter eigenen Namen an res.render.

Typische Fehler zu Aufgabe 16.1

Symptom	Ursache
Accept: text/html ergibt JSON.	Der Controller wertet Accept nicht aus; res.format fehlt.
500 „Internal server error“, im Terminal „Error: Failed to lookup view "beer-list" in views directory "…"“	Der Name in res.render passt zu keiner Datei in views/.
Die Spalte Brauerei ist leer.	findAll lädt die Relation zur Brauerei nicht.
Das JSON enthält "brewery":{"id":1,"name":"Privatbrauerei Schlumberger",…}.	Die Relation ist geladen, und kein Schema entfernt das Feld.
Die Zelle Durchschnitt ist bei drei Bieren leer.	Das Template behandelt den Wert null von averageRating nicht.

Typische Fehler zu Aufgabe 16.2

Symptom	Ursache
Build: „TS1241: Unable to resolve signature of method decorator when called as an expression.“	@ZodResponse steht noch an einer Methode mit @Res().
Accept: application/json ergibt HTML.	Die Methode ruft immer res.render auf.
Die Seite zeigt keine Bewertung, obwohl GET /beers/1/ratings zwei liefert.	res.render bekommt die Bewertungen nicht.
Die Überschrift ist leer: .	Der Name im Template passt nicht zum Namen, unter dem res.render das Bier übergibt.
Ein Kommentar "Süffig" erscheint fett statt als Text.	Das Template setzt den Kommentar mit drei Klammern ein; Cross-Site Scripting wie in der Fehlersuche der Vorlesung.

Das solltest du jetzt können

- ✓ Eine Template-Engine in NestJS einrichten
- ✓ Ein Template mit Platzhaltern, Schleife und Layout schreiben
- ✓ Einen Endpunkt je nach Accept JSON oder HTML liefern lassen
- ✓ Erklären, warum zwei Klammern maskieren und wann drei Klammern gefährlich sind
- ✓ HTML-Antworten in Postman mit eigenen Tests prüfen

Als Nächstes: Architektur-Rückblick und Ausblick – die Beer API als Ganzes, Kosten und technische Schulden.

WEB-PROGRAMMIERUNG · EINHEIT 17

Architektur-Rückblick und Ausblick · Skript

Worum es in dieser Einheit geht

Die Beer API ist fertig gebaut. Für die regionalen Brauereien beginnen jetzt die Kosten für Betrieb, Wartung und Weiterentwicklung. Diese Einheit benennt die Architektur, die im Semester entstanden ist, und zeigt, was echte Systeme ergänzen: Events, Queues, Microservices, Tests und Deployment. Die Einheit hat keine Story für die Beer API. In dieser Einheit schreibst du keinen Code. Du begründest Entscheidungen und entwirfst ein neues Projekt.

Am Ende der Einheit kannst du:

- die Architektur der Beer API benennen und Erweiterungen einordnen,
- für eine neue Aufgabe Stories, Datenmodell und Endpunkte entwerfen.

Kosten über die Nutzungsdauer

Die Kostenarten mit Beispielen aus der Beer API stehen auf Folie 17.4 der Vorlesung. Total Cost of Ownership zählt sie über die ganze Nutzungsdauer zusammen.

- Technische Schulden sind Abkürzungen beim Bau, die spätere Änderungen teurer machen.
- Martin Fowler nennt den Mehraufwand bei jeder neuen Funktion die Zinsen dieser Schulden.
- Beispiel: Bis Einheit 12 lagen die Daten in einem Array. Der Wechsel zur Datenbank betraf vor allem die Services.
- Die Zinsen waren klein, weil die Services das Array seit Einheit 6 kapselten.

Dokumentation: https://martinfowler.com/bliki/TechnicalDebt.html

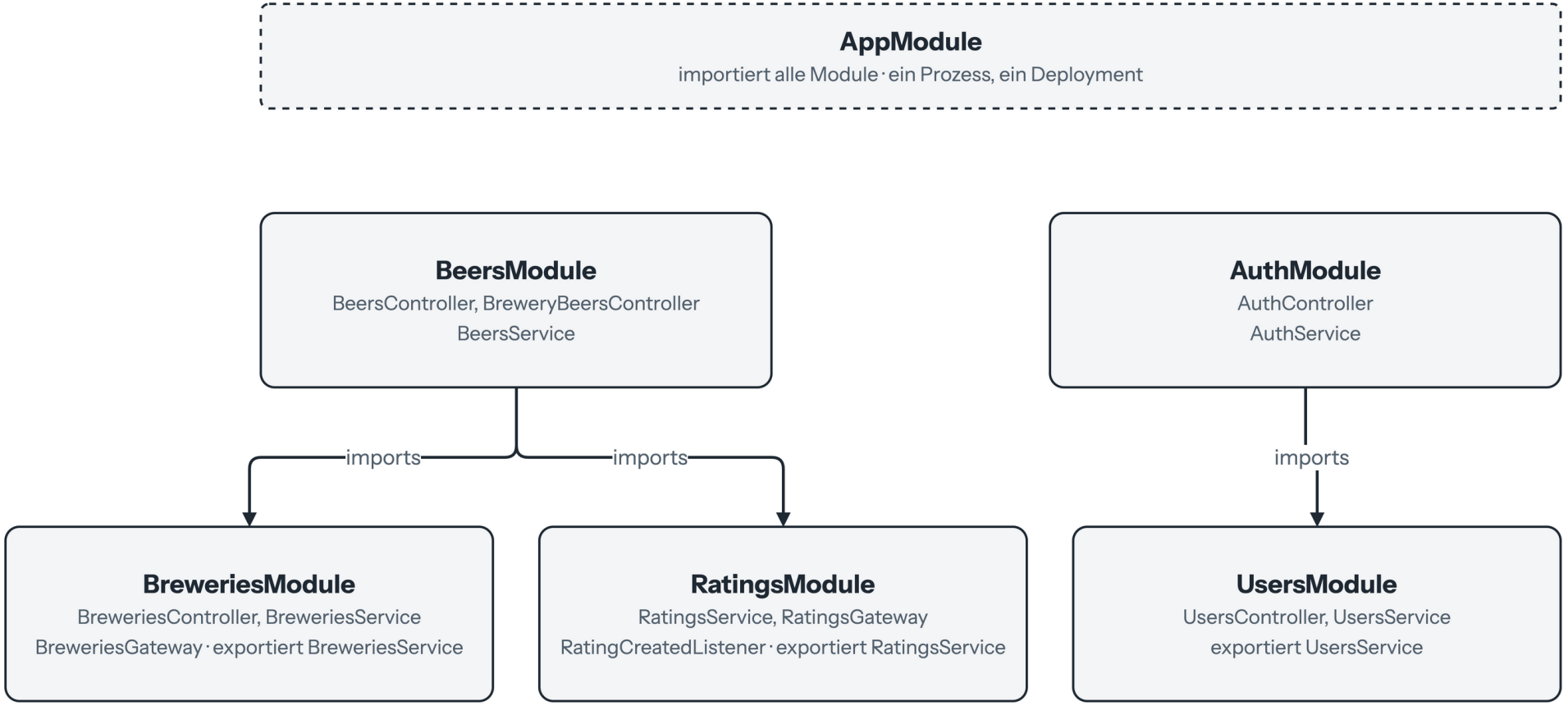
Make or Buy für die ganze Beer App

Make or Buy kennst du aus Einheit 2 an einzelnen Teilen. Folie 17.6 der Vorlesung zeigt vier Wege für die ganze Beer App. Vier Fragen helfen bei der Entscheidung:

- Unterscheidet der Teil den Betreiber von anderen? Die Regeln zu Bewertungen und Durchschnitt gehören dazu, eine Anmeldung nicht.
- Wer haftet bei einem Leak? Die Verantwortung für die Daten bleibt beim Betreiber, auch bei einem gekauften Dienst.
- Wie stark bindet der Teil an einen Anbieter: Preise, Abschaltung, Export der Daten?
- Was kostet es laufend je Monat, was einmalig mit Wartung?

Dokumentation: https://martinfowler.com/bliki/UtilityVsStrategicDichotomy.html

Die Beer API ist ein modularer Monolith: ein Prozess, fünf Module



Fünf Module der Beer API: BeersModule importiert BreweriesModule und RatingsModule, AuthModule importiert UsersModule; AppMod ule importiert alle; ein Prozess, ein Deployment

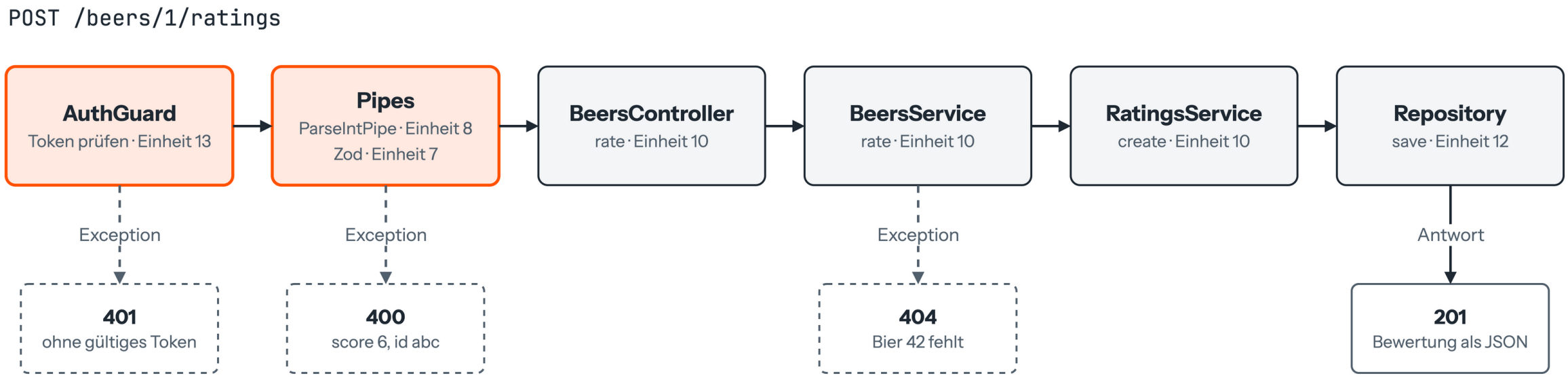
Module und Schichten

Imports und Exports von Modulen erklärt Skript 9.4, den Weg einer Anfrage durch NestJS Skript 5.6. Die drei Schichten stehen auf Folie 17.8 der Vorlesung.

- Monolith heißt: ein Prozess, ein Deployment. Modular heißt: innen Module je Ressource mit klaren Exporten.
- Aufrufe gehen von oben nach unten. Ausnahme: RatingsService ruft das Gateway auf; ein Event würde das entkoppeln (Folie 17.10).
- Ein Service darf einen anderen Service aufrufen: BeersService nutzt RatingsService für den Durchschnitt.
- Guard und Pipes prüfen vor dem Controller; wer zuerst ablehnt, bestimmt die Antwort (Skript 13.7).

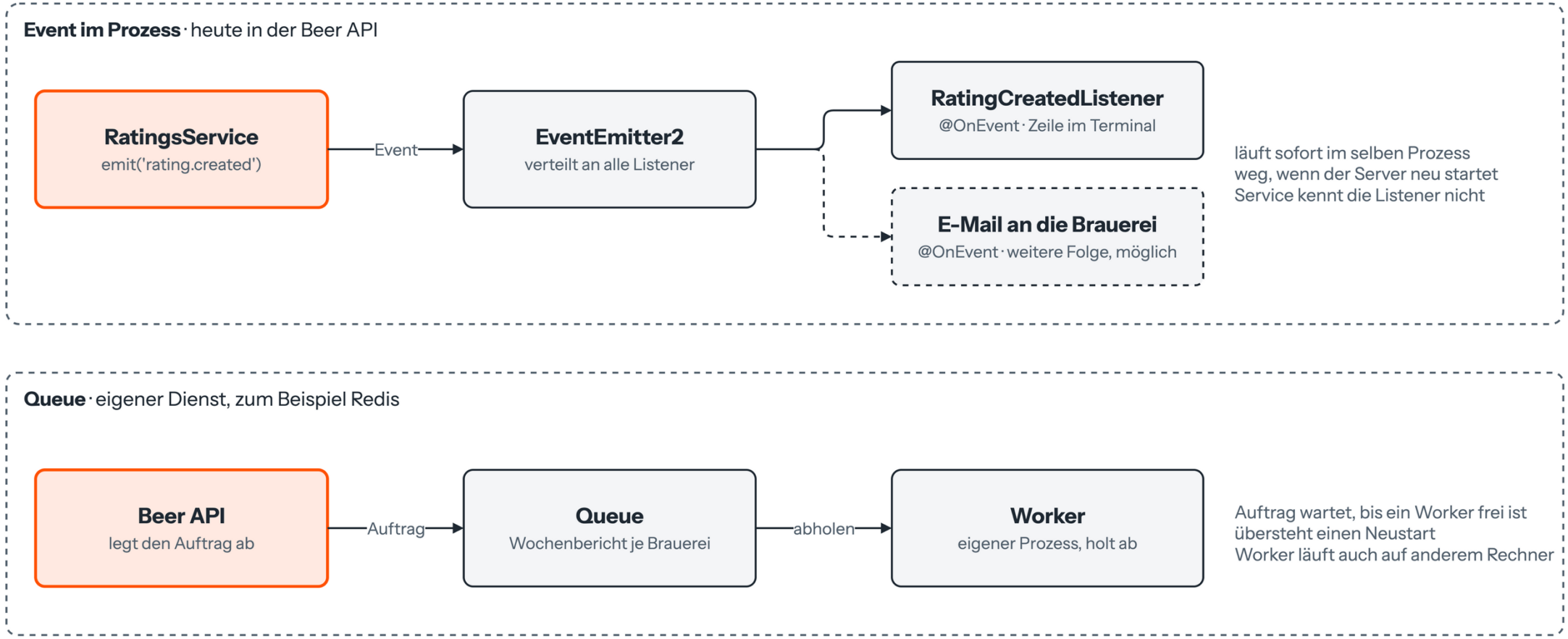
Dokumentation: https://docs.nestjs.com/modules

Guard und Pipes prüfen an der Grenze, bevor die Logik läuft



Weg von POST /beers/1/ratings: AuthGuard, Pipes, BeersController, BeersService, RatingsService, Repository; der AuthGuard bricht ohne gültiges Token mit 401 ab, die Pipes mit 400 bei score 6 oder id abc, der BeersService mit 404, wenn das Bier fehlt; sonst 201 mit der Bewertung als JSON

Ein Event entkoppelt die Folgen, eine Queue bewahrt Aufträge über einen Neustart



Oben Event im Prozess: RatingsService sendet rating.created an EventEmitter2, der an alle Listener verteilt: RatingCreatedListener schreibt eine Zeile ins Terminal, eine E-Mail an die Brauerei wäre eine weitere Folge. Unten Queue als eigener Dienst: Die Beer API legt einen Auftrag wie den Wochenbericht je Brauerei ab, ein Worker als eigener Prozess holt ihn ab, auch nach einem Neustart und auf einem anderen Rechner.

Events, Queues und Microservices

Monolith und Microservices vergleicht Folie 17.11 der Vorlesung.

- Ein Event nutzt du, wenn eine Folge sofort im selben Prozess laufen darf und ein Neustart sie verlieren darf.
- Eine Queue nutzt du, wenn ein Auftrag dauert, keine Anfrage blockieren und nicht verloren gehen darf.
- Microservices lohnen sich, wenn viele Teams unabhängig ausliefern. Jeder Aufruf zwischen zwei Diensten geht dann über das Netz.
- Ein modularer Monolith lässt sich später an den Grenzen seiner Module zerlegen.

Dokumentation: https://docs.nestjs.com/techniques/events, https://docs.nestjs.com/techniques/queues, https://martinfowler.com/bliki/MonolithFirst.html

Tests und Deployment

Unit-Test und e2e-Test vergleicht Folie 17.12 der Vorlesung, die Schritte des Deployments Folie 17.13.

- Ein Unit-Test ersetzt alles, was die Klasse per Dependency Injection bekommt, durch feste Werte (Mocks).
- Ein roter Unit-Test zeigt auf eine Regel. Ein e2e-Test zeigt, dass die Teile zusammen funktionieren.
- npm run start:dev übersetzt bei jeder Änderung neu und ist nur für die Entwicklung.
- Auf dem Server setzt die Umgebung die Secrets und NODE_ENV=production. Dann legt die Migration keine Beispiel-User an.

Dokumentation: https://docs.nestjs.com/fundamentals/testing, https://docs.nestjs.com/deployment

Beispiel: ein Event nach jeder Bewertung

BEISPIEL

- Code im Lösungsstand der Einheit 17, Ordner src/ratings
- rating-created.listener.ts reagiert mit @OnEvent
- ratings.service.ts sendet das Event nach dem Speichern
- POST /beers/1/ratings mit Token von gast ergibt 201
- Das Terminal von start:dev zeigt dazu die Zeile

```
LOG [RatingCreatedListener] Bier 1 bewertet mit 5
```

Beispiel: ein Unit-Test für den Durchschnitt

BEISPIEL

- Test in src/ratings/ratings.service.spec.ts
- Repository liefert die Bewertungen 4, 4 und 5
- Starte den Test mit npm test, ohne Server
- Ohne Rundung in roundedAverage wird er rot
- Die Warnung zu VM Modules ist erwartet

```
Tests:       1 passed, 1 total

Ohne Rundung:
  Expected: 4.3
  Received: 4.333333333333333
  ...
```

Vorgehen für ein eigenes Projekt

Die fünf Schritte stehen auf Folie 17.15 der Vorlesung. Aufgabe 17.2 übt die ersten drei:

- Eine Story hat die Form „Als Rolle möchte ich Funktion, damit Nutzen“.
- Wie du aus einer Regel Fälle ableitest, zeigen Skript 10.3 und 10.4.
- Jedes Akzeptanzkriterium ist eine Anfrage mit erwartetem Statuscode.
- Das JSON zeigt je Ressource ein vollständiges Beispiel mit id und Verweisen auf andere Ressourcen.
- Ein Wert, der sich aus anderen Daten ergibt, wird berechnet und nicht gespeichert, wie der Durchschnitt.
- Die Endpunkt-Tabelle nennt Methode, Pfad, Code bei Erfolg und Codes bei Fehlern.

Beispiel: die Bewertungen der Beer App in drei Schritten

Story Als Gast möchte ich ein Bier mit 1 bis 5 Sternen bewerten, damit andere Gäste sehen, was gut ankommt.

Kriterium POST /beers/42/ratings antwortet mit 404.

JSON { "id": 1, "beerId": 1, "userId": 2, "score": 5, "comment": null, "createdAt": "2026-10-25T…" }

Methode	Pfad	Erfolg	Fehler
POST	/beers/:id/ratings	201	400, 401, 404
GET	/beers/:id/ratings	200	400, 404

Aufgabe 17.1 · Eine Leitfrage zu viert begründen

ZIEL

Deine Gruppe beantwortet eine Leitfrage aus dem Semester mit drei begründeten Punkten.

AUSGANGSLAGE

Die Leitfragen stehen in Skript 17.16. Jede Gruppe bekommt eine Frage, die Begriffe stehen in Skript 17.3 und 17.4.

SCHRITTE

1. Sammelt Antworten zu eurer Leitfrage.
2. Wählt die drei Punkte mit der stärksten Begründung.
3. Schreibt jeden Punkt mit Begründung an die Tafel.

FERTIG, WENN

- An der Tafel stehen drei Punkte.
- Jeder Punkt nennt Kosten, Nutzen oder Risiko.

Leitfragen zu Aufgabe 17.1

Gruppe	Leitfrage	Ausgangspunkt im Semester
A	Was kostet 99,9 % Verfügbarkeit für die Beer API?	Einheit 1: rund 8 Stunden 46 Minuten Ausfall im Jahr; heute ein Prozess und eine Datei
B	Hat sich NestJS für die Beer API gelohnt?	Folie 5.7 und Folie 17.5 der Vorlesung
C	Was baut der Betreiber selbst, was kauft er?	Make or Buy aus Einheit 2, Folie 17.6 der Vorlesung

Aufgabe 17.2 · Die Lagerinventur entwerfen

ZIEL

Du entwirfst für einen neuen Auftrag die ersten drei Schritte eines Projekts.

AUSGANGSLAGE

Der Fall steht in Skript 17.18, die Vorlage für die Endpunkte in 17.19. Arbeite zu zweit.

ZIELE

- Du schreibst zwei Stories mit je zwei Akzeptanzkriterien.
- Du entwirfst ein JSON-Beispiel für zwei Ressourcen.
- Du füllst die Endpunkt-Tabelle für beide Stories.

FERTIG, WENN

- Jedes Kriterium ist eine Anfrage mit Statuscode.
- Jede Ressource hat id und Verweise; jeder Endpunkt hat Methode, Pfad, Erfolgs- und Fehlercode.

Der Fall Lagerinventur

Ein mittelständisches Unternehmen der Region braucht eine Software für die Lagerinventur.

- Im Backoffice verfolgt es über eine Webseite die aktuellen Lagerbestände.
- Unterschreitet ein Artikel seinen Mindestbestand, wird das Backoffice informiert.
- Lagerarbeiter scannen ein- und ausgehende Artikel mit Handscannern. Die Software der Handscanner ist vorhanden.
- Jeder Artikel trägt einen QR-Code mit den wichtigsten Informationen.
- Ein Artikel liegt in einem bestimmten Regal. Auch das Regal trägt einen QR-Code.

Deine Software ist der Server mit API. Die Webseite und die Software der Handscanner sind ihre Clients.

Vorlage: Endpunkte zu Aufgabe 17.2

Story	Methode	Pfad	Erfolg	Fehler
Beispiel Beer App: Bier bewerten	POST	/beers/:id/ratings	201	400, 401, 404
erste Story				
erste Story				
zweite Story				
zweite Story				

Zusatz 17.3 · Die Lagerinventur bauen oder kaufen

ZIEL

Du entscheidest für fünf Teile der Lagerinventur, ob das Unternehmen sie baut oder kauft. Der Zusatz ist keine Voraussetzung für Einheit 18.

AUSGANGSLAGE

Aufgabe 17.2 ist gelöst. Die Vorlage steht in Skript 17.21, die vier Fragen in Skript 17.4.

SCHRITTE

1. Prüfe jeden Teil der Vorlage mit den vier Fragen.
2. Trage Entscheidung und Begründung in die Tabelle ein.

FERTIG, WENN

- Jeder Teil hat eine Entscheidung.
- Jede Begründung nennt Kosten, Nutzen oder Risiko.

Vorlage zu Zusatz 17.3

Teil der Lagerinventur	bauen oder kaufen	Begründung
Anmeldung für Backoffice und Lagerarbeiter		
Speicher für Artikel, Regale und Buchungen		
Schnittstelle für die Handscanner		
Meldung bei Unterschreiten des Mindestbestands		
Webseite für das Backoffice		

Hinweise zu Aufgabe 17.1

- Stufe 1: Lest Folie 17.4 (A), 17.5 (B) oder 17.6 (C) der Vorlesung.
- Stufe 2: Ordnet jeden Punkt einer Kostenart von Folie 17.4 der Vorlesung zu.
- Stufe 3, A: Zählt auf, was der Betreiber kaufen oder mieten müsste, und wer nachts Bereitschaft hat.
- Stufe 3, B: Legt Folie 5.7 neben die Kosten auf Folie 17.5. Welche Risiken sind eingetreten?
- Stufe 3, C: Prüft jeden der vier Wege mit den vier Fragen aus Skript 17.4.

Hinweise zu Aufgabe 17.2

- Stufe 1: Sucht die Dinge, die das Unternehmen zählt, und die Ereignisse, die den Bestand ändern.
- Stufe 2: Eine Ressource ist ein Substantiv. Ein Scan ändert eine Ressource oder legt eine an.
- Stufe 2: Fragt bei jedem Feld: Wird es gespeichert oder aus anderen Daten berechnet?
- Stufe 3: Schreibt zuerst ein vollständiges JSON für einen Artikel und spielt damit einen Scan durch.
- Stufe 3: Leitet aus jedem Kriterium eine Zeile der Endpunkt-Tabelle ab, dann die Fehlercodes.

Hinweise zu Zusatz 17.3

- Stufe 1: Fragt bei jedem Teil, ob er das Unternehmen von anderen unterscheidet.
- Stufe 2: Achtet darauf, was schon gekauft ist und welche Daten das Unternehmen nicht abgeben will.
- Stufe 3: Schätzt für jeden Teil die laufenden Kosten gegen Bau und Wartung über fünf Jahre.

Typische Fehler

Symptom	Ursache
Ein Pfad enthält ein Verb, etwa /scanArticle	Der Scan ist als Aktion modelliert. Die Methode nennt die Aktion; der Pfad nennt die Ressource, die der Scan anlegt oder ändert.
Bestand am Artikel und Buchungen weichen voneinander ab	Derselbe Wert ist an zwei Stellen gespeichert. Eine Stelle muss die Quelle sein, die andere wird berechnet.
Ein Endpunkt wie POST /notifications ohne Client, der ihn aufruft	Die Meldung entsteht im Server, zum Beispiel durch ein Event oder eine Queue.
Ein Kriterium lautet „Der Scan funktioniert“	Das Kriterium nennt keine Anfrage und keinen Statuscode und lässt sich nicht prüfen.
Der Scan eines Regals findet seine Artikel nicht	Der Artikel nennt sein Regal nur als Text; ein Verweis über die id fehlt.

Das solltest du jetzt können

- ✓ Die Architektur der Beer API mit Modulen, Schichten und Grenzen benennen
- ✓ Den Weg einer Anfrage von Guard bis Repository beschreiben
- ✓ Event, Queue und Microservice abgrenzen, Unit- und e2e-Test unterscheiden
- ✓ Für eine neue Aufgabe Stories, JSON und Endpunkt-Tabelle entwerfen
- ✓ Eine Entscheidung Make or Buy mit Total Cost of Ownership begründen

Als Nächstes: Einheit 18 als Bonus – die Beer API bekommt ihren ersten Client in Angular.